

CPT102 Data Structures and Algorithms

数据结构和算法

内容基于LearningMall**的PPT。仅供学习和交流使用。任何机构或个人不得将其用于商业用途或未经授权的传播。如需引用或分享，请注明出处并保留此声明。



GITHUB

REPOSITORY



VISIT NOTES AUTHOR'S PAGE

CLICK ME

算法基础与问题求解 Algorithmic Foundations And Problem Solving

■ [Week 1]

Week 1 Lecture 1 + Week 1 Lecture 2 (Tutorial)

[0] 课程情况

教师信息

Teacher Name	Office Room	Email	Office Hour
Dr. Jia Wang (Module. Leader)	SD537	jia.wang02@xjtlu.edu.cn	14:00-16:00, Tuesday
Dr. Yushi Li	SD561	yushi.li@xjtlu.edu.cn	14:00-16:00, Tuesday
Dr. Pengfei Fan	SD559	pengfei.fan@xjtlu.edu.cn	14:00-16:00, Tuesday

学分组成

评估项	占比	其他
Assessment 1	10%	week 6 - week 7
Assessment 2	10%	week 12 – week 13
Final Examination	80%	Date TBA.

总内容

Methodology/Problems	Asymptotic Idea 渐进思想	Brute Force 蛮力法（暴力）	Divide & Conquer 分治法	Dynamic Programming（动态规划）	Greedy 贪心	Space/Time	Branch & Bound 分支界定法	Complexity Theory 复杂性理论
Efficiency 效率	Big-O							
Sorting 排序		Selection/Bubble/insertion	Mergesort		Count sorting			
Searching 搜索			Binary-searching					
String Matching 串匹配						Horspool algorithm		
Graph 图		DFS/BFS		Bellman-ford/Warshall/Assembly-line	MST/Dijkstra's/Shortest path		Traveling salesman, job assignment	
Complexity 复杂性								P/NP

程序 = 数据结构 + 算法

[1] 伪代码简述

伪代码（Pseudo）

```
1  p = 1 # 赋值
2  _____
3  for i = 1 to n do # 循环
4      p = p * x
5  _____
6  for i = 1 to n do # 循环 2
7      begin
8          statement
9      end
10 _____
11 output p # 输出
12 _____
13 if p < 0 then # if 语句
14     output -p
15 _____
16 while a > 10 do # while 语句
17     statement
```

```

18 | _____
19 | while condition do # while 语句2
20 | begin
21 |     a = ask for a number # 输入数字
22 | end
23 | _____
24 | repeat           # repeat语句
25 |     statement
26 | until a<0
27 | _____

```

小练习

1. Find the product of all integers in the interval $[x, y]$, assuming that x and y are both integers

寻找连续区间 $[x, y]$ 的乘积。例如 $[4, 10] = 4 \times 5 \times \dots \times 10$

```

1 | sum = 1
2 | for i = x to y do
3 | begin
4 |     sum *= i
5 | end
6 | # sum *= i 即 sum = sum * i; 同时begin和end可省去（无歧义）；

```

2. List all factors of a given positive integer x

列出所有 x 的因数

```

1 | for i = 1 to x do
2 |     if x % i == 0 then
3 |         output i

```

能否把他们转为 while 和 repeat 呢？尝试一下吧。

为什么我们需要更好的算法？ 接下来是一个直观的例子。

假设计算机的速度是：5 次操作/s

现在有一个算法的速度如下：比较 n 个数，需要 n^2 次操作

因此，处理 5 个数据需要 $5 \times 5 = 25$ 次操作，也就是 5 s

假设把计算机速度提高100倍，也就是 500 次操作 / s

如果不优化算法，在相同时间 5 s内，只能完成 $\sqrt{5 \times 500} = 50$ 个数据。

可见，即便把计算机速度提高 100 倍，如果不改变算法，也只能处理比原来多 10 倍的数据。

[2] 多项式

涉及 n 的幂的函数（例如， n , $n \log(n)$, n^2 ，称为**多项式函数**）与 n 作为指数的函数（例如， 2^n ，称为**指数函数**）之间存在巨大差异。简而言之就是 n 在上面还是下面差别很大。

在算法分析和时间复杂度中， $\log(n)$ 通常表示的是以 2 为底的对数，即 $\log_2(n)$ 。

多项式函数 polynomial functions

指数函数 exponential functions

对于多项式函数而言，齐次幂的更具有可比性。例如 $f_3(n) = n^2 - 3^n + 6$ 和 $f_4(n) = 2n^2$

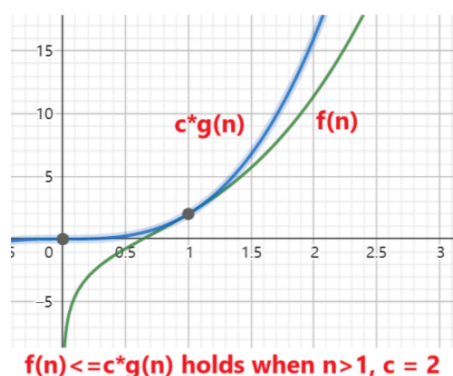
PPT习题

证明 $2n^2 + 4n$ 是 $O(n^2)$

- 因为对于所有 $n_0 > 4$, $c = 3$, $n \leq n^2$, 我们有 $2n^2 + 4n \leq 3n^2$ 对于所有 $n > n_0$
- 因此, 根据定义, $2n^2 + 4n$ 是 $O(n^2)$ 。

证明 $n^3 + n^2 \log n + n$ 是 $O(n^3)$

- 因为对于所有 $n_0 > 1$, $c = 2$, 我们有 $n^3 + n^2 \log(n) + n \leq 2n^3$, 对于所有 $n > n_0$
- 因此, 根据定义, $n^3 + n^2 \log n + n$ 是 $O(n^3)$ 。



这些证明展示了如何通过不等式分析, 将多项式表达式归约为主导项, 从而证明其渐近复杂度。

一、证明以下函数的数量级 (Prove the order of magnitude)

1. $n^3 + 3n^2 + 3$

该函数的数量级是 $O(n^3)$ 。证明如下:

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $n_0^3 + 3n_0^2 + 3 \leq 2 * n_0^3 \implies 3n_0^2 + 3 \leq n_0^3$ 。

可以取 $n_0 = 4$ 。因此, $\exists c = 2, n_0 = 4$ 使得 $n^3 + 3n^2 + 3 \leq n^3$ 对于所有 $n > n_0$ 成立。

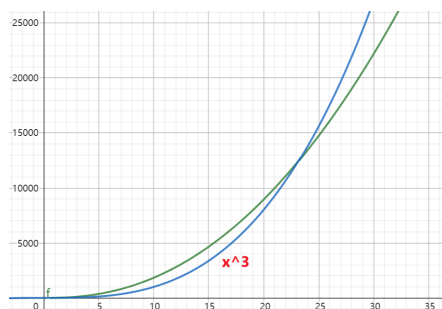
2. $4n^2 \log(n) + n^3 + 5n^2 + n$

该函数的数量级是 $O(n^3)$ 。证明如下:

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $4n_0^2 \log(n_0) + 5n_0^2 + n_0 \leq n_0^3$ 。如果以2为底, 大约取 $n_0 = 25$ 即可。

因此, $\exists c = 2, n_0 = 25$ 使得 $4n^2 \log(n) + n^3 + 5n^2 + n \leq 2 * n^3$ 对于所有 $n > n_0$ 成立



3. $2n^2 + n^2 \log(n)$

该函数的数量级是 $O(n^2 \log(n))$ 。证明如下：

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $2n_0^2 \leq n_0^2 \log(n_0) \implies n_0 \geq 4$ 。可选 $n_0 = 4$ 。

因此, $\exists c = 2, n_0 = 4$ 使得 $2n^2 + n^2 \log(n) \leq 2 * n^2 \log(n)$ 对于所有 $n > n_0$ 成立

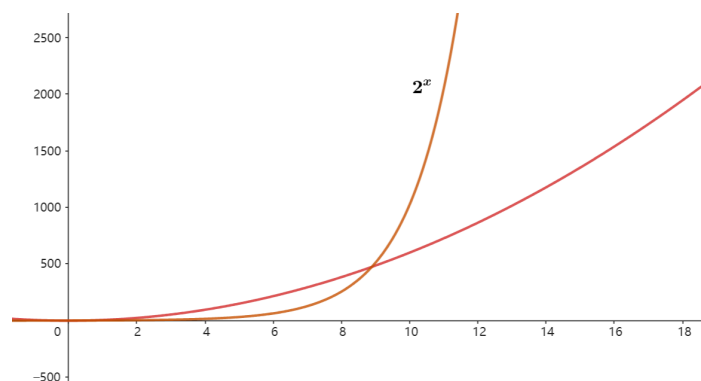
4. $6n^2 + 2^n$

该函数的数量级是 $O(2^n)$ 。证明如下：

假设 $c = 1 + 1 = 2$ 。

求解出 n_0 : $6n_0^2 \leq 2^{n_0}$, 可选 $n_0 = 10$ 。

因此 $\exists c = 2, n_0 = 10$ 使得 $6n^2 + 2^n \leq 2 * 2^n$ 对于所有 $n > n_0$ 成立

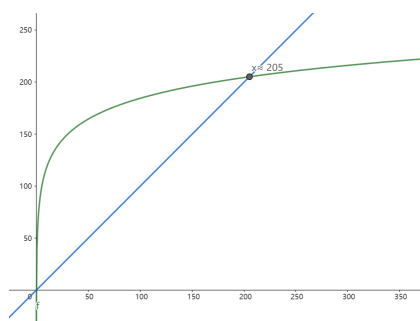


二、写出 $6n^{20} + 2^n$ 的数量级

$6n^{20} + 2^n$ 的数量级是 $O(2^n)$ 。为什么不是 n^{20} ?

不妨假设 $c = 1 + 1 = 2$ 。先求出 n_0 : $6n_0^{20} \leq 2^{n_0}$ 。如果说直接看的话, 就很难看出来结果。但是我们可以试着取对数:

$$\begin{aligned} 6n_0^{20} &\leq 2^{n_0} \\ 20 * \log(6n_0) &\leq n_0 \log(2) \\ 20 * \log(6n_0) &\leq n_0 \\ n_0 &\approx 205 \end{aligned}$$



也就是, $\exists c = 2, n_0 = 205$ 使得 $6n^{20} + 2^n \leq 2 * 2^n$ 对于所有 $n > n_0$ 成立。

■ [Week 2]

算法效率 + 搜索/排序

- ✓ 了解一些多项式时间和指数时间算法的例子
- ✓ 能够对算法进行简单的渐进分析
- ✓ 能够应用搜索/排序算法并推导其时间复杂度

[1] 线性搜索

```
1 public static int linearSearch(int[] array, int number)
2 {
3     for(int i =0;i<array.length;i++)
4         if(array[i]==number)
5             return i;          // 找到了, 返回index
6     return -1;                 // 没找到index, 返回-1
7 }
```

最好情况: $O(1)$, 最坏情况, $O(n)$ 。平均而言 $O(\frac{n}{2})$ 也就是 $O(n)$ 。

[2] 二分搜索

如果说数据有序（例如升序），那么我们就可以使用二分搜索的方法来减少时间复杂度。

递归 Recursive 方法：

```
1 public static int binarySearch(int[] array, int number){
2     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
3     else return binarySearch(array,number,0,array.length-1); // 开始二分搜索
4 }
5 private static int binarySearch(int[] array,int number, int left, int right){
6     if(left>right) return -1; // 如果左边大于右边, 直接返回-1。因为越界代表找不到
7     int middleValue = array[(left+right)/2]; // 定义中间值
8     if(number == middleValue) return (left+right)/2; // 如果相等就返回
9     if(number < middleValue) return binarySearch(array,number,left,(left+right)/2-1); // 如果number
    比中间值小, 就往左边找
10    return binarySearch(array,number,(left+right)/2+1,right); // 如果number比中间值大, 就往右边找
11 }
```

循环方法（更简单）：

```
1 public static int binarySearchLoop(int[] array, int number)
2 {
3     if(array == null || array.length==0) return -1; // 如果数组是空的, 返回-1
4     int left = 0;
5     int right = array.length-1;
6     while(left<=right)
7     {
8         int middleValue = array[(left+right)/2];
9         if(number > middleValue){ // 往右边找
10             left = (left+right)/2 +1;
11             continue;
12         }
13         if(number < middleValue){ // 往左边找
14             right = (left+right)/2-1;
15         }
16     }
17 }
```

```

15         continue;
16     }
17     if(number == middleValue)
18         return (left+right)/2;
19 }
20 return -1;
21 }

```

最好情况 $O(1)$ 。现在计算最坏情况。假设我们有 n 个数据。那么假设这个数据恰好要对比到最后一次，也就是需要比较 $\lceil \log_2(n) \rceil + 1$ 次。

如果一个数组 `arr = [0,1,2,3,4,5,6,7,8,9]`。我们想找到数字 `6`。

第一次: `middle = arr[(0+9)/2] = arr[4] = 4`。继续搜索下标 `5~9`。

第二次: `middle = arr[7] = 7`。继续搜索下标 `5~6`。

第三次: `middle = arr[5] = 5`。继续搜索下标 `6~6`。

第四次: 找到了 `6`。

一共计算了 $\log_2(10) + 1 = 4$ 次。

因此最坏的时间复杂度是 $O(\log_2(n) + 1)$ 。平均而言就是 $O((\log_2(n) + 2)/2) = O(\frac{1}{2}\log_2(n) + 1)$ 。最后，时间复杂度是 $O(\log_2(n))$

对比而言，二分搜索更有效率！

经过测试，如果用线性搜索 400,000,000 个数据，平均每次查找需要 81.34 ms。而使用二分搜索，平均每次查询只需要 5.578×10^{-4} ms。

[3] 字符串匹配

给定一个文本 `text` (String型)和一个模式 `pattern`。要求找到这个文本 `text` 内是否含有 `pattern`。

```

1 public static boolean exist(String text, String pattern)
2 {
3     if(text==null||pattern==null|| text.isEmpty() || pattern.isEmpty()) return false;
4     if(pattern.length()>text.length()) return false;
5
6
7     for(int i = 0;i<text.length();i++)                // 第一层循环
8     {
9         for(int j = 0; j<pattern.length();j++)        // 第二层循环
10        {
11            if(pattern.charAt(j)!=text.charAt(j+i)) break;
12            if(j==pattern.length()-1) return true;
13        }
14    }
15    return false;
16 }

```

假设 `text` 的长度是 n ，`pattern` 的长度是 m 。这里嵌套了两层循环。最好的情况是 `pattern` 就在开头，也就是 $O(m)$ 时间。

最坏的情况是找到末尾才结束。也就是大约遍历了 $m \times n$ 次，即 $O(nm)$ 。总时间复杂度就是 $O(nm)$ 。

[4] 选择排序

步骤如下：

原始数组 `originalArray= {5,1,3,2,4,0}`。

找出 `originalArray= {5,1,3,2,4,0}` 最小的，得到 0，放入新数组 `sortedArray = {0}`

找出 `originalArray= {5,1,3,2,4}` 最小的，得到 1，放入新数组 `sortedArray = {0,1}`

找出 `originalArray= {5,3,2,4}` 最小的，得到 2，放入新数组 `sortedArray = {0,1,2}`

...

这样就能排序好了。

实际上，并不需要创建两个数组。只需要进行**交换操作**即可

```
1 public static void selectionSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int i = 0; i<array.length-1;i++)
4     {
5         for(int j=i+1;j<array.length;j++)
6         {
7             if(array[j]<array[i])
8                 swap(array,i,j);
9         }
10    }
11 }
12 private static void swap(int[] array, int indexA, int indexB)
13 {
14     int temp = array[indexA];
15     array[indexA] = array[indexB];
16     array[indexB] = temp;
17 }
```

计算时间复杂度：

这个算法没有极端情况和最好情况。

当 `i = 0` 的时候，`j` 遍历 `n-1` 次。

当 `i = 1` 的时候，`j` 遍历 `n-2` 次。

...

因此总共需要的执行数是 $(n-1) + (n-2) + \dots + 1 = \frac{n \times (n-1)}{2}$

因此时间复杂度是 $O(n^2)$ 。

[5] 冒泡排序

冒泡排序的思路和选择排序差不多。可以看[这个视频](#)。

```

1 public static void bubbleSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int end = array.length-1; end>=0; end--){
4         {
5             for(int begin = 0; begin<end; begin++){
6                 {
7                     if(array[begin]>array[begin+1]) swap(array, begin, begin+1);
8                 }
9             }
10        }

```

同样的，它的时间复杂度仍然是 $O(n^2)$ 。

选择排序交换的是起始索引（一直往后移动）和动索引，界限是前面。而冒泡排序则是交换两个相邻索引，界限是在后面。

[6] 插入排序(Optional)

插入排序，可以看[这个视频](#)。

```

1 public static void insertSort(int[] array){
2     if(array == null || array.length==0) return;
3     for(int i = 1; i<array.length; i++){
4         {
5             int j = i-1;
6             while(j>=0 && array[j]>array[j+1])
7                 {
8                     swap(array, j, j+1);
9                     j--;
10                }
11        }
12    }

```

总比较数（最坏情况）： $1 + 2 + \dots + (n - 1) = \frac{n \times (n - 1)}{2}$ ，时间复杂度为 $O(n^2)$ 。

[7] 小结

选择排序、冒泡排序、插入排序：这三种算法在最坏情况下的时间复杂度均为 $O(n^2)$ 。

是否存在更高效的排序算法？**是的**，我们后续会学习它们。

基于比较的最快排序算法的时间复杂度是多少？ $O(n \log(n))$

一些**指数时间复杂度**的算法包括：

- **旅行商问题**（Traveling Salesman Problem）
- **背包问题**（Knapsack Problem）

旅行商问题

输入：共有 n 个城市。

输出：找到从某一特定城市出发的最短路径，要求恰好访问每个城市一次，并最终返回到起点城市。

这个问题被称为**哈密顿回路**（Hamiltonian Circuit）。

路径	计算方式	总距离
a -> b -> c -> d -> a	$2 + 8 + 1 + 7$	18

路径	计算方式	总距离
a -> b -> d -> c -> a	2 + 3 + 1 + 5	11
a -> c -> b -> d -> a	5 + 8 + 3 + 7	23
a -> c -> d -> b -> a	5 + 1 + 3 + 2	11
a -> d -> b -> c -> a	7 + 3 + 8 + 5	23
a -> d -> c -> b -> a	7 + 1 + 8 + 2	18

但是，随着城市的增加，可能会出现指数级别的爆炸。如果城市数量为 n ，那么需要考虑的数量是 $(n - 1)!$ 时间复杂度为 $O((n - 1)!)$ 。比如上图一共四个城市，总共可能的回路数量是 $(4 - 1)! = 6$

背包问题

输入：给定 n 个物品，每个物品有重量 $w_1, w_2, \dots, w_n$ 和价值 $v_1, v_2, \dots, v_n$ ，以及一个容量为 w 的背包。

输出：找到一个可以放入背包且总价值最大的物品子集。

应用：一架运输机需要将最有价值的物品集运送到一个偏远地点，同时不超出其容量限制。

如果有四个物品，也就是 $n = 4$ ，这种情况可能的数量是 $2^4 = 16$ 。时间复杂度： $O(2^n)$ ，因为这不是多项式时间，所以这个属于 NP-Hard

Q&A

假设你忘记了由5个字符组成的密码。你只记得：

- 这5个字符都是不同的。
- 这5个字符分别是 B、D、M、P、Y。

如果你想尝试所有可能的组合，总共有多少种？

5!

如果这5个字符可以是26个大写字母中的任意一个，那么总共有多少种？

26^5 （如果重复）
 A_{26}^5 （如果不重复）

假设密码还包含2个数字，即总共有7个字符：

- 所有字符都是不同的。
- 5个字母分别是 B、D、M、P、Y。
- 数字只能是 0 或 1。

总共有多少种组合？

$C_7^2 A_2^2 A_5^5$

假设密码的形式是 **adaaada**，其中：

- a** 表示字母，**d** 表示数字。
- 所有字符都是不同的。
- 5个字母分别是 B、D、M、P、Y。

- 数字只能是 0 或 1。

总共有多少种组合？

$$A_2^2 A_5^5$$

[8] Tutorial

1. Give the trace table and the output of the following algorithm for $m = 16$.

```

1  input m
2  count = 0
3  x = 1
4  while x < m do
5      begin
6          x = x * 2
7          count = count + 1
8      end
9  output count

```

What is the output for **general** m that is a positive power of 2 (i.e., $m = 2, 4, 8, 16, 32, \dots$)?

$$\log_2(m) = \log_2(16) = 4$$

2. Rewrite the following for-loop into a while loop.

```

1  input m
2  x = 0
3  for i = 1 to m do
4      begin
5          x = x + i
6      end
7  output x

```

如果 $m=2$ 这里的 for 就循环2次，等价于 `for(i=1;i<=2;i++)`

```

1  input m
2  x = 0
3  while m>0 do
4      begin
5          x = x + i
6          m--;
7      end
8  output x

```

3. Rewrite the above for-loop into a repeat loop.

```

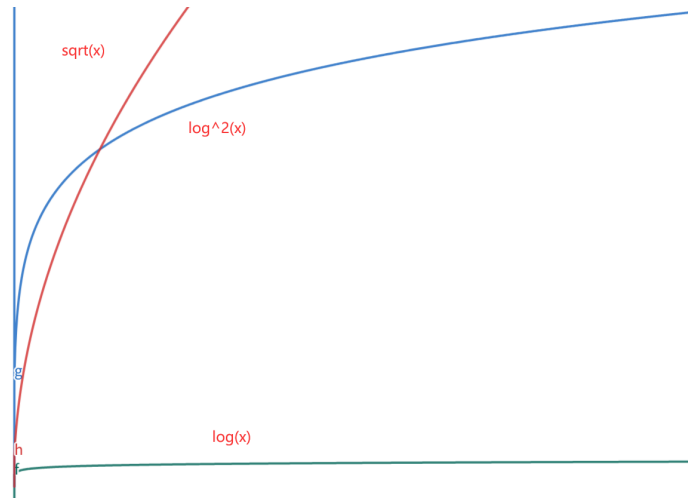
1  input m
2  x = 0
3  repeat
4      x = x + i;
5      m--;
6  until m = 0

```

4. List the following functions from lowest order to highest order of magnitude:

将以下函数从最低阶到最高阶的量级进行排序：

$O(1)$ 、 $O(\log(n))$ 、 $O(\log^2(n))$ 、 $O(\sqrt{n})$ 、 $O(n)$ 、 $O(n\log(n))$ 、 $O(n^{\frac{3}{2}})$ 、 $O(n^2)$ 、 $O(n^2\log^4(n))$ 、 $O(n^3)$ 、 $O(n^{\frac{10}{3}})$ 、 $O(n^4)$

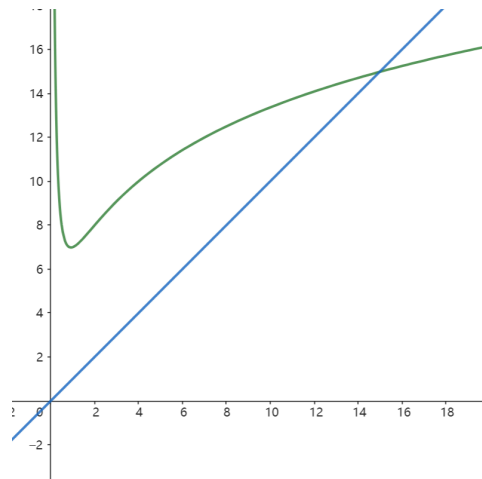


不管 $\log$ 多少次方，都比 $x^{0.5}$ 也就是 $\sqrt{x}$ 数量级小，后者始终超越前者

5. Prove that the function $f(n) = 3n^2\log n + 2n^3 + 3n^2 + 4n$ is $O(n^3)$.

令 $c = 2 + 1 = 3$ 。解不等式 $3n_0^2\log n_0 + 2n_0^3 + 3n_0^2 + 4n_0 < 3n_0^3$ 得到 $n_0 \approx 15$ 取 $n_0 = 16$

所以 $\exists c = 3, n_0 = 16$ 使得 $f(n) \leq c \cdot g(n), g(n) = n^3$ 对于所有的 $n > n_0$ 成立



6. Consider a string T of n characters $T[0..(n-1)]$. Design and write a pseudo code of an algorithm to determine if a given character, denoted by C , appears uniquely in $T[0..(n-1)]$, i.e., whether the character C appears exactly once in T .

For example, if T is "Hello, how are you?" and C is **H**, then the algorithm should report "Yes, the character appears uniquely.". However, if C is **e**, then the algorithm should report "No, the character does not appear uniquely."

考虑一个由 n 个字符组成的字符串 T ，其索引范围为 $T[0..(n-1)]$ 。设计并编写一个伪代码算法，用于判断给定的字符 C 是否在 $T[0..(n-1)]$ 中唯一出现，即字符 C 是否在 T 中恰好出现一次。

例如，如果 T 是 "Hello, how are you?"，且字符 C 是 H，那么算法应报告 "Yes, the character appears uniquely." ("是的，该字符唯一出现。")。然而，如果 C 是 e，那么算法应报告 "No, the character does not appear uniquely." ("不，该字符并未唯一出现。")。

```
1 public static void isAppearAtOnce(String text, String pattern){
2     if(countAppearTime(text,pattern)>1)
3         System.out.println("No, the character does not appear uniquely.");
4     else
5         System.out.println("Yes, the character appears uniquely.");
6 }
7 private static int countAppearTime(String text,String pattern){
```

```

8  if(text==null||pattern==null|| text.isEmpty() || pattern.isEmpty()) return 0;
9  if(pattern.length()>text.length()) return 0;
10 int count = 0;
11 for(int i = 0;i<text.length();i++)
12 {
13     for(int j = 0; j<pattern.length();j++)
14     {
15         if(pattern.charAt(j)!=text.charAt(j+i)) break;
16         if(j==pattern.length()-1) count++;
17     }
18 }
19 return count;
20 }

```

Pesudo Code

```

1  input T
2  input C
3  count = 0
4  for i=0 to T.length do:
5  begin
6      if T[i] == C do:
7          count++
8  end
9  if count != 1
10     print("No, the character does not appear uniquely.")
11 else print("Yes, the character appears uniquely.")

```

■ [Week 3]

分治 (Divide and Conquer) 算法

本周有两个Lecture无Tutorial，均包含在本节

学习目标

- 理解分治法的工作原理，并能够通过求解递推关系来分析分治法的时间复杂度。
- 查看分治法的示例。

[1] 关于分治

分治法是最著名的算法设计技术之一。

核心思想：

1. 将一个问题的实例划分为多个相同问题的较小实例，理想情况下这些实例的规模大致相同。
2. 递归地解决这些较小的实例。
3. 将较小实例的解合并，得到原问题的解。

回顾二分搜索

回顾我们之前学过的二分搜索/查找：

输入： 一个包含 n 个已排序数字的序列 $a_0, a_1, \dots, a_{n-1}$ ；以及一个数字 X 。

算法思想：

1. 将 X 与中间位置的数字进行比较。
2. 根据 X 是小于还是大于中间数字，将搜索范围缩小到前半部分或后半部分。
3. 将需要搜索的数字数量减少一半。

我们用了 递归 Recursive 的方法，也用了一般while循环的方法，都可以实现。

时间复杂度

设 $T(n)$ 表示二分查找算法在 n 个数字上的时间复杂度

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\frac{n}{2}) + 1 & \text{otherwise} \end{cases}$$

我们称这个公式为一个递推关系

二分搜索的时间复杂度

假设 $T(n) \leq 2 \log n$

递推关系为：

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(\frac{n}{2}) + 1 & \text{otherwise} \end{cases}$$

验证基本情况：

猜： $T(n) \leq 2 \log n$

$$T(n) \leq 2 \log n$$

其中L.H.S (Left Hand Side, 左边) 为 $T(n)$

R.H.S 为 $2 \log n$

代入法 (Substitution method)

当 $n = 1$ 时, 假设不成立!

$$\text{左边 (L.H.S)} = T(1) = 1$$

$$\text{右边 (R.H.S)} = c \log 1 = 0$$

显然, $L.H.S > R.H.S$

然而, 当 $n = 2$ 时:

$$\text{左边 (L.H.S)} = T(2) = T(1) + 1 = 2$$

$$\text{右边 (R.H.S)} = 2 \log 2 = 2$$

此时, $L.H.S \leq R.H.S$ 成立。

具体而言怎么操作呢?

$$T(n) = T\left(\frac{n}{2}\right) + 1 \leq 2 \log\left(\frac{n}{2}\right) + 1 = 2(\log n - 1) + 1 = 2 \log n - 1$$

$\uparrow$ 带入 $T(n) = 2 \log n$

$$T(n) \leq 2 \log n - 1$$

相当于我们把recursive的部分当作 $\log$ 处理。

那么这个时间复杂度就是 $O(2 \log n - 1) = O(\log n)$

练习

Q1

求如下算法时间复杂度

$$T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + 1 & \text{otherwise} \end{cases}$$

在这里, 我们先试着枚举一下:

$$T(1) = 1$$

$$T(2) = 2T(1) + 1 = 3$$

$$T(3) = 2T(1) + 1 = 3 \text{ (Rounding, } 3 \div 2 = 1)$$

$$T(4) = 2T(2) + 1 = 7$$

$$T(5) = 2T(2) + 1 = 7 \text{ (Rounding)}$$

$$T(6) = 2T(3) + 1 = 7$$

$$T(7) = 2T(3) + 1 = 7 \text{ (Rounding)}$$

$$T(8) = 2T(4) + 1 = 15$$

可以看到, 数字 2, 4, 6, 8 的规律是: $2n - 1$ 。那么就假设这个时间复杂度满足

$$T(n) \leq 2n - 1$$

证明如下：

$$T(n) = 2T\left(\frac{n}{2}\right) + 1 \leq 2\left(2\left(\frac{n}{2}\right) - 1\right)$$

↑这里要证明 $T(n) \leq 2n - 1$ ，那么就直接带入 $T(n) = 2n - 1$
如果最后化简出来的能满足 $\leq 2n - 1$ 就可以证明了
 $= 2n - 1$

显然是满足的

因此时间复杂度 $O(2n - 1) = O(n)$

PPT介绍了猜测的方法，但是我认为这样比较难猜

一般而言，我们可以采用以下方法：考虑 $n > 1$ 的情况

$$T(n) = 2T\left(\frac{n}{2}\right) + 1 \quad \text{接下来展开 } T\left(\frac{n}{2}\right)。 \text{展开第1次}$$

$$= 2\left(2T\left(\frac{n}{4}\right) + 1\right) + 1 = 4T\left(\frac{n}{4}\right) + 3 \quad \text{展开 } T\left(\frac{n}{4}\right)。 \text{展开第2次}$$

$$= 4\left(2T\left(\frac{n}{8}\right) + 1\right) + 3 = 8T\left(\frac{n}{8}\right) + 7 \quad \text{展开第3次}$$

$$= 16T\left(\frac{n}{16}\right) + 15 \quad \text{展开第4次}$$

$$= \dots \quad \text{因为是2的指数倍，所以一共展开了 } \log_2 n \text{ 次，因此}$$

$$= 2^{\log_2 n} T(1) + 2^{\log_2 n} - 1 = nT(1) + n - 1 = 2n - 1$$

因此时间复杂度 $O(2n - 1) = O(n)$

Q2

求如下算法时间复杂度

$$T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + n & \text{otherwise} \end{cases}$$

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$= 4T\left(\frac{n}{4}\right) + 2n$$

$$= 8T\left(\frac{n}{8}\right) + 3n$$

$$= 16T\left(\frac{n}{16}\right) + 4n$$

$$= \dots$$

$$= 2^{\log_2 n} T(1) + n \log_2 n$$

$$= n + n \log_2 n$$

时间复杂度: $O(n + n \log n) = O(n \log n)$

使用猜测的方法也有一些技巧, 如下:

根据递推关系, 我们可以猜测其数量级:

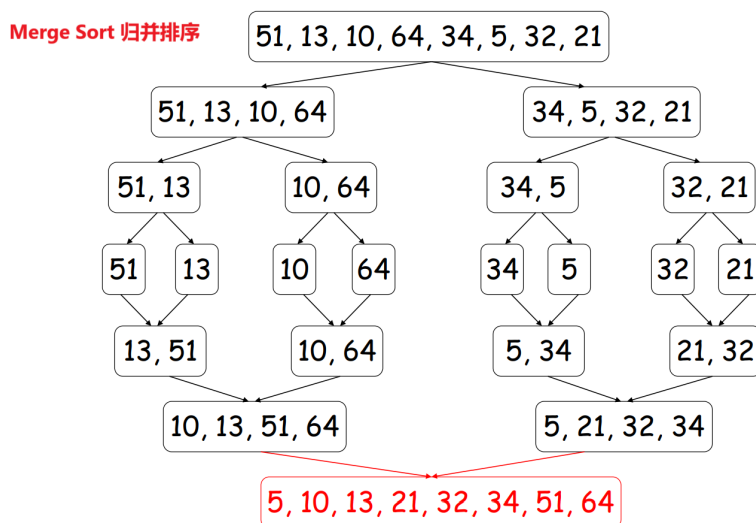
- $T(n) = T\left(\frac{n}{2}\right) + 1$, $T(n)$ 是 $O(\log n)$
- $T(n) = 2T\left(\frac{n}{2}\right) + 1$, $T(n)$ 是 $O(n)$
- $T(n) = 2T\left(\frac{n}{2}\right) + n$, $T(n)$ 是 $O(n \log n)$

[2] 归并排序

Merge Sort, 归并排序是分支算法的一个很经典的例子!

归并排序 (Merge Sort) 是一种基于分治法 (Divide and Conquer) 的排序算法。

其主要思想是将待排序的序列不断分成两半, 递归地对每一半进行排序, 然后将两个已排序的子序列合并成一个完整的有序序列。归并排序的时间复杂度为 $O(n \log n)$, 是一种稳定的排序算法。



```
1 public static void mergeSort(int[] arr) {
2     if (arr == null || arr.length < 2) return;
3     mergeSort(arr, 0, arr.length - 1);
4 }
5
6 private static void mergeSort(int[] array, int left, int right) {
7     // 这里左右都要分, 分到最小单元之后merge。
8     if (left < right) {
9         int middle = left + (right - left) / 2;
10        mergeSort(array, left, middle);
11        mergeSort(array, middle + 1, right);
12        merge(array, left, middle, right);
13    }
14 }
15
16 private static void merge(int[] array, int left, int middle, int right) {
17     // 第一个数组: left 到 middle
18     // 第二个数组: middle+1 到 right
19     // 现在选出两个数组最小的放在temp中
20     int[] temp = new int[right - left + 1];
```

```

21     int indexOfTemp = -1;
22     int firstArrayIndex = left ;
23     int secondArrayIndex = middle+1;
24     while (firstArrayIndex <= middle && secondArrayIndex <= right)
25         if (array[firstArrayIndex] < array[secondArrayIndex])
26             temp[++indexOfTemp] = array[firstArrayIndex++];
27         else
28             temp[++indexOfTemp] = array[secondArrayIndex++];
29
30     // 复制剩余数组到 temp
31     if (firstArrayIndex == middle+1) // 如果first到末尾了
32         while (secondArrayIndex <= right) temp[++indexOfTemp] = array[secondArrayIndex++];
33     else while (firstArrayIndex <= middle) temp[++indexOfTemp] = array[firstArrayIndex++];
34
35     // temp替换原来数组
36     indexOfTemp = -1;
37     while (++indexOfTemp < temp.length)
38         array[left + indexOfTemp] = temp[indexOfTemp];
39 }
40
41 public static void main(String[] args) {
42     int[] arr = {51, 13, 10, 64, 34, 5, 32, 21};
43     mergeSort(arr);
44     System.out.println(Arrays.toString(arr));
45 }

```

分析时间复杂度：

对于含有 n 个元素的数组，先拆分为 n 个数组，然后：

归并1次：得到 $\frac{n}{2}$ 个数组

归并2次：得到 $\frac{n}{4}$ 个数组

归并3次：得到 $\frac{n}{8}$ 个数组

...

一共需要归并 $\log_2 n$ 次。然而每次归并都需要遍历 n 次，因此时间复杂度是 $O(n \log n)$

归并排序的时间复杂度递推公式 $T(n) = 2T\left(\frac{n}{2}\right) + n$ 的推导如下：

1. Divide (分) :

将长度为 n 的序列分成两部分，每部分的长度为 $\frac{n}{2}$ ，即需要递归地对两个子序列进行排序。因此， $T(n)$ 包含了两次对长度为 $\frac{n}{2}$ 的序列的排序，即 $2T\left(\frac{n}{2}\right)$ 。

2. Conquer (治) :

合并两个已排序的子序列时，需要线性时间 $O(n)$ ，因为每个元素最多被比较和移动一次。因此， $T(n)$ 还包含一个线性项 n 。

3. Base Case (基本情况) :

当 $n = 1$ 时，不需要任何操作，因此 $T(1) = 1$ 。

综上所述，归并排序的时间复杂度递推公式为：

$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + n & \text{otherwise} \end{cases}$$

[3] 关于图

学习目标

✓ 能够区分什么是无向图 (Undirected Graph)，什么是有向图 (Directed Graph)

掌握如何使用矩阵 (Matrix) 和邻接表 (List) 表示图

✓ 理解欧拉路径 (Euler Path) / 欧拉回路 (Euler Circuit)，并能够判断在一个无向图中是否存在这样的路径 / 回路

✓ 能够应用广度优先搜索 (BFS) 和深度优先搜索 (DFS) 遍历图

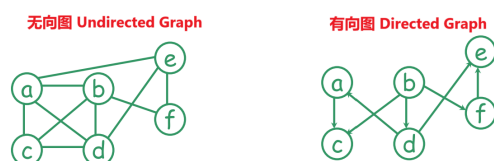
✓ 能够描述什么是树 (Tree)

图论：一个历史悠久但具有许多现代应用的主题。

起源于 18 世纪。

无向图 $G = (V, E)$ 由一组顶点 V 和一组边 E 组成。每条边是一个顶点的无序对。(例如， $\{b, c\}$ 和 $\{c, b\}$ 表示同一条边。)

有向图 $G = (V, E)$ 由一组顶点 V 和一组边 E 组成。每条边是一个顶点的有序对。(例如， (b, c) 表示一条从 b 到 c 的边。)



图的应用

在计算机科学中，图通常用于建模：

- 计算机网络，
- 进程之间的优先关系，
- 下棋的状态空间（人工智能应用），
- 资源冲突等。

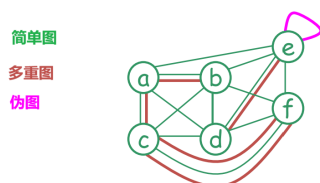
在其他学科中，图也用于对对象的结构进行建模。例如：

- 生物学——进化关系，
- 化学——分子结构。

无向图

无向图分为以下几种：

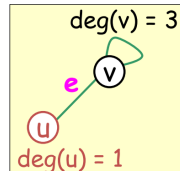
- **简单图 (Simple Graph)**：两个顶点之间最多有一条边，不存在自环（即从一个顶点到自身的边）。
- **多重图 (Multigraph)**：允许两个顶点之间存在多条边。
- **伪图 (Pseudograph)**：允许存在自环。



提示：一个无向图 $G = (V, E)$ 由一组顶点 V 和一组边 E 组成。每条边是一个顶点的无序对。

在无向图 G 中, 假设 $e = \{u, v\}$ 是 G 的一条边:

- u 和 v 被称为**相邻 (adjacent)**, 并且互为**邻居 (neighbors)**。
- u 和 v 被称为边 e 的**端点 (endpoints)**。
- 边 e 被称为**关联 (incident)** 于顶点 u 和 v 。
- 边 e 被称为**连接 (connect)** 顶点 u 和 v 。
- 顶点 v 的**度 (degree)**, 记作 $\deg(v)$, 是与其关联的边的数量 (自环对度的贡献为 2 次)。



无向图的表示

无向图可以通过以下方式表示:

- 邻接矩阵 (Adjacency Matrix)
- 邻接表 (Adjacency List)
- 关联矩阵 (Incidence Matrix)
- 关联表 (Incidence List)

邻接矩阵和**邻接表**记录顶点之间的邻接关系, 即一个顶点与哪些其他顶点相邻。

关联矩阵和**关联表**记录边与顶点之间的关联关系, 即一条边关联于哪两个顶点。

邻接矩阵 / 邻接表

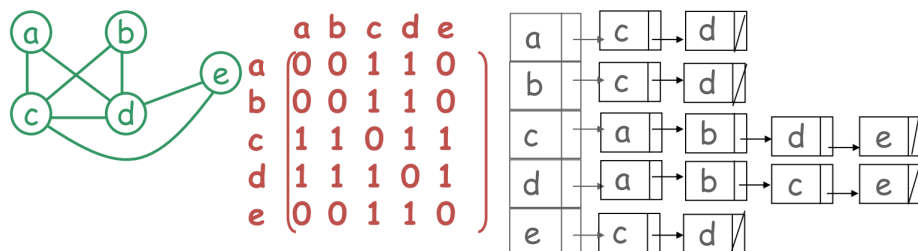
Adjacency Matrix / Adjacency List

是**点和点的关系**

对于一个具有 n 个顶点的**简单无向图**, 其**邻接矩阵** M 定义如下:

- M 是一个 $n \times n$ 的矩阵
- 如果顶点 i 和顶点 j 相邻, 则 $M(i, j) = 1$, 否则 $M(i, j) = 0$

邻接表: 每个顶点都有一个列表, 记录与之相邻的所有顶点。



关联矩阵 / 关联表

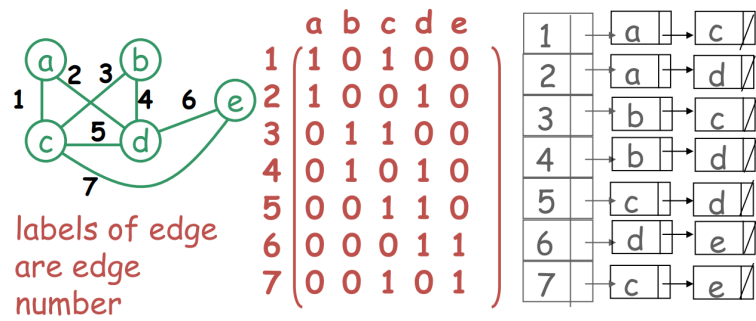
Incidence Matrix / Incidence List

是点和边的关系

对于一个具有 n 个顶点和 m 条边的简单无向图，其关联矩阵 M 定义如下：

- M 是一个 $m \times n$ 的矩阵
- 如果边 i 和顶点 j 相关联，则 $M(i, j) = 1$ ，否则 $M(i, j) = 0$

关联表：每条边都有一个列表，记录与之关联的所有顶点。

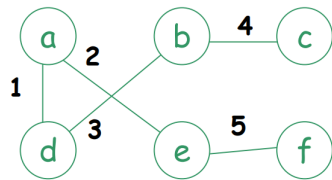


主要是记住英文，Adjancy(记为Adj)是邻接，Incidence是关联。

临界的是点对点，而关联的是边对点。

练习

给定以下图，请写出其邻接矩阵和关联矩阵。



邻接矩阵：

	a	b	c	d	e	f
a				1	1	
b			1	1		
c		1				
d	1	1				
e	1					1
f					1	

空白处填0

关联矩阵：

	a	b	c	d	e	f
1	1			1		
2	1				1	
3		1		1		
4		1	1			
5					1	1

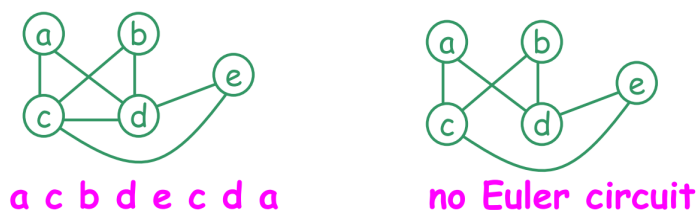
空白处填0

欧拉回路、欧拉路径

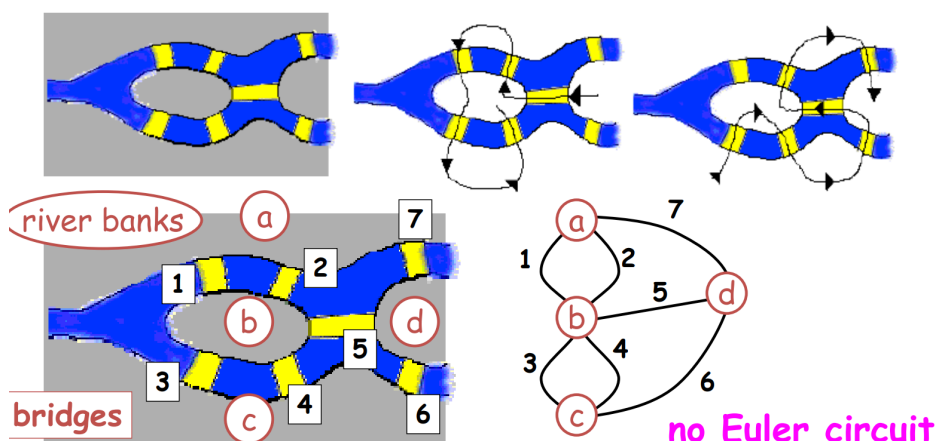
- 在无向图中，从顶点 u 到顶点 v 的路径 Path 是一系列边 $e_1 = \{u, x_1\}, e_2 = \{x_1, x_2\}, \dots, e_n = \{x_{n-1}, v\}$ ，其中 $n \geq 1$ 。
- 该路径的长度为 n 。
- 注意，从 u 到 v 的路径意味着从 v 到 u 的路径。
- 如果 $u = v$ ，则该路径称为回路（环） Circuit。

欧拉回路 Euler Circuit

一个简单的回路必须要访问（且仅访问1次）每一条边
（注意：顶点可以被重复访问）
每个图都有欧拉回路吗？显然不是



历史背景：在德国柯尼斯堡，一条河流穿过城市，并建造了七座桥梁。市民们想知道是否有一种方式可以让人们环游城市，同时恰好每座桥只经过一次。可惜，并没有这种路径



靠直接判断肯定行不通，但是有什么办法能判断这个图有无欧拉回路呢？

一个无向图 G 被称为连通的(connected)，如果每一对顶点之间都存在一条路径。

不连通固然没有欧拉回路。但是即使图是连通的，也可能不存在欧拉回路。

设 G 为连通图。 引理： G 包含一条欧拉回路当且仅当每个顶点的度数都是偶数

欧拉路径 Eula Path

简而言之，欧拉路径是一个简单的路径：必须要访问（且仅访问1次）每一条边，但是不用回到起始点

（注意：顶点可以被重复访问）

定理：一个无向图包含一条欧拉路径当且仅当它是连通的，并且恰好有两个顶点的度数是奇数。

简而言之，这个图有欧拉路径，必须要满足

- 1. 每一对顶点之间都存在一条路径，也就是连通的
- 2. 恰好有2个顶点的度是奇数或者没有（如果没有，那就是全偶数度数，变成欧拉回路了！）

	存在欧拉回路？	存在欧拉路径？
所有顶点的度数均为偶数	是	是
恰好两个顶点的度数为奇数	否	是
多于两个顶点的度数为奇数	否	否

所以实际上欧拉路径包含了欧拉回路，欧拉回路条件更苛刻：要回到起始点。

所以这两个点也很好记住。不过要注意他们的英文：一个是回路Circuit，一个是路径Path。

[4] 哈密顿回路/路径

Hamiltonian circuit / path

简而言之，就是和欧拉回路/路径相反的。

欧拉回路/路径保证每一条边被遍历（且仅被遍历）1次，但是点不限次数。

哈密顿回路/路径就是：保证每一个点被遍历（且仅被遍历）1次，但是边不限次数。

需要注意的是，哈密顿回路或路径不一定遍历所有边，也就是你可以遍历这个边，也可以不遍历。

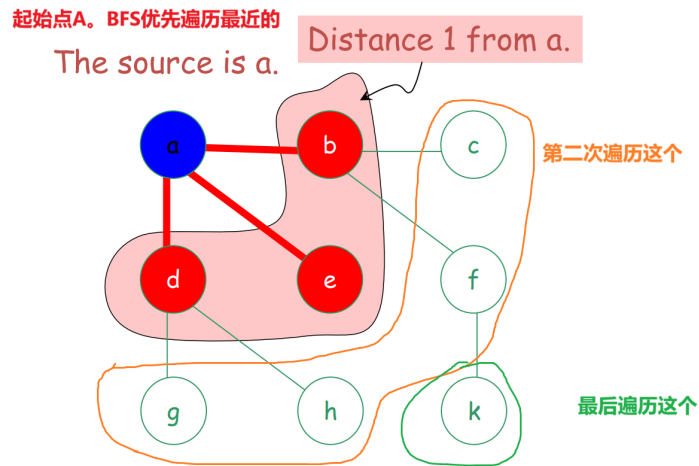
与欧拉回路/路径的情况不同，确定一个图是否包含哈密顿回路（路径）是一个非常困难的问题。（NP 难问题）

[5] 广度优先搜索

BFS - Breadth First Search

在探索距离为 $k + 1$ 的任何顶点之前，先探索所有距离起点 s 为 k 的顶点。

看图，就知道是什么意思了：



实际上就是一层一层遍历。

```

1 // 伪代码
2 取消所有点标记
3 选择一个待遍历点 s
4 标记 s
5 把 s 放入 L 中
6 while L is nonempty do // 当L不是空的时候
7   begin
8     remove a vertex v from front of L
9     visit v
10    for each unmarked neighbor w of v do
11      mark w and insert w into tail of list L
12  end

```

这个代码的意思是：选择一个点 S，然后放入一个队列 L 中。当 L 不是空的时候一直执行：从 L 头取出一个点 S，访问这个点没被标记的邻居们，然后插入 L 的尾巴。

比如上图，我们访问 a，接着把 a 的邻居 bcd 放入 L 中：[bde] 接着拿出 b，访问其邻居 cf 放入队列 [decf]，拿出 d 访问 gh 放入队列 [ecfgh] 然后再访问 e，发现没邻居。最后队列 [cfgh] 就是下一层了。这个就是广度优先，用一个 while 循环就能写

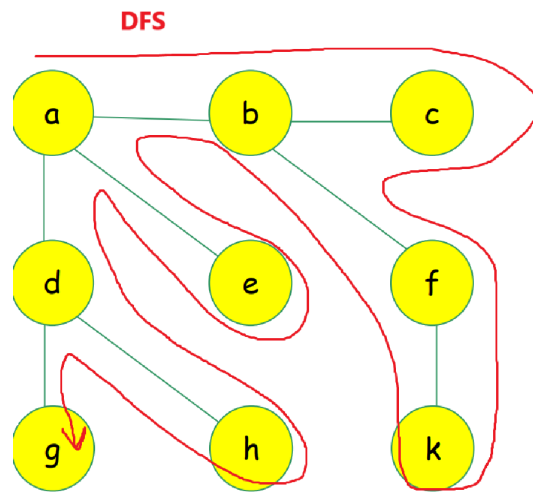
[6] 深度优先搜索

DFS - Depth First Search

深度优先搜索 (DFS)

深度优先搜索是另一种探索图的策略；它尽可能在图中“深入”搜索。

- 从最近发现的顶点 v 出发，探索其尚未被探索的边。
- 当顶点 v 的所有边都被探索完毕后，搜索会“回溯”到发现 v 的顶点，继续探索其未探索的边。



深度优先搜索的实现，其实使用递归是非常方便的：

选择点a，然后访问其所有邻居，对这些邻居们分别重新递归到 `dfs()` 函数。

[7] 实现BDS和DFS

查阅同目录下的 [\[Algorithm.pdf\]](#)

■ [Week 4]

[1] 有向图

有向图的入度 / 出度

无向图有度（Degree），但是有向图有方向，所以引入了入度和出度。也是见名知意的：

顶点 v 的**入度**（in-degree）是指向该顶点的边的数量；

顶点 v 的**出度**（out-degree）是从该顶点出发的边的数量。

顶点	入度 in-deg (v)	出度 out-deg (v)
a	0	1
b	2	2
c	0	2
d	1	1
e	3	0
总和	6	6

提问 这个图中所有的点的入度、出度 总和总是相等吗？（Always equal?）

答案：是的。所有顶点入度之和 = 所有顶点出度之和 = 边的总数（每条边贡献 1 个入度和 1 个出度）

接下来让我们看看有向图的邻接矩阵、邻接表吧！

邻接矩阵 / 邻接表

Adjacency Matrix / Adjacency List

是点和点的关系

有向图的邻接矩阵 M （针对含 n 个顶点的有向图）：

- M 是一个 $n \times n$ 矩阵；
- 若存在有向边 (i, j) ，则 $M(i, j) = 1$ 。

邻接表：

- 每个顶点 u 对应一个列表，存储由 u 出发的边所指向的顶点。

关联矩阵 / 关联表

Incidence matrix / list Incidence matrix M for a

对于包含 n 个顶点和 m 条边的有向图，其**关联矩阵** M 是一个 $m \times n$ 的矩阵：

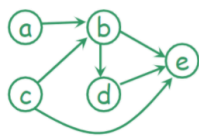
- 若边 i 从顶点 j 出发，则 $M(i, j) = 1$ ；
- 若边 i 指向顶点 j ，则 $M(i, j) = -1$ 。

关联表：每条边对应一个由两个顶点组成的列表，第一个顶点是边的起始顶点（出发顶点），第二个顶点是边的终止顶点（指向顶点）。

如果是无向图，a和b链接直接把(a,b)=(b,a)=1

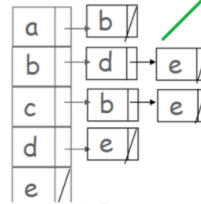
有向图的邻接矩阵、邻接表

如果是无向图，那么a-b必有b-a
a | 右边是a连接的所有点

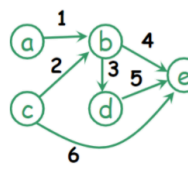


	a	b	c	d	e
a	0	1	0	0	0
b	0	0	0	1	1
c	0	1	0	0	1
d	0	0	0	0	1
e	0	0	0	0	0

点和点

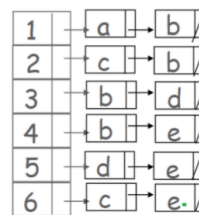


点和点



	a	b	c	d	e
1	1	-1	0	0	0
2	0	-1	1	0	0
3	0	1	0	-1	0
4	0	1	0	0	-1
5	0	0	0	1	-1
6	0	0	1	0	-1

对于数字边，from点标记为1，to点标记为-1
边和点



边和点

如果是无向图，就没有先后顺序，但是有向图有

如果是无向图，只要是数字边关联的全部写成1

[2] 树

树是一种特殊的图。但是树（记为G）：

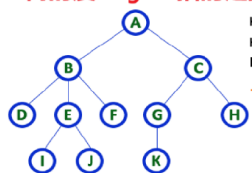
1. 没有环
2. 任意两个点之间必然存在一个路径
3. G是连通的，如果断开一条边必然会割裂这个数
4. 边数 = 点数 - 1。简而言之 $|E| = |V| - 1$ （E是Edge，V是Vertex）

其中，树还有一些术语：

Root 根，就是最上面的点

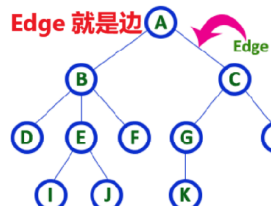
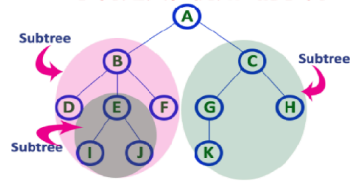


树的度Degree指的是孩子数量



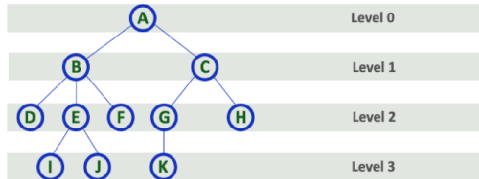
Here Degree of B is 3
Here Degree of A is 2
Here Degree of F is 0
- In any tree, 'Degree' of a node is total number of children it has.

Subtree 子树定义，例如A的子树

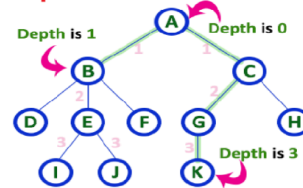


Edge 就是边

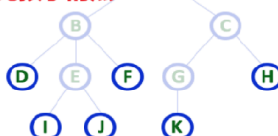
Level 定义如下



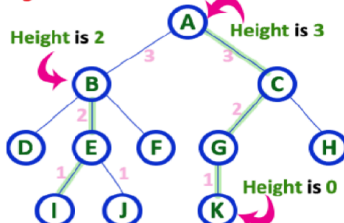
Depth 和 Level 一样



Leaf 叶子节点，无孩子的点



Height 定义如下，刚好和level反过来



二叉树

三种遍历方式

这里一定要记住三种遍历方式的英文，不然考试的时候看不懂题。

遍历：**Traversal**

递归：**Recursion**（函数调用自身，适合深度优先搜索）

迭代：**Iteration**（while 循环等，适合广度优先搜索）

下面的三种遍历方式，前中后序遍历实际上就是看递归位置是前、中还是后

1. 前序遍历 **Preorder**，输出顺序：中左右

```
1 private void getTree(Nodes nodes) {
2     if (treeHead == null) return; // 结束条件
3     tempList.add(nodes.data); // 先添加根节点
4     if (nodes.left != null) getTree(nodes.left); // 然后访问左节点
5     if (nodes.right != null) getTree(nodes.right); // 然后访问右节点
6 }
```

2. 中序遍历 Inorder，输出顺序：左中右。如果是平衡二叉树，那么输出就是从小到大，**这是最常见的遍历方式**

```
1 private void getTree(Nodes nodes) {
2     if (treeHead == null) return;
3     if (nodes.left != null) getTree(nodes.left);
4     tempList.add(nodes.data);
5     if (nodes.right != null) getTree(nodes.right);
6 }
```

3. 后序遍历 Postorder，输出顺序：左右中

```
1 private void getTree(Nodes nodes) {
2     if (treeHead == null) return;
3     if (nodes.left != null) getTree(nodes.left);
4     if (nodes.right != null) getTree(nodes.right);
5     tempList.add(nodes.data);
6 }
```

[3] Tutorial

Question 1

```
1 ALGORITHM BubbleSort(A[0..n - 1])
2 //Sorts a given array by bubble sort
3 //Input: An array A[0..n - 1] of orderable elements
4 //Output: Array A[0..n - 1] sorted in ascending order
5 for i=0 to n - 2 do
6     for j = n-1 downto i+1 do
7         if A[j] < A[j-1] swap A[j] and A[j - 1]
```

1. 给定数组 $A[0..5]=[6, 1, 2, 3, 4, 5]$ ，请问用如上冒泡算法排序到**升序**一共需要交换多少次数据？

在这个算法中， $n-1$ 含义如下：

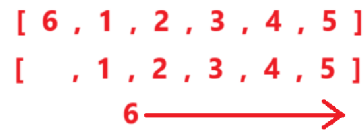
Array = [0,1,2,3,4,5,6,7]，那么 $n-1 = 7$ ，就相当于最后一个下标，我们不妨记为 `lastIndex`。

第一层循环， i 从 0 到 $\text{lastIndex}-1$

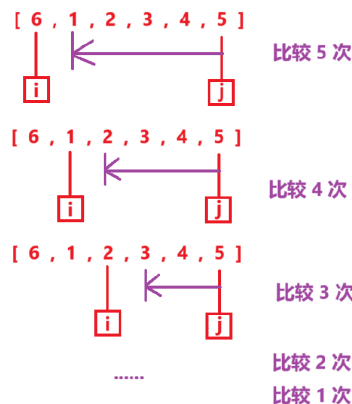
第二层循环， j 从 lastIndex 到 $i+1$

如果 $A[j]$ 小于 $A[j-1]$ 就交换

显然，这很容易模拟，其实就是把 6 不断往后移动，因此交换了 5 次数据



2. 那么请问需要比较的次数是？



比较次数是 $\sum 5 = 5 + 4 + 3 + 2 + 1 = 15$

回顾：

1. 冒泡排序是固定最后 $j=\text{last}$ ，然后前面 $i=1$ 到 $i=j-1$
2. 选择排序是固定 $j=1$ ， $i=j+1$ 到 $i=\text{last}$

Question 2

归并排序

请问对于数组 $A[0..5] = [6, 1, 2, 3, 4, 5]$ ，需要多少次比较？

Algorithm Mergesort($A[0..n-1]$) 现在我们要归并A

if $n > 1$ then begin

copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$ 复制A的前半段给B

copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lfloor n/2 \rfloor - 1]$ 复制A的后半段给C

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$) 递归自身

Mergesort($C[0..\lfloor n/2 \rfloor - 1]$) 递归自身

Merge(B, C, A) 当递归到尽头，就运行Merge方法，把BCA传递过去

End

Algorithm Merge($B[0..p-1], C[0..q-1], A[0..p+q-1]$)

Set $i=0, j=0, k=0$

while $i < p$ and $j < q$ do

begin 其实就是让BC比大小，放入小的元素

if $B[i] \leq C[j]$ then set $A[k] = B[i]$ and increase i

else set $A[k] = C[j]$ and increase j

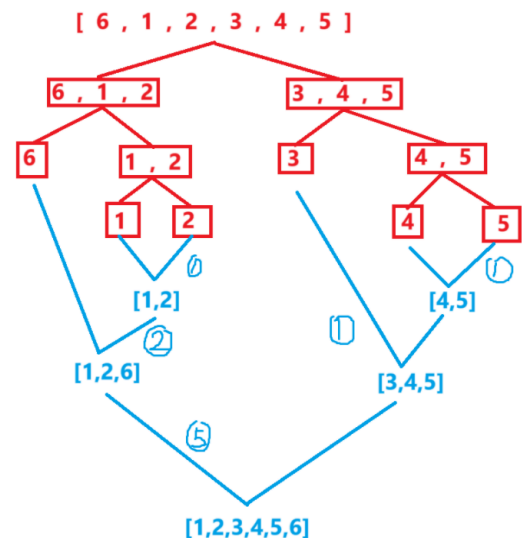
$k = k + 1$

end

if $i == p$ then copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

结束了，剩余的数组可以直接全部copy到A中



总共比较了10次

Question 3

已知上述的归并排序的时间复杂度可以被表述为以下的递推关系：

$$T(n) \begin{cases} 1 & \text{if } n = 1 \\ 2T(\frac{n}{2}) + n & \text{if } n > 1 \end{cases}$$

证明 $T(n) = O(n \log n)$

using the iterative method

要求用迭代的方法，实际上展开就行：

$$\begin{aligned} T(n) &= 2T(\frac{n}{2}) + n \\ &= 2(2T(\frac{n}{4}) + \frac{n}{2}) + n = 4T(\frac{n}{4}) + 2n \\ &= 8T(\frac{n}{8}) + 3n \\ &= 16T(\frac{n}{16}) + 4n \\ &\dots \quad \text{进行了 } \log_2 n \text{ 轮，终于把 } n \text{ 除尽为1了} \\ &= 2^{\log_2 n} T(1) + (\log_2 n)n \\ &= nT(1) + n \log n \\ &\rightarrow O(n \log n) \end{aligned}$$

Question 4

- 编写一个分治算法（Divide and Conquer）的伪代码(PseudoCode)，用于在一个包含n个数字的数组中找到最大元素的位置。
- 针对你的算法所进行的关键比较次数，建立并求解（当 $n = 2^k$ 时）一个递推关系。

解答：

- 还记得分治算法的核心吗？把大东西拆成小东西，这是分，然后再从小到大（例如合并），这是治。

我们可以参考归并排序来写

```
1 findMax(Array[l...r])
2   if l == r return Array[l],l
3   leftMax = findMax(Array[l...(l+r)/2])
4   rightMax = findMax(Array[(l+r)/2+1...r])
5   if leftMax[0]>rightMax[0]
6     return leftMax[0],leftMax[1]
7   else
8     return rightMax[0],rightMax[1]
```

- 显然，如果数组只有一个元素，那么不需要比较，因此 $T(1) = 0$ 。

如果数组有n个元素，那么就需要“分”的结果那么多次数（也就是分成AB俩数组的比较次数），然后再比较1次（左右的最大值）。因此是 $2T(\frac{n}{2}) + 1$

总的来说：

$$T(n) \begin{cases} 0 & \text{if } n = 1 \\ 2T(\frac{n}{2}) + 1 & \text{if } n > 1 \end{cases}$$

如果 $n = 2^k$ ，直接带入即可

这个和归并排序很相似。现在我们来求他的时间复杂度（用迭代方法）

$$\begin{aligned} T(n) &= 2T(\frac{n}{2}) + 1 \\ &= 2(2T(\frac{n}{4}) + 1) + 1 = 4T(\frac{n}{4}) + 3 \\ &= 8T(\frac{n}{8}) + 7 \\ &= 16T(\frac{n}{16}) + 15 \\ &\dots \quad \text{进行了 } \log_2 n \text{ 轮，终于把 } n \text{ 除尽为 } 1 \text{ 了} \\ &= 2^{\log_2 n} T(1) + 2^{\log_2 n} - 1 \\ &= nT(0) + n - 1 \\ &= n - 1 \\ &\rightarrow O(n) \end{aligned}$$

Question 5

1. 设计一个分治算法，用于找出一个包含 n 个数字的数组中最大值和最小值这两个值。
2. 针对你所设计算法进行的关键比较次数，建立并求解（当 $n = 2^k$ 时）一个递推关系。

和上一题差不多，我们只需要改一下代码

```
1 findMinAndMax(Array[l...r])
2     if l == r return Array[l],Array[r]
3     leftMinMax = findMinAndMax(Array[l...(l+r)/2])
4     rightMinMax = findMinAndMax(Array[(l+r)/2+1...r])
5     return min(leftMinMax[0],rightMinMax[0]),
6           max(leftMinMax[1],rightMinMax[1])
```

如果 $n = 0$ ，那么 $T(0) = 0$

如果 $n = 1$ ，那么 $T(1) = 0$

如果 $n = 2$ ，那么 $T(2) = 1$

其他情况 $T(n) = 2T(\frac{n}{2}) + 2$ 这里 +2 是因为有俩比较，不是一次比较了

因此：

$$T(n) \begin{cases} 0 & \text{if } n = 0, 1 \\ 2T(\frac{n}{2}) + 2 & \text{if } n > 1 \end{cases}$$

■ [Week 5]

看课件讲算法不如去B站找视频看更清晰！



[1] 贪心算法

Greedy Method，顾名思义，每一步都最优解从而达到全局最优解。但有时局部最优解并非全局最优解。

贪心算法的例子

[最小生成树\(看这个视频\)](#)

Minimum spanning tree MST最小生成树

- Prim 算法
- Kruskal 算法

单源最短路径

- Dijkstra算法，看[这个视频](#)

[2] Prim算法

适合边更多的图

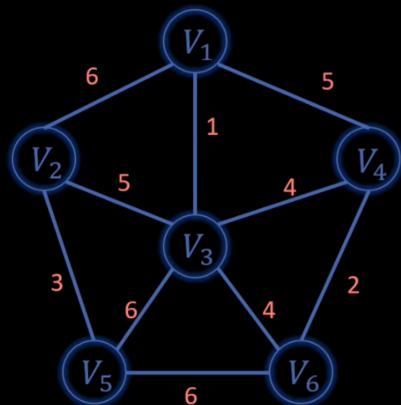
假设我们有 v1 到 v6 这六个城市，然后有若干条路链接。这些路有一个权重 w 代表修路成本。

现在需让这六个城市连通的同时修路成本最小...

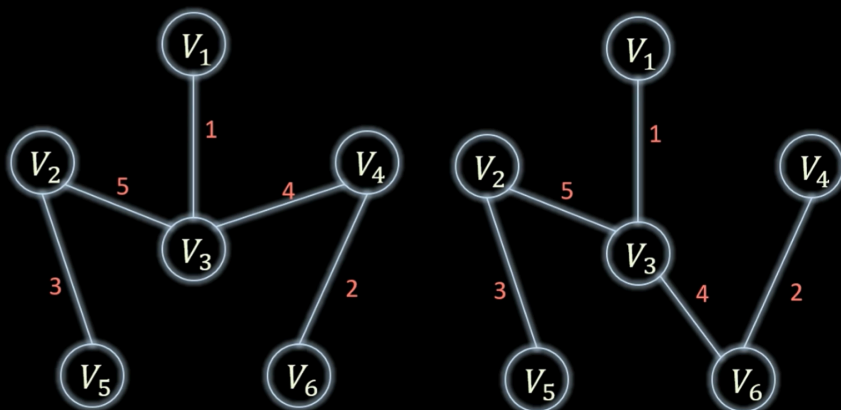
你要成本最小，必然是边越少越好，因此：如果有 n 个点，那么边最少就是 $n - 1$ ，从而称为一棵树，这就是最小生成树！

Prim算法是贪心算法，但是得出来的最小生成树并非唯一的

原无向图



Prim算法求最小生成树



Prim算法的步骤：

首先确定一个城市，在这里，就先选城市 v_1 ，点亮它（这时候就是最小生成树的一部分）

对于那些没有点亮的点，找出一个点满足如下条件：这个没点亮的城市（点）在所有没点亮的城市中距离某个点亮城市修路成本最少

比如我们选完 v_1 后，查看剩下没点亮的点，有 v_2, v_3, v_4, v_5, v_6 。这些没点亮的点中只有 v_3 距离点亮的 v_1 距离最短（其中 v_5 和 v_6 不需要考虑因为他们连着的城市都不是点亮的）。

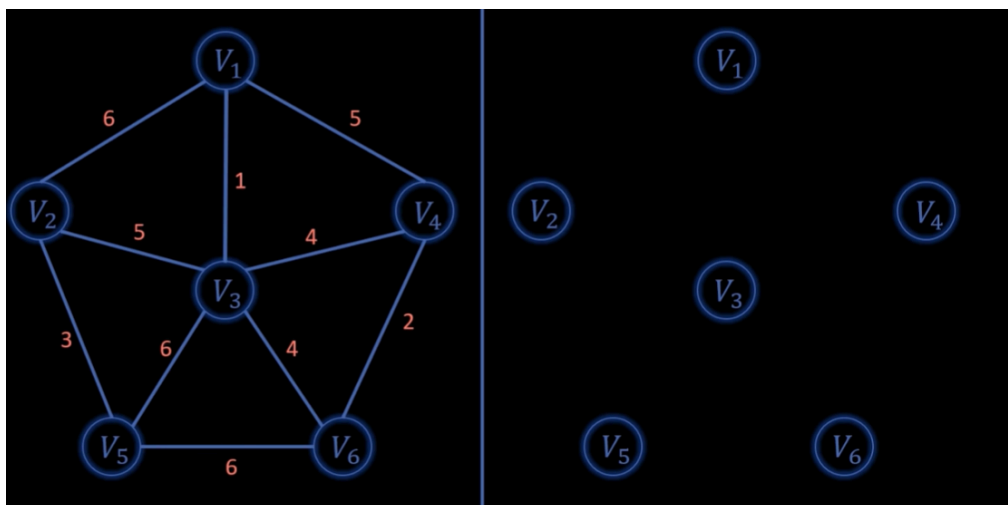
然后重复这个步骤...

[3] Kruskal 算法

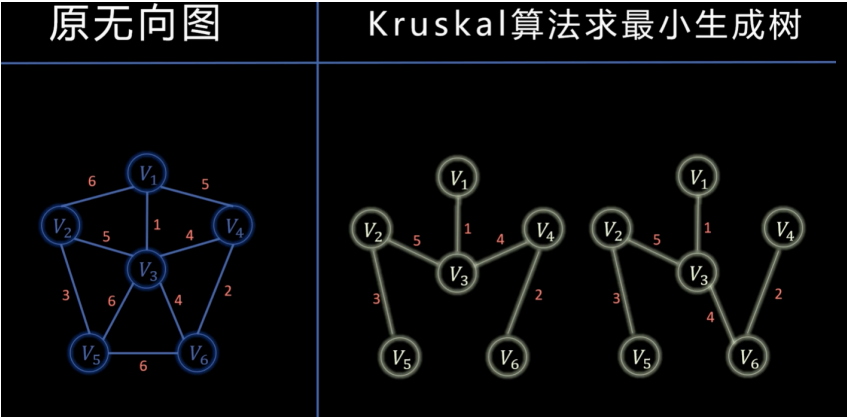
适合点更多的图

这个和Prim算法是反过来的，步骤更简单：

初始化如下：默认所有边都没使用



然后对于所有的未使用使用的边，从小到大依次遍历：如果这个边所连接的 va 和 vb 两个点是非连通的，那么吧这个边添加到右侧的图中，并且标记为已使用



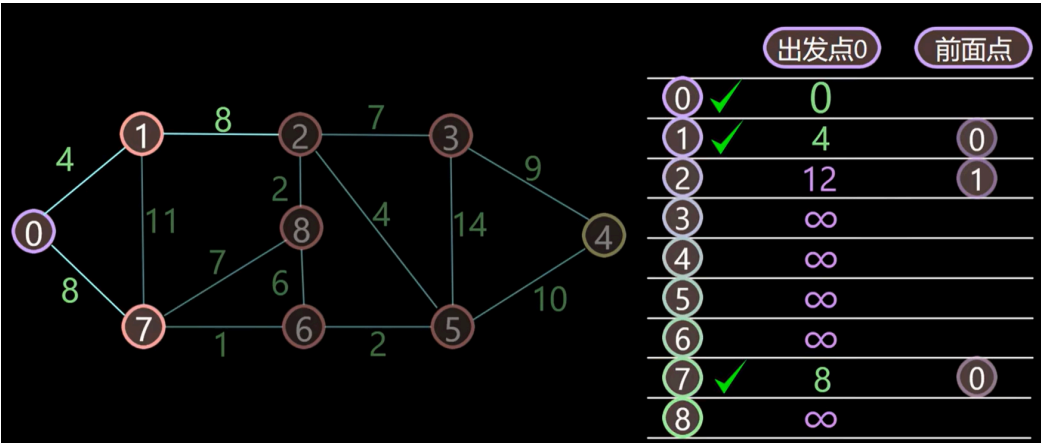
[4] Dijkstra算法

Dijkstra 算法和前两个不同，不属于MST的范畴了。类似于：我现在有很多没有连通的城市，但是我使用了Prim和Kruskal算法来构建了每个城市之间的高速公路。然后我也同时加了很多新的路

但是如何找到城市A到其他所有城市的最短距离呢？

[这个视频](#)讲述了Dijkstra算法是如何实现的。简而言之：

[初始化：标记出发节点到出发节点的距离为0，其余为 ∞]



1. 在未标记的节点中选择距离出发点最近的节点
2. 标记这个节点为最优路径
3. 更新这个节点的临近节点（更新距离如果更短（和前驱节点））
4. 重复1-3

[5] 时间复杂度

| V | 代表图中顶点集合 V 里顶点的数量， E 代表图中边集合 E 里边的数量。

- Prim算法: $O(|E|\log|V|)$
- Kruskal算法: $O(|E|\log|E|)$
- Dijkstra 算法 $O(V^2)$

[6] 实现Prim、Kruskal和Dijkstra算法

查阅同目录下的 [Algorithm.pdf]

■ [Week 6]

Dynamic Programming 动态规划

学习成果

- 理解动态规划的基本思想
- 能够运用动态规划计算斐波那契数
- 能够运用动态规划解决装配线调度问题

在 20 世纪 40 年代后期（当时计算机还很罕见），“programming”指的是“表格法”...

关于动态规划

动态规划是：一种实现某些分治算法的高效方法，其基本步骤如下：

- 定义问题并识别子问题
- 为该问题制定递归解决方案
- 将子问题的解存储在表格或数组中（记忆化）
- 使用存好的子问题的解来计算该问题的最优解

例子 1

斐波那契数列的动态规划...

我们知道斐波那契数列是这样的

$$F(n) = \begin{cases} 1 & \text{if } n = 0 \text{ or } 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

n	0	1	2	3	4	5	6	7	8	9	10
F(n)	1	1	2	3	5	8	13	21	34	55	89

但是，当我们求一个很大的数，那么 $F(1)$ 、 $F(2)$... $F(10)$... 可能这种靠前的就会用到很多次

因此，我们可以用动态规划的思想去储存这些数字！！

```
1 // 原先的斐波那契数列--不使用动态规划
2 public static int FibonacciOrigin(int n) {
3     if(n<=0 || n == 1 || n == 2) return 1;
4     return FibonacciOrigin(n-1) + FibonacciOrigin(n-2);
5 }
6 // 新的斐波那契数列--使用动态规划
7 private static final int[] fibonacciArray = new int[10000];
8 public static int FibonacciNew(int n) {
9     if (n <= 0 || n == 1 || n == 2) {
10         fibonacciArray[n] = 1;
11         return fibonacciArray[n];
12     }
13     if(fibonacciArray[n-1] !=0 && fibonacciArray[n-2] !=0) // 如果有记录直接返回
14         return fibonacciArray[n-1] + fibonacciArray[n-2];
15     // 否则就运算
```



```

16     fibonacciArray[n] = fibonacciArray[n - 1] + fibonacciArray[n - 2]; // 记录
17     return FibonacciNew(n - 1) + FibonacciNew(n - 2);
18 }

```

让我们测试一下!

我们模拟程序不断读取斐波那契数列，例如读取1000次！每次求 1~40之间的随机数字

```

1  public static void main(String[] args) {
2      int loopTime = 1000;
3      long start = System.currentTimeMillis();
4      for (int i = 0; i < loopTime; i++) {
5          FibonacciOrigin(i % 40);
6      }
7      long end = System.currentTimeMillis();
8      System.out.println("原始方法耗时: " + (end - start) + "ms");
9      start = System.currentTimeMillis();
10     for (int i = 0; i < loopTime; i++) {
11         FibonacciNew(i % 30);
12     }
13     end = System.currentTimeMillis();
14     System.out.println("动态规划耗时: " + (end - start) + "ms");
15 }

```

原始方法耗时: 7884ms
动态规划耗时: 0ms

动态规划只需要运算一次，后面的都是常数时间，因此非常非常快...

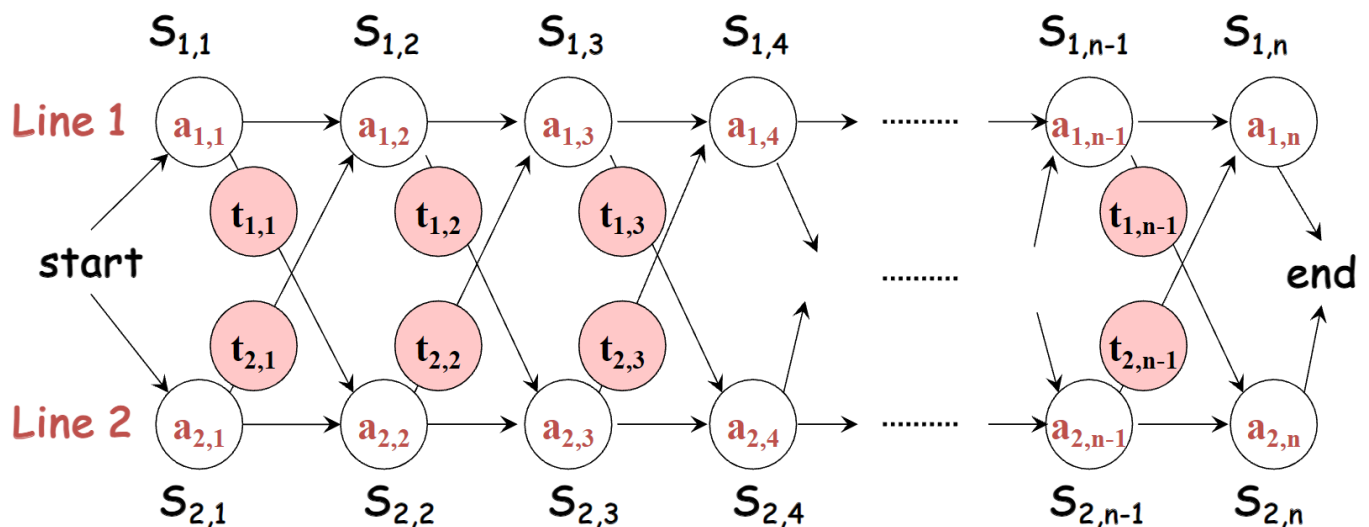
例子 2

装配线调度 Assembly line scheduling

首先，我们有两条调度线，Line1和Line2. 你的任务是从 **start** 走到 **end**，并且耗时需要尽可能短

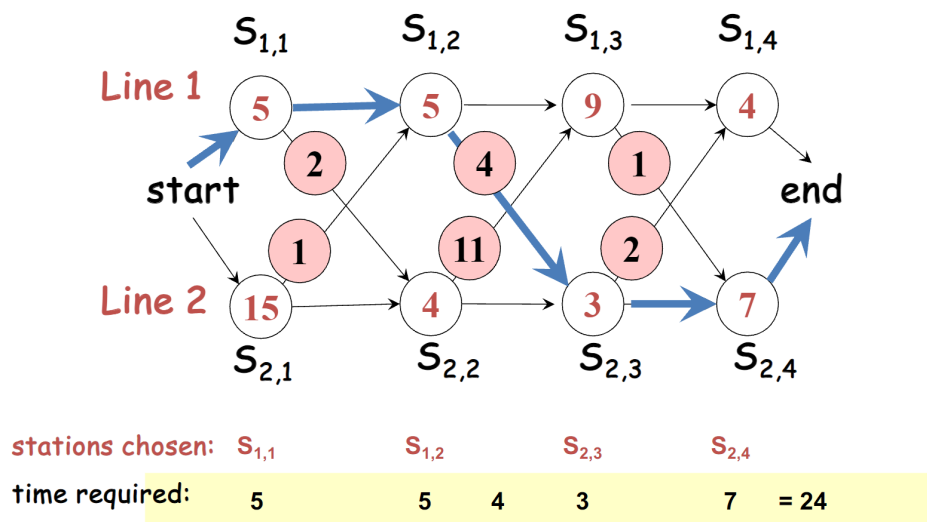
每到达一个站点，就需要装货，装货要时间。

然后去下一个站点，如果换到另一条线，也需要时间（除非你不换线）。

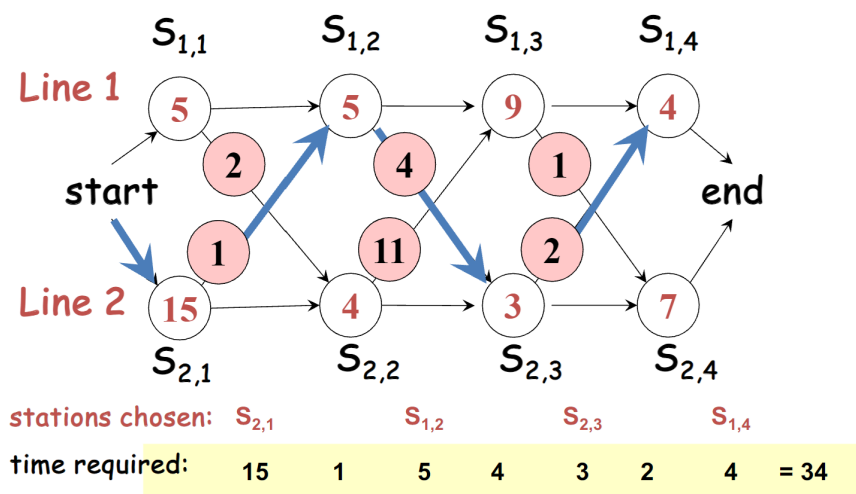


什么意思呢？先让我们看看下标。比如 $S_{i,j}$ 其中第一个参数 i 代表第几个调度线， j 代表第几个站点。

让我们看个例子...比如我们可以这样走：



这样耗时是24！我们尝试另一种走法...

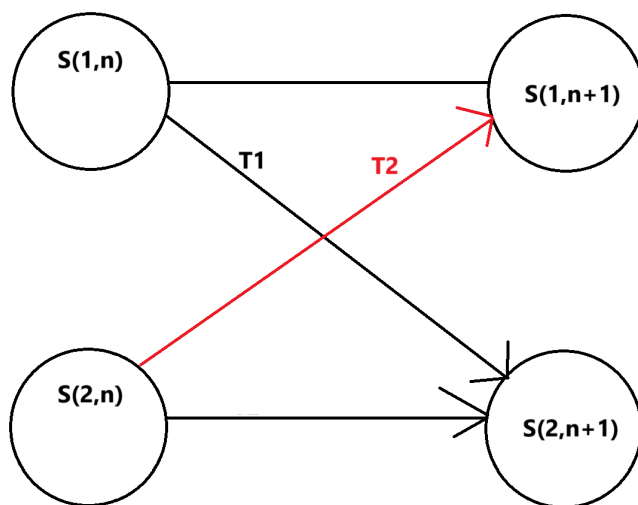


这样耗时更多了！

那么我们如何确定耗时最少的走法呢？我们可以这样想....

假设我走到第 $S_{1,n}$ 个站点（也就是第 1 条线的第 n 个站点），就记录到此站点所需要最小时间为 $\min[1][n]$

那么到达下一个站点 $S_{1,n+1}$ 和 $S_{2,n+1}$ 需要的时间，是不是就肯定依赖上一个站点 n 的最小时间，对吧...



比如，

$$S_{1,n+1} = \min[S_{1,n} \mid S_{2,n} + T_2]$$

同理 $S_{2,n+1}$...如果我们这样推导，问题就解决了！

到 end 所需要的最小时间就是 $\min[S_{1,m-1} \mid S_{2,m-1}]$

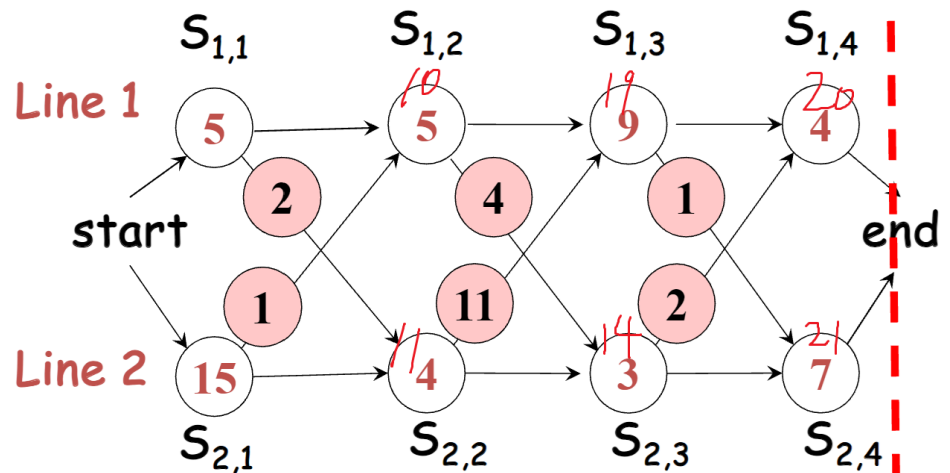
然后又 $S_{1,m-1}$ 又可以变为 $S_{1,m-1} = \min[S_{1,m-2} \mid S_{2,m-2} + T] \dots$

```
1  import java.util.Arrays;
2
3  public class Assemble {
4      private static final int[] Line1ToLine2Time = new int[]{2,4,1};
5      private static final int[] Line2ToLine1Time = new int[]{1,11,2};
6      private static final int[] S1 = new int[]{5,5,9,4};
7      private static final int[] S2 = new int[]{15,4,3,7};
8      private static final int[] S1Time = new int[4];
9      private static final int[] S2Time = new int[4];
10     public static void main(String[] args) {
11         // 初始化
12         Arrays.fill(S1Time, Integer.MAX_VALUE);
13         Arrays.fill(S2Time, Integer.MAX_VALUE);
14         System.out.println("End前S1最短时间: "+minTime(3,1));
15         System.out.println("End前S2最短时间: "+minTime(3,2));
16         System.out.println("因此总体最短时间是"+Math.min(minTime(3,1),minTime(3,2)));
17     }
18     private static int minTime(int stationNumber, int line){
19         if(stationNumber == 0) // 递归到底了，结束条件
20         {
21             if(line == 1)
22             {
23                 S1Time[stationNumber] = S1[stationNumber]; // 储存
24                 return S1Time[stationNumber];
25             }else {
26                 S2Time[stationNumber] = S2[stationNumber]; // 储存
27                 return S2Time[stationNumber];
28             }
29         }
30         // 如果访问过了，那么就直接返回(动态规划)
31         if(line == 1 && S1Time[stationNumber] != Integer.MAX_VALUE)
32             return S1Time[stationNumber];
33         if(line == 2 && S2Time[stationNumber] != Integer.MAX_VALUE)
34             return S2Time[stationNumber];
35         // 还没访问过
36         if(line == 1)
37         {
38             S1Time[stationNumber] = Math.min(minTime(stationNumber-1,1), minTime(stationNumber-1,2)
+ Line2ToLine1Time[stationNumber-1] )+ S1[stationNumber]; // 储存
39             return S1Time[stationNumber];
40         }
41         else //if(line == 2)
42         {
43             S2Time[stationNumber] = Math.min(minTime(stationNumber-1,2) , minTime(stationNumber-
1,1) + Line1ToLine2Time[stationNumber-1] )+ S2[stationNumber]; // 储存
44
45             return S2Time[stationNumber];
46         }
47     }
```

输出

- 1 End前S1最短时间：20
- 2 End前S2最短时间：21
- 3 因此总体最短时间是20

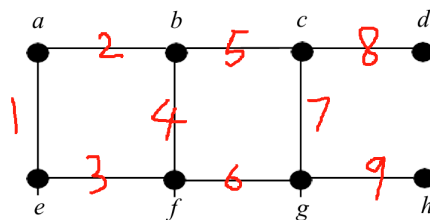
j	$f_1[j]$	$f_2[j]$
1	5	15
2	10	11
3	19	14
4	20	21



Tutorial

Question 1

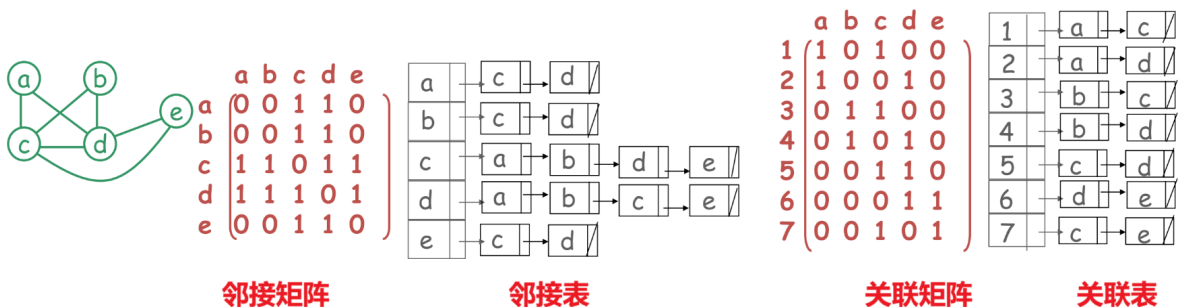
考虑下图 G



(此处为一个包含顶点 a 、 b 、 c 、 d 、 e 、 f 、 g 、 h 及对应边的图)

1. 给出图 G 的邻接矩阵和邻接表。
2. 给出图 G 的关联矩阵和关联表。

还记得无向图的邻接矩阵邻接表吗



3. 从顶点 a 开始，按照顶点字母顺序解决并列情况，用广度优先搜索（BFS）遍历该图，并构建相应的广度优先搜索树。
4. 从顶点 a 开始，按照顶点字母顺序解决并列情况，用深度优先搜索（DFS）遍历该图，并构建相应的深度优先搜索树（此处PPT有误这里改过来了）。

1. 邻接矩阵和邻接表

邻接矩阵:

设图 $G = (V, E)$, $V = \{a, b, c, d, e, f, g, h\}$, 邻接矩阵 A 是一个 8×8 的矩阵。如果顶点 i 和顶点 j 之间有边, 则 $A[i][j] = 1$, 否则 $A[i][j] = 0$ 。

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

邻接表:

$a : [b, e]$
 $b : [a, c, f]$
 $c : [b, d, g]$
 $d : [c, h]$
 $e : [a, f]$
 $f : [b, e, g]$
 $g : [c, f, h]$
 $h : [d, g]$

2. 关联矩阵和关联表

关联矩阵:

设图有 m 条边, $n = 8$ 个顶点。边依次设为

$e_1 = (a, b), e_2 = (a, e), e_3 = (b, c), e_4 = (b, f), e_5 = (c, d), e_6 = (c, g), e_7 = (d, h), e_8 = (e, f), e_9 = (f, g), e_{10} = (g, h)$, 共 $m = 10$ 条边。关联矩阵 B 是一个 8×10 的矩阵。如果顶点 i 与边 j 相关联, 则 $B[i][j] = 1$, 否则 $B[i][j] = 0$ 。

$$B = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

关联表:

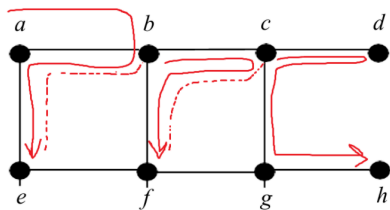
$a : [e_1, e_2]$
 $b : [e_1, e_3, e_4]$
 $c : [e_3, e_5, e_6]$
 $d : [e_5, e_7]$
 $e : [e_2, e_8]$
 $f : [e_4, e_8, e_9]$
 $g : [e_6, e_9, e_{10}]$
 $h : [e_7, e_{10}]$

3. 广度优先搜索 (BFS) 及 BFS 树

- **BFS遍历顺序:**

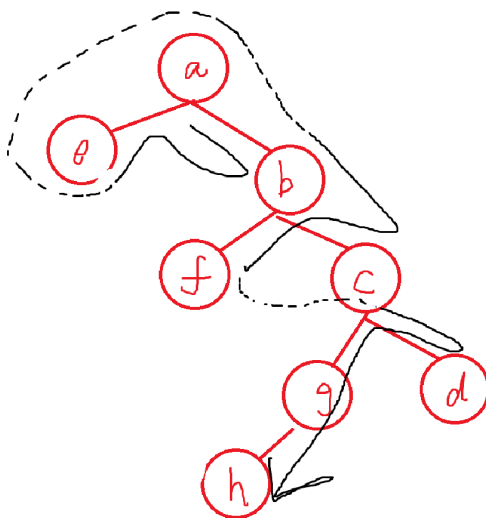
从顶点 a 开始, 首先访问 a , 然后访问 a 的邻接点 b 和 e (按字母顺序), 接着访问 b 的未访问邻接点 c 和 f , 再访问 e 的未访问邻接点 f (已访问过, 跳过), 然后访问 c 的未访问邻接点 d 和 g , 以此类推。遍历顺序为 a, b, e, c, f, d, g, h 。

- 当然, 也可以使用一个队列来做。首先队列只有 $[a]$, 接下来拿出 a , 放入其邻居, $[be]$, 接下来拿出 b 放入其邻居, $[ecf]$... 重复这个过程。这是最简单、最不容易出错的方法!



- **BFS树:**

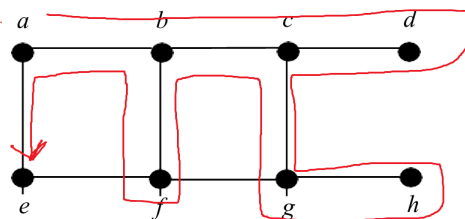
根节点为 a , a 的子节点为 b 和 e ; b 的子节点为 c 和 f ; e 无新子节点; c 的子节点为 d 和 g ; f 无新子节点; d 的子节点为 h ; g 无新子节点。



4. 深度优先搜索 (DFS) 及DFS树

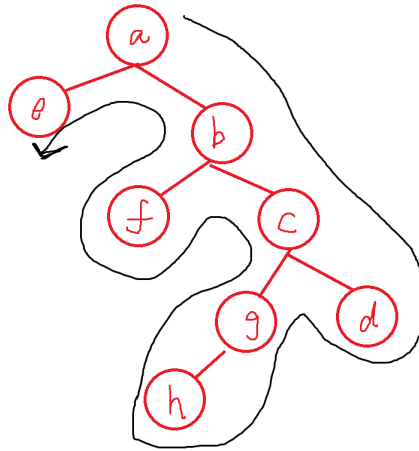
- **DFS遍历顺序:**

从顶点 a 开始, 访问 a , 然后访问 a 的邻接点 b , 接着访问 b 的邻接点 c , 再访问 c 的邻接点 d , d 无未访问邻接点, 回溯到 c , 访问 c 的另一个邻接点 g , g 的邻接点 h , h 无未访问邻接点, 回溯到 g , 再回溯到 c , 回溯到 b , 访问 b 的另一个邻接点 f , f 的邻接点 e , e 无未访问邻接点, 回溯到 f , 再回溯到 b , 最后回溯到 a 。遍历顺序为 a, b, c, d, g, h, f, e 。



- **DFS树:**

根节点为 a , a 的子节点为 b ; b 的子节点为 c ; c 的子节点为 d 和 g ; d 无新子节点; g 的子节点为 h ; h 无新子节点; c 回溯后, b 的另一个子节点为 f ; f 的子节点为 e ; e 无新子节点。



Question 2

考虑以下递归函数：

$$f(n) = \begin{cases} 1 & \text{若 } 0 \leq n \leq 3 \\ f(n-1) + f(n-2) + f(n-3) + f(n-4) & \text{若 } n > 4 \end{cases}$$

1. 编写一个递归（自顶向下）算法来计算该函数。

```
1 public static int function(int n) {
2     if(n <= 3) return 1;
3     return function(n - 1) + function(n - 2) + function(n - 3) + function(n - 4);
4 }
```

2. 你的算法的时间复杂度（用大 O 表示法）是多少？

$O(4^n)$ ：每一个根节点都会延伸出 4 个叶子节点

但是更标准的答案是：

若直接采用递归方法（无记忆化存储），每次调用 $f(n)$ 会递归调用

$f(n-1), f(n-2), f(n-3), f(n-4)$ ，形成一棵递归树。

- **递推关系**：每个状态 n 分解为 4 个子状态 $n-1, n-2, n-3, n-4$ 。

- **时间复杂度**：递归树的节点数近似为指数级，满足递推式

$$T(n) = T(n-1) + T(n-2) + T(n-3) + T(n-4) + O(1)。$$

其通解的主导项为 $\Theta(\lambda^n)$ ，其中 λ 是特征方程 $x^4 - x^3 - x^2 - x - 1 = 0$ 的最大实根（约为 1.9276）。

因此，**暴力递归的时间复杂度为 $O(\lambda^n)$ （指数级）**，效率极低。

Without memoization, $f(n)$ satisfied with **recurrence relation** $T(n) = T(n-1) + \dots + T(n-4) + O(1)$, thus the time complexity is $O(\lambda^n)$ where λ is the maximum real root of $x^4 - x^3 - x^2 - x - 1 = 0$

当用Java计算 $Function(38)$ 的时候就用了 **19687ms**

3. 利用动态规划的思想，设计并编写一个更快的非递归（自底向上）算法的伪代码。

注意这里的原题是自底向上(Bottom-up Algorithm)，说人话就是非递归的算法，也就是从最小的子问题开始解决
那么伪代码就是

```
1 input n
2 array a[0...n-1]
3 for i = 0 to n-1 do
4 begin
5     if i <=3 a[i] = 1
6     else a[i] = a[i-1]+a[i-2]+a[i-3]+a[i-4]
7 end
8 print a[n-1]
```

那么自顶向下 (Top-down) 的算法就是递归了：

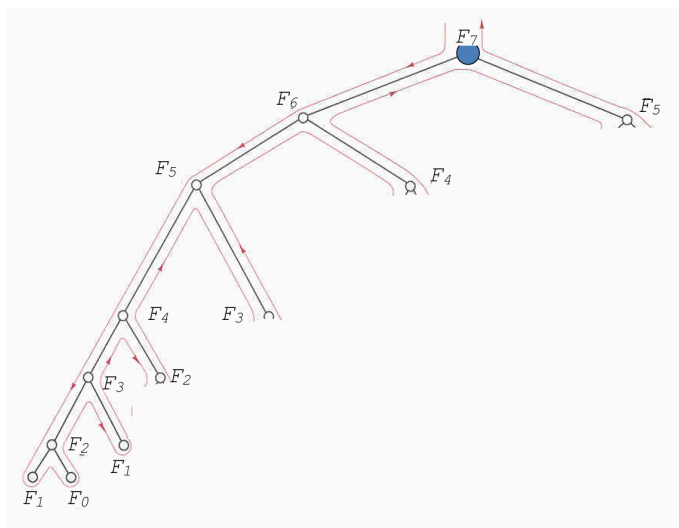
```
1 private static final int[] data = new int[200];
2 public static int function(int n) {
3     if(n <=3 )
4     {
5         data[n] = 1;
6         return data[n];
7     }
8     if(data[n] != 0)
9         return data[n];
10    data[n] = function(n - 1) + function(n - 2) + function(n - 3)+function(n - 4);
11    return data[n];
12 }
```

4. 这个更快的算法的时间复杂度（用大O表示法）是多少？

$O(n)$ 。当用了记忆化搜索（或者动态规划）：储存了数据之后

不管是迭代（自底向上）还是递归（自顶向下）都是 $O(n)$ 线性时间。

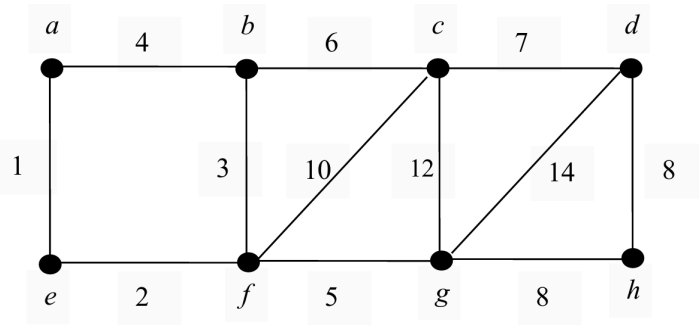
这样这个图就是...只有一边从上到下遍历，其他的就因为标记过而没有更多的子节点



类似于这种

Question 3

给定下图

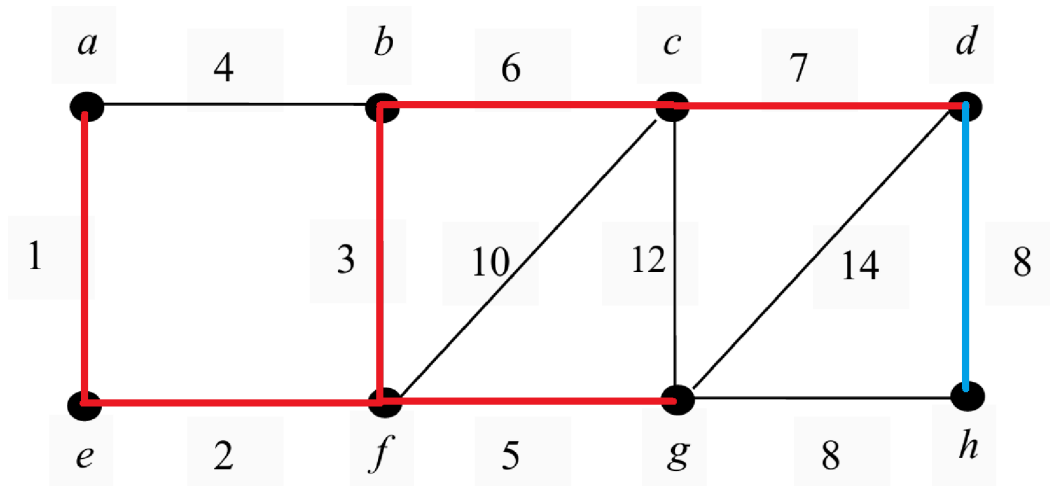


给定如下的 Prim 算法伪代码，画出最小生成树

```

MST-Prim-Algorithm( $G, w, \text{root}$ )
// Given a weighted graph  $G=(V,E)$  and a root vertex
for each  $u$  in  $V$  do
  begin
     $\text{key}[u] = \infty$ 
     $\pi[u] = \text{NIL}$ 
  end
 $\text{key}(\text{root}) = 0$ 
 $V_T = \emptyset$ 
while  $V - V_T \neq \emptyset$  do
  begin
    choose the vertex  $u$  in  $V - V_T$  with minimum  $\text{key}(u)$ 
     $V_T = V_T \cup \{u\}$ 
    for every vertex  $v$  in  $V - V_T$  that is a neighbour of  $u$  do
      if  $w(u,v) < d(v)$  then //  $w(u,v)$  is the weight of edge  $(u,v)$ 
         $d(v) = w(u,v)$ 
         $\pi[v] = u$ 
  end
  
```

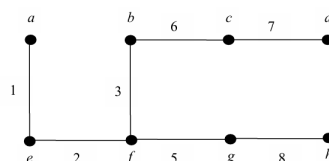
这个伪代码有点抽象，不过我们只需要看一些关键部分，这里选择的是最近的边，因此可以写出最小生成树如下：



其中蓝色部分是，既可以选择 $d-h$ 也可以选择 $g-h$

使用Kruskal算法画出最小生成树

Kruskal和Prim是差不多的图，不过其添加边的顺序是 1, 2, 3, 4, 5, 6, 7, 8



使用Dijkstra算法求出点 a 到其他点的最短路径

红色为标记	距离出发点	前面点
a	INF	NULL
b	INF	NULL
c	INF	NULL
d	INF	NULL
e	INF	NULL
f	INF	NULL
g	INF	NULL
h	INF	NULL

红色为标记	距离出发点	前面点
a	0	
b	INF	NULL
c	INF	NULL
d	INF	NULL
e	INF	NULL
f	INF	NULL
g	INF	NULL
h	INF	NULL

红色为标记	距离出发点	前面点
a	0	
b	4	a
c	INF	NULL
d	INF	NULL
e	1	a
f	INF	NULL
g	INF	NULL
h	INF	NULL

红色为标记	距离出发点	前面点
a	0	
b	4	a
c	INF	NULL
d	INF	NULL
e	1	a
f	3	e
g	INF	NULL
h	INF	NULL

红色为标记	距离出发点	前面点
a	0	
b	4	a
c	10	b
d	17	c
e	1	a
f	3	e
g	8	f
h	16	g

红色为标记	距离出发点	前面点
a	0	
b	4	a
c	10	b
d	22	g
e	1	a
f	3	e
g	8	f
h	16	g

红色为标记	距离出发点	前面点
a	0	
b	4	a
c	10	b
d	INF	NULL
e	1	a
f	3	e
g	8	f
h	INF	NULL

红色为标记	距离出发点	前面点
a	0	
b	4	a
c	15	f
d	INF	NULL
e	1	a
f	3	e
g	8	f
h	INF	NULL

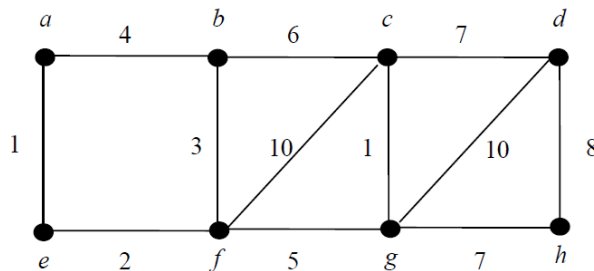
红色为标记	距离出发点	前面点
a	0	
b	4	a
c	10	b
d	17	c
e	1	a
f	3	e
g	8	f
h	16	g

比如点 h 到 a 的最短路径是16，其中如果要知道路径，就通过前面点递归即可。

注意！在考试中你的规范步骤要这样写（示例）

Question 5 (30 marks)

Consider the following graph G. The label of an edge is the cost of the edge.



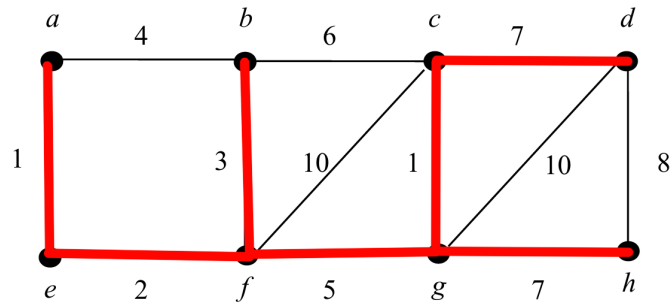
- Using *Prim's* algorithm, draw a *minimum spanning tree* (MST) of the graph. Also write down the change of the priority queue step by step and the order in which the vertices are selected. Is the MST drawn unique? (i.e., is it the one and only MST for the graph?) (10 marks)
- Using *Kruskal's* algorithm, draw a *minimum spanning tree* (MST) of the graph G. Write down the order in which the edges are selected. Is the MST drawn unique? (i.e., is it the one and only MST for the graph?) (10 marks)
- Referring to the same graph above, find the shortest paths from the vertex *a* to *all* other vertices in the graph G using *Dijkstra's* algorithm. Show the changes of the priority queue step by step and give the order in which edges are selected. (10 marks)

N.B. There may be more than one solution. You only need to give one of the solutions.

Question 5 (30 marks)

Solution (a)

The (one possible version of) MST of **Prim's** Algorithm is as follows...



The priority queue is as follows...

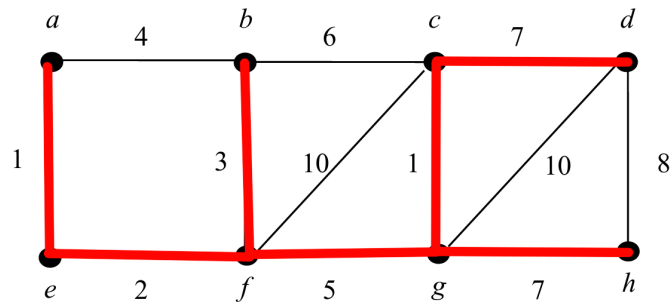
Order Selected	a(-,0)	b(-,∞)	c(-,∞)	d(-,∞)	e(-,∞)	f(-,∞)	g(-,∞)	h(-,∞)
a(-,-)	-	(a,b,4)	-	-	(a,e,1)	-	-	-
e(a,1)	-	(a,b,4)	-	-	-	(e,f,2)	-	-
f(e,2)	-	(f,b,3)	(f,c,10)	-	-	-	(f,g,5)	-
b(f,3)	-	-	(b,c,6)	-	-	-	(f,g,5)	-
g(f,5)	-	-	(g,c,1)	(g,d,10)	-	-	-	(g,h,7)
g(c,1)	-	-	-	(c,d,7)	-	-	-	(g,h,7)
d(c,7)	-	-	-	-	-	-	-	(g,h,7)
(g,h,7)	-	-	-	-	-	-	-	-

MST IS unique (这句话有分的，说明Prim算法生成的MST是唯一的)

注意点：(e,f,2)中的2是ef两点的距离而不是aef的3。手动模拟即可。e(a,1)意思是e距离前驱节点a的距离是1

Solution (b)

The MST of **Kruskal's Algorithm** is as follows...



The priority queue is as follows...

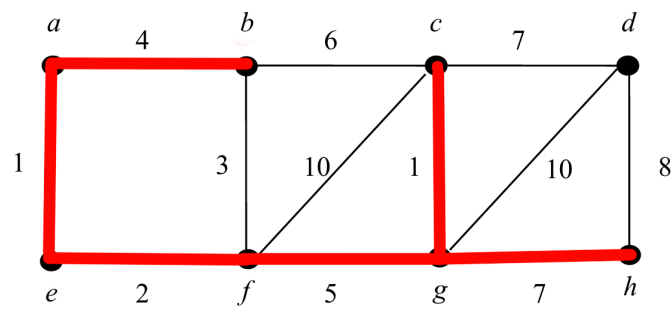
Points	Weight(Ascending Order)
[a,e]	1
[c,g]	1
[e,f]	2
[f,b]	3
[a,b] DELETED	4 DELETED
[f,g]	5
[b,c] DELETED	6 DELETED

Points	Weight(Ascending Order)
[c,d]	7
[g,h] DELETED	7 DELETED
[d,h] DELETED	8 DELETED
[f,c] DELETED	10 DELETED
[g,d] DELETED	10 DELETED

MST IS unique

Solution (C)

Dijkstra's Algorithm is as follows...



Priority queue is as follows...

Order Selected	a(0,-)	b(-,∞)	c(-,∞)	d(-,∞)	e(-,∞)	f(-,∞)	g(-,∞)	h(-,∞)
a(-,0)		b(a,4)	INF	INF	e(a,1)	INF	INF	INF
e(a,1)		b(a,4)	INF	INF		f(e,3)	INF	INF
f(e,3)		b(a,4)	c(f,13)	INF			g(f,8)	INF
b(a,4)			c(b,10)	INF			g(f,8)	INF
g(f,8)			c(g,9)	d(g,18)				h(g,15)
c(g,9)				d(c,16)				h(g,15)
h(g,15)				d(c,16)				
d(c,16)								

The selection order of edges are... a-e , f-e , a-b , f-g , c-g , h-g , c-d

然后会发现其实Dijkstra算法和Prim算法几乎是一样的

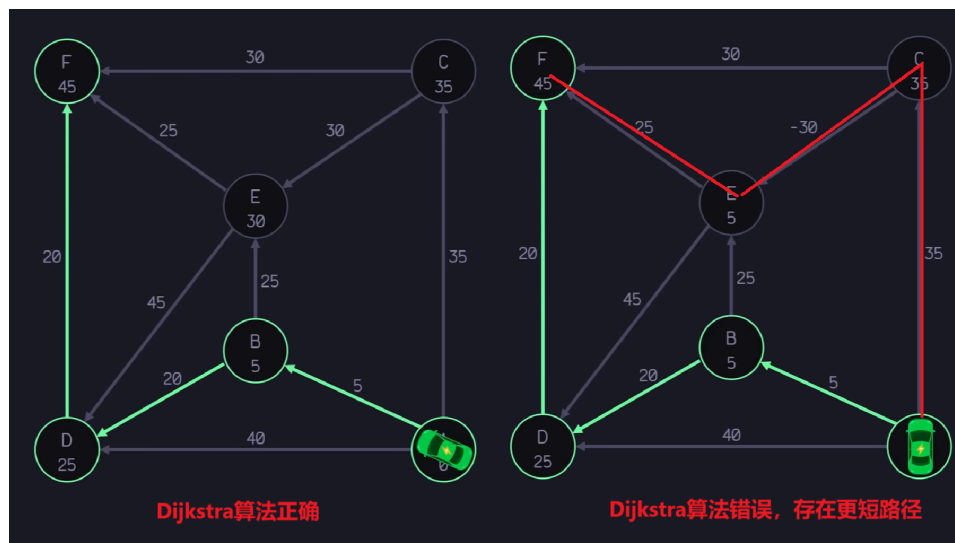
■ [Week 7]

[1] Bellman-Ford算法

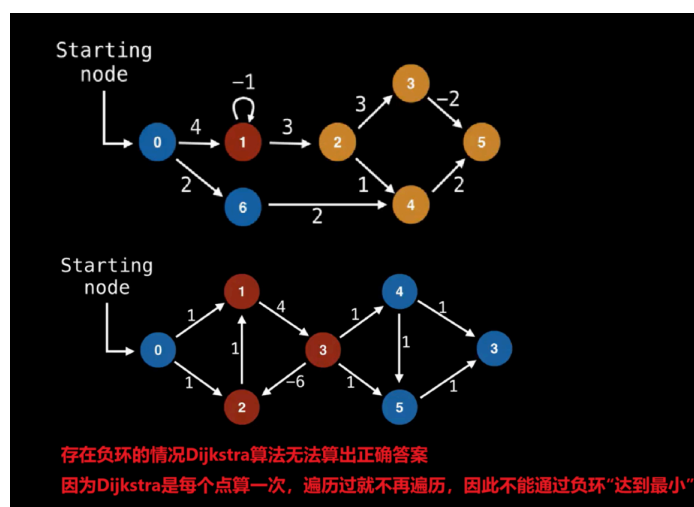
看 [这个视频!](#) 和 [这个视频!](#)

单源最短路径：从一个给定的点到其他点的最短路径。

Dijkstra 算法无法处理权重为负数、存在负环的图！但是Bellman-Ford算法可以。



再比如这个红色点存在负环，那么黄色点就可以到达时候达到无穷小。因此Dijkstra算法不适用（算出来不是正确的）。



然而Dijkstra算法时间复杂度是 $O(V^2)$ （朴素实现，如果是斐波那契堆优化则是 $O(V \log V + E)$ ），Bellman-Ford算法时间复杂度更高为 $O(EV)$ 。

Bellman-Ford算法很简单，首先规定一些变量：

1. **V** 代表Vertex，点的数组
2. **E** 代表Edge，边的数组
3. **S** 代表Start，起始点
4. **D** 代表Distance， $D[A]$ 代表起始点S到A的最短路径值

算法如下：

1. 把数组 **D** 值设为 **+Inf**，代表起始点S到其他每个点的最短路径值都是无穷大
2. 把 $D[S]$ 设置为 0 代表我们已经在这里了（起始点到起始点距离当然是0）

3. 对于每条边，执行**松弛操作**

4. 松弛操作如下

```
1 for i in v # i没有用到，只是作为循环，循环了顶点次数
2     for edge in graph # 循环每个边
3         # 更新最短路径
4         if(D[edge.from] + edge.value < D[edge.to])
5             D[edge.to] = D[edge.from] + edge.value
```

5. 环边检查，只需重复 **松弛操作**的改版：

```
1 for i in v # i没有用到，只是作为循环，循环了顶点次数
2     for edge in graph # 循环每个边
3         # 如果第二次还更新了证明这是带负数的环，因此更新为无穷小
4         if(D[edge.from] + edge.value < D[edge.to])
5             D[edge.to] = -INF
```

这个用直觉来看也是非常合理的！

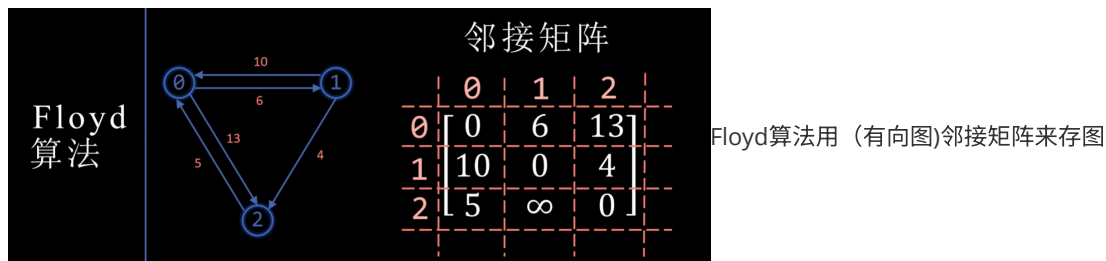
6. 注意点：这个算法并不需要按照特定边的顺序，而且，每次 `for edge in graph # 循环每个边` 语句执行完毕只有可能得到不一样的D数组。但是等到两个 `for` 循环都结束的时候，我们殊途同归。

[2] Floyd 算法

观看 [这个视频！](#)

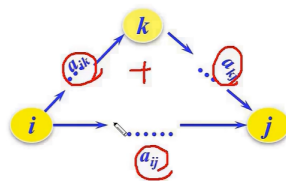
主要看那个distance表如何更新，绕行顶点表如何记录（不用全看完）

Floyd算法和Dijkstra算法不同，Floyd算法是专门求**任意两点**的最短路径的。但是它的时间复杂度很大，是 $O(n^3)$



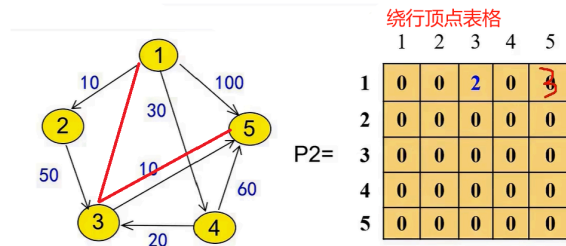
当然对于稀疏图，可以用哈希来存。

算法核心就是权重(`i` 到 `j`) 和 (`i` 到 `k` 加上 `k` 到 `j`) 进行比较



关于这个路径（绕行顶点）

绕行顶点表：Detour Vertex Table



这个表格存的是，例如我现在想找 1 到 5 的路径，除了原本的100，我发现一个更短的路径！

其中，绕行顶点表格 [1,3] 处的值是2，意味着我们通过顶点2绕行。 绕行顶点表格 [1,5] 意味着我们通过顶点3绕行。

那么如何获得路径？答案就是：

```

1  if(from_to > from_thru+thru_to)
2  {
3      // 需要更新！通过绕行Thru这个点，From 到 To 更短了
4      distance.put(new Node(From,To),from_thru+thru_to);
5      path.put(new Node(From,To),Thru);
6  }

```

当我们发现能够通过点 Thru 绕行让 From 到 To 更短的时候，我们就可以更新表格 [From,To] 为 Thru。

这个代码的地方用了哈希充当表格，但是原理是一样的。

[3] Warshall's 算法

适用于有向图、无向图。无向图ab相连那么矩阵(a,b)和(b,a)都标记为1即可。

这个算法用来计算传递闭包。就是上个学期学的线性代数和多元微积分那里提到的。

比如，A城市连通了B城市，B城市连通了C城市，那么请问AC城市连通吗？也就是问你，AC是不是传递闭包的。

有向图和无向图都是适用的。例如我们就考虑有向图 A->B, B->C，请问 A->C 吗？

[看这个视频！](#) 你就知道怎么算了。

n 个城市就迭代 n 次。例如下面 $n = 3$ 就要算到 $W^{(3)}$

$$\begin{aligned}
 W^{(0)} &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} & W^{(1)} &= \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} & W^{(2)} &= \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} & W^{(3)} &= \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}
 \end{aligned}$$

核心就是：求 W_i 就找出第 i 行数值为1的所有列 和 第 i 列数值为1的所有行，其交叉变为1。

传递闭包: 有向图, 无向图均适用.

沃舍尔算法: 核心: 求 M_{ij} , 就把第 i 行为 1 的列找出来,

第 j 列为 1 的行找出来.

交点的值变为 1

1 2 3 4.

$$\begin{array}{l} \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} \\ M_0 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}, M_1 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}, M_2 = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \\ M_3 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}, M_4 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \end{array}$$

■ [Week 8]

字符串搜索算法

1. 空间换时间的概念
2. 分布计数排序算法
3. Horspool字符串搜索算法

Quiz：假如你需要写一个交换两个 `int` 的代码段，你会怎么写？

```
1 | int a = 4;
2 | int b = 6;
```

写法1：

```
1 | {
2 |     int temp = a;
3 |     int a = b;
4 |     int b = temp;
5 | }
```

写法2：

```
1 | {
2 |     a = a - b; // -2, 这个是a和b的差值
3 |     b = a + b; // b 变成了a, b = 4
4 |     a = b - a; // a 变成了b, a = 6
5 | }
```

写法1显然是更为常见的，但是写法2就不会创建出第三个变量 `temp`。

[1] 空间换时间

Space for time

两种空间换时间算法：

- **输入增强**：对输入（或其部分）进行预处理，存储一些信息，以便后续解决问题时使用。

输入增强的两种算法如下：

1. 分布计数排序
2. Horspool 字符串搜索算法

- **预结构化**：使用一种数据结构对输入进行预处理，使访问其元素更加容易。

之前讲到的排序算法都是基于**比较**的，最快的是分治排序，其中分治排序包含了归并排序和快速排序。这两种排序都有 $O(n \log n)$ 的速度。但是这就是最快的排序了吗？

只要我们基于比较排序， $O(n \log n)$ 确实就是最快的了。这是可以被证明的。

但是如果不用比较排序呢？

[2] 计数排序

counting sort

计数排序**不需要任何比较**，观看这个[两分钟的计数排序视频](#)。


```

4  }
5  public static int getIndex(String text, String pattern) {
6      if(text == null || pattern == null) return -1;
7      if(text.length() < pattern.length()) return -1;
8
9      HashMap<Character, Integer> map = getBadMatchTabel(pattern);
10     int index = pattern.length()-1;
11     while(index <= text.length()-1)
12     {
13         boolean equals = true;
14         int patternPointer = pattern.length()-1;
15         int textPointer = index;
16         while(patternPointer >= 0 && equals)
17             if(text.charAt(textPointer) == pattern.charAt(patternPointer))
18                 {textPointer--;patternPointer--;}
19             else
20                 equals = false;
21
22         if(equals)
23             return index - pattern.length() + 1;
24         else
25             index += map.getOrDefault(text.charAt(index), pattern.length());
26     }
27     return -1;
28 }
29 private static HashMap<Character,Integer> getBadMatchTabel(String pattern) {
30     HashMap<Character,Integer> map = new HashMap<>();
31     int lengthOfPattern = pattern.length();
32     for(int index = 0; index < lengthOfPattern-1; index++)
33         map.put(pattern.charAt(index), lengthOfPattern - index -1);
34     map.putIfAbsent(pattern.charAt(lengthOfPattern-1), lengthOfPattern);
35     return map;
36 }

```

除了 Horspool 算法之外，还有 KMP 算法，可以查看 [Algorithm.pdf](#)。

测试：14亿长度的字符串中（1407483000）查找一个姓名（用9长度来表示），耗时多久（假设目标索引是在末尾100000位左右）？

```

Java自带搜索[Time = 0.947 s]
Horspool搜索[Time = 1.99 s]
KMP搜索[Time = 0.0 s]此方法超出内存
暴力搜索[Time = 3.039 s]

Process finished with exit code 0

```

可见Horspool比朴素搜索要快上很多

■ [Week 9]

动态规划

[1] 最长公共子序列

Longest Common Subsequence (LCS)

运用动态规划的思想，找到状态转移方程。

请参考[力扣这一题](#)

1143. 最长公共子序列

已解答 

中等

 相关标签

 相关企业

 提示

Ax

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长 **公共子序列** 的长度。如果不存在 **公共子序列**，返回 `0`。

一个字符串的 **子序列** 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

- 例如，`"ace"` 是 `"abcde"` 的子序列，但 `"aec"` 不是 `"abcde"` 的子序列。

两个字符串的 **公共子序列** 是这两个字符串所共同拥有的子序列。

示例 1：

输入：`text1 = "abcde"`，`text2 = "ace"`

输出：3

解释：最长公共子序列是 `"ace"`，它的长度为 3。

示例 2：

输入：`text1 = "abc"`，`text2 = "abc"`

输出：3

解释：最长公共子序列是 `"abc"`，它的长度为 3。

示例 3：

输入：`text1 = "abc"`，`text2 = "def"`

输出：0

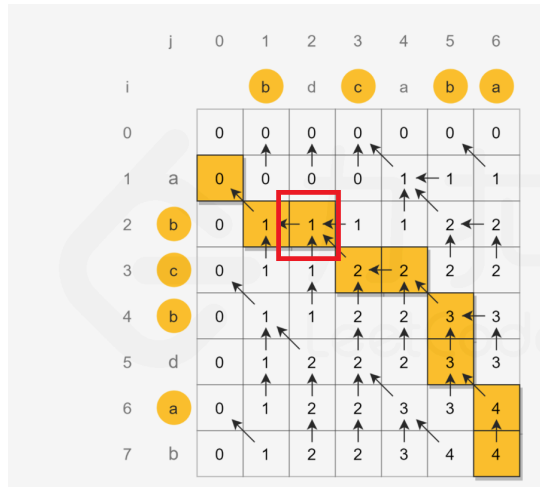
解释：两个字符串没有公共子序列，返回 0。

设函数 $F(x, y)$ 意思是 `text1` 长度为 x 的子串和 `text2` 长度为 y 的子串的最大公共子序列

显然，如果 `text1[x-1] == text2[y-1]`（也就是这两个子串最后一个字符相等）状态可以转移为

$$F(x, y) = F(x - 1, y - 1) + 1 \quad \text{if } \text{text}_1[x-1] = \text{text}_2[y-1]$$

如果不相等，就要求 $\max(F(x - 1, y), F(x, y - 1))$ ，为什么？请看这个例子：



一般而言，如果不匹配可以直接等于上一次的结果。但是如果本行中有数据更新（例如红圈部分前一个数据更新了！就不能用上一个数据了），就要用更新的数据！所以需要Max函数比较。

总体的状态转移方程：

$$F(x, y) = \begin{cases} F(x-1, y-1) + 1 & \text{if } \text{text}_1[x-1] = \text{text}_2[y-1] \\ \max(F(x-1, y), F(x, y-1)) & \text{if } \text{text}_1[x-1] \neq \text{text}_2[y-1] \end{cases}$$

代码也很简洁：

```

1  class Solution {
2      public int longestCommonSubsequence(String text, String pattern) {
3          int[][] F = new int[pattern.length() + 1][text.length() + 1];
4          for(int x = 1; x <= pattern.length(); x++)
5              {
6                  for(int y = 1; y <= text.length(); y++){
7                      if(text.charAt(y-1) == pattern.charAt(x-1))
8                          F[x][y] = F[x-1][y-1] + 1;
9                      else
10                         F[x][y] = Math.max(F[x-1][y], F[x][y-1]);
11                  }
12              }
13          return F[pattern.length()][text.length()];
14      }
15  }

```

[2] 序列比对动态规划矩阵[生物信息学]

Alignment：比对

全局比对

Global Alignment

一个和课件几乎完全相同的[视频](#)（链接在视频 9：50s空降）

这个视频中的方法是全局最优解

其动态规划思想是：对于每一个F（x，y）

XYZ or XY- or XYZ
ABC ABC AB-

选取左边、上边、左上边的最好的情况来判断

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \end{cases}$$

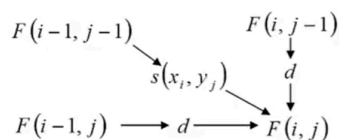
把左边、上边的路径看作 gap（空白位）

$s(x, y)$ 是评分， d 是 gap penalty（空位惩罚分）

只需要记住这个状态转移方式和 s 、 d 的含义即可

DP for sequence alignment: Example

	A	C	G	T
A	2	-7	-5	-7
C	-7	2	-7	-5
G	-5	-7	2	-7
T	-7	-5	-7	2



Find the optimal alignment of AAG and AGC.
Use a linear gap penalty of $d=-5$.

		A	A	G
	0	-5	-10	-15
A	-5	2	-3	-8
G	-10	-3	-3	-1
C	-15	-8	-8	-6

那些箭头就是可回溯的路径。代表这个最终结果-6是怎么来的。回溯的路径必须是右下角走到左上角

总而言之：左上角看表格，左、上用 gap 惩罚分。

初始化：这个表格第1个位置是空的。然后横纵方向分别使用gap惩罚填写。填写好之后就可以开始正式运行这个算法了。

局部比对

Local alignment

[同款视频](#)

$$F(i, j) = \max \begin{cases} F(i-1, j-1) + s(x_i, y_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \\ 0 \end{cases}$$

局部最优解是全局最优解的基础上加了一个0作为最小限定（因为左上角就是0，0是最小值）。

局部最优解不要求从右下角走到左上角，而是任意的两个位置都可以（选择最大值）。

		H	E	A	G	A	W	G	H	E	E
	0	0	0	0	0	0	0	0	0	0	0
P	0	0	0	0	0	0	0	0	0	0	0
A	0	0	0	5	0	5	0	0	0	0	0
W	0	0	0	0	2	0	20	12	4	0	0
H	0	10	2	0	0	0	12	18	22	14	6
E	0	2	16	8	0	0	4	10	18	28	20
A	0	0	8	21	13	5	0	4	10	20	27
E	0	0	6	13	18	12	4	0	4	16	26

Optimal local alignment

AWGHE
AW-HE

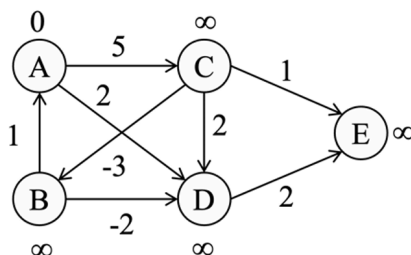
Highest score

全局比对和局部比对是难点。后面做题会有更详细的解释。

[3] Tutorial

Question 1

Apply Bellman-Ford algorithm to find the shortest paths from the source to all other vertices. G is as following and A is the source vertex.



求Bellman-Ford算法下的最短路径。回顾Bellman-Ford算法：

```

1  for i in v # i没有用到，只是作为循环，循环了顶点次数
2      for edge in graph # 循环每个边
3          # 更新最短路径
4          if(D[edge.from] + edge.value < D[edge.to])
5              D[edge.to] = D[edge.from] + edge.value

```

因此：我们列出所有的边，分别是：

1. A 到 C 距离 5
2. A 到 D 距离 2
3. B 到 A 距离 1
4. B 到 D 距离 -2
5. C 到 B 距离 -3
6. C 到 D 距离 2
7. C 到 E 距离 1
8. D 到 E 距离 2

然后因为顶点 $|V| = 5$ 因此要执行5轮更新

注意Bellman-Ford算法要迭代 $|V|$ 次！并且记录的距离是叠加的，距离参照当前迭代数据而非前一次数据。

	Initial Distance	1th Distance	2nd Distance	3rd Distance	4th Distance	5th Distance
A	0	0	0	0	0	0
B	∞	∞	2 From C	2 From C	2 From C	2 From C
C	∞	5 From A	5 From A	5 From A	5 From A	5 From A
D	∞	2 From A	0 From B	0 From B	0 From B	0 From B
E	∞	4 From D	2 From D	2 From D	2 From D	2 From D

根据 5th Distance 就可以找到最短路径了：

- E 到 A 的最短路径是 2：E--D--B--C--A
- D 到 A 的最短路径是 0：D--B--C--A
- C 到 A 的最短路径是 5：C--A
- B 到 A 的最短路径是 2：B--C--A
- A 到 A 的最短路径是 0：A--A

Question 2

Assuming that the set of possible list is $\{a, b, c, d\}$, sort the following list in alphabetical order by the counting algorithm:

b, c, d, c, b, a, a, b

```
int[] count = new int[4];
```

逐个计数之后：

```
count[0] = 2、count[1] = 3、count[2] = 2、count[3] = 1
```

Question 3

Consider the problem of searching for genes in DNA sequences using Horspool's algorithm. A DNA sequence is represented by a text on the alphabet $\{A, C, G, T\}$, and the gene or a gene segment is a pattern.

3A. Construct the shift table for the following gene segment.

TCCTATTCTT

3B. Apply Horspool's algorithm to locate the pattern in the following DNA sequence.

TTATAGATCTGGTATTCTTTTATAGATCTCCTATTCTT

回顾 Horspool's 算法

其table的核心就是

$$\text{value} = \text{length} - \text{index} - 1$$

并且

1. 最后一个字母：如果已存在，离开。不存在，值为value

2. 表格中添加 * 来代表不匹配的情况，值为 length

举个例子例如Pattern为 `TOOTHEO` 那么最后一个字母为 `O`，因为已存在所以不需要管他，结束

再例如 `TOOTH`，这里最后一个字母 `H` 并不存在表中，因此它的值为 length

T

O

O

T

H

0

1

2

3

4

Length = 5

Letter	T	O	H	*
Value	1	2	5	5

在这个题目中的 Pattern 是 `TCCTATTCTT`，构建table如下

	Value	Index (Final)
A	5	4
C	2	7
T	1	8
*	10 (如果不匹配)	

然后应用这个算法，就能得到（当然是自己看会比模拟算法快得多）

TTATAGATCTGGTATTCTTTTATAGATC

TCCTATTCTT

TCCTATTCTT

t	t	a	t	a	g	a	t	c	t	g	g	t	a	t	t	c	t	t	t	a	t	a	g	a	t	c	t	c	c	t	a	t	t	c	t	t
t	c	c	t	a	t	t	c	t	t																											
	t	c	c	t	a	t	t	c	t	t																										
											t	c	c	t	a	t	t	c	t	t																
												t	c	c	t	a	t	t	c	t	t															
																		t	c	c	t	a	t	t	c	t	t									
																		t	c	c	t	a	t	t	c	t	t									
																				t	c	c	t	a	t	t	c	t	t							
																					t	c	c	t	a	t	t	c	t	t						
																						t	c	c	t	a	t	t	c	t	t					
																											t	c	c	t	a	t	t	c	t	t

Question 4 [重要]

在这题的Solution中给的箭头是指向From的，也就是指向左上角的。这里的箭头我直接是按照手写习惯写的。

Using a gap penalty of $d = -1$ and scoring matrix as below

	A	C	G	T
A	1	-3	-2	-3
C	-3	1	-3	-2
G	-2	-3	1	-3
T	-3	-2	-3	1

1. Optimal global alignment

a. Using dynamic programming, fill in the table in computing the score of the optimal global alignment of GAGT and ACATGT.

b. Based on the table, find all the optimal global alignments of GAGT and ACATGT.

2. Optimal local alignment

a. Using dynamic programming, fill in the table in computing the score of the optimal local alignment of GAGT and ACATGT

b. Based on the table, find all the optimal local alignments of GAGT and ACATGT.

对于第一问全局最优解

		A	C	A	T	G	T	
		0	-1	-2	-3	-4	-5	-6
G		-1	-2	-3	-4	-5	-3	-4
A		-2	0	-1	-2	-3	-4	-5
G		-3	-1	-2	-3	-4	-2	-3
T		-4	-2	-3	-4	-2	-3	-1

评分方案：- 若所有内容（数字 + 箭头）均正确（一个单元格中可能同时包含向左和向上的箭头，这种情况视为正确），得10分。- 每出现一处错误内容，扣0.5分，最多扣10分。

标准答案中的箭头是指向左上角的!!! 标准答案箭头方向如下:

		0	1	2	3	4	5	6
			A	C	A	T	G	T
0		0	-1	-2	-3	-4	-5	-6
1	G	-1	-2	-3	-4	-5	-6	-7
2	A	-2	-1	-2	-3	-4	-5	-6
3	G	-3	-1	-2	-3	-4	-5	-6
4	T	-4	-2	-3	-4	-5	-6	-7

因为需要到达左上角，有很多路线：

(从0, 0开始，往右就是左边串多加一个 - ，往下就是上边串多加一个 - ，斜线就是上下串匹配)

(简单记住：垂直就是垂直那个串（上串）多加一个 - ，水平就是水平那个串（左边）多加一个 -)，这些都是gap。

路线1：

		A	C	A	T	G	T
	0	-1	-2	-3	-4	-5	-6
G	-1	-2	-3	-4	-5	-6	-7
A	-2	0	-1	-2	-3	-4	-5
G	-3	-1	-2	-3	-4	-5	-6
T	-4	-2	-3	-4	-5	-6	-7

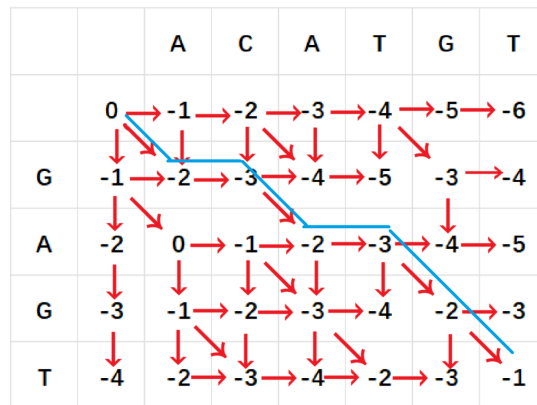
A C - A T G T
 - - G A - G T

路线2：

		A	C	A	T	G	T
	0	-1	-2	-3	-4	-5	-6
G	-1	-2	-3	-4	-5	-6	-7
A	-2	0	-1	-2	-3	-4	-5
G	-3	-1	-2	-3	-4	-5	-6
T	-4	-2	-3	-4	-5	-6	-7

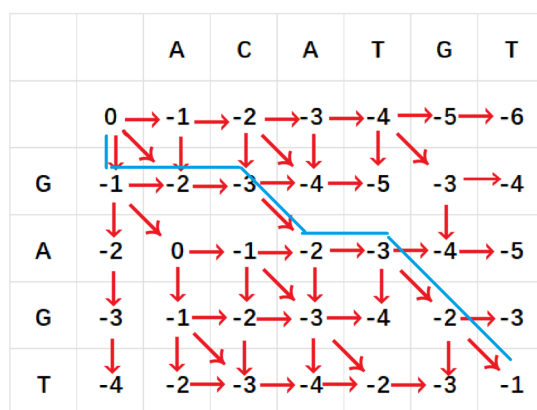
A - C A T G T
 - G - A - G T

路线3：



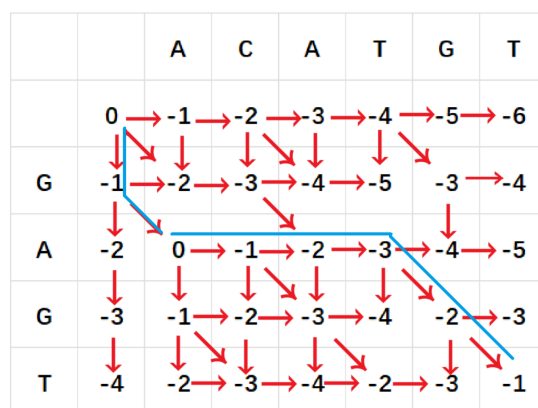
A C A T G T
G - A - G T

路线4:



- A C A T G T
G - - A - G T

路线5:



- A C A T G T
G A - - - G T

- 回溯过程描述正确，得 1 分。
- 给出上述比对结果之一，得 2 分。
- 给出表格中对应的路径且正确，加 2 分。

对于第二问局部最优解

		A	C	A	T	G	T
	0	0	0	0	0	0	0
G	0	0	0	0	0	1	0
A	0	1	0	1	0	0	0
G	0	0	0	0	0	1	0
T	0	0	0	0	1	0	2

评分标准：

- 若所有内容（数字和箭头）均正确（一个单元格中可同时包含向左和向上的箭头，这种情况视为正确），得10分。
- 每一处错误内容扣0.5分，最多扣10分。

只有一条路线

		A	C	A	T	G	T
	0	0	0	0	0	0	0
G	0	0	0	0	0	1	0
A	0	1	0	1	0	0	0
G	0	0	0	0	0	1	0
T	0	0	0	0	1	0	2

A T G T
A - G T

注意：只有非0数字才能像右、下延申以及被左上角延申。

Question 5

Suppose there are 10 people in a room. Each person shakes hands with some other people in the room. Prove that the number of people having an odd number of handshakes is even.

(Challenge: This puzzle is equivalent to the question in an undirected graph, “prove that the number of vertices with odd degree is even”. Try to think why the two questions are equivalent.)

假设房间里有10个人。每个人都和房间里的其他人握手。证明握手次数为奇数的人数是偶数个呢。（拓展：这个问题等价于无向图中的问题“证明度数为奇数的顶点数量是偶数”。思考一下为什么这两个问题是等价的。）

假设 A 和 B 握手，认为在一个无向图 G 中， $A \rightarrow B$ 。这时候顶点 A 的度数加 1。

因为在无向图中，所有顶点的度数之和等于边的两倍，也就是，所有顶点的 $\sum d(V) = 2 \times |E|$

可以展开：

$$\sum d_{\text{odd}}(V) + \sum d_{\text{even}}(V) = 2 \times |E|$$

要证明 **握手次数为奇数的人数是偶数** 转变为：

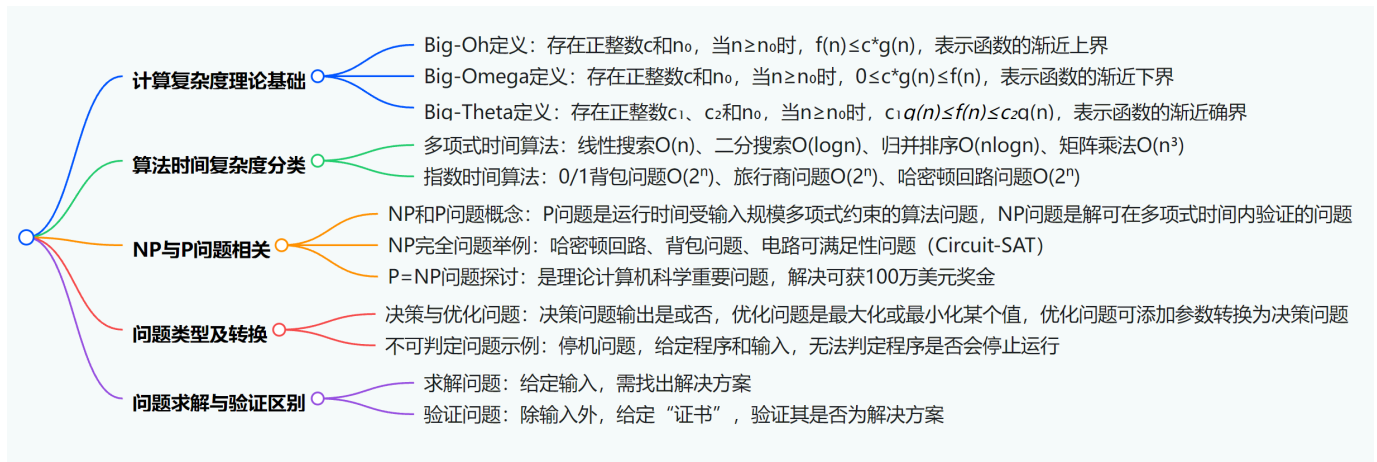
证明 **度为基数的点有偶数个**

因为所有点的**度**加起来是偶数（因为 $2 \times |E|$ ），因此：要么 $\sum d_{\text{odd}}(V)$ 和 $\sum d_{\text{even}}(V)$ 是俩奇数、要么是俩偶数，他们相加才可能是偶数。

再分析。对于 $\sum d_{\text{even}}(V)$ 中的点而言，这些点的度都是偶数，因此不管这些点是奇是偶数个， $\sum d_{\text{even}}(V)$ 总是偶数（奇、偶数 $\times$ 偶数=偶数）

因此， $\sum d_{\text{odd}}(V)$ 也只能是偶数，所以，对于度为基数的点，这些点的数量也是偶数个才行。（奇数 $\times$?=偶数呢，很显然是偶数！）

■ [Week 10]



[1] 复杂性定义

大 O 表示法

回顾其定义：

$$f(n) = O(g(n))$$

如果存在 c 和 n_0 对于所有的 $n \geq n_0$ 使得

$$f(n) \leq cg(n)$$

成立。

大 Ω 表示法

Big-Omega

定义：

$$\Omega(g(n)) = f(n)$$

如果存在 c 和 n_0 对于所有的 $n \geq n_0$ 使得

$$0 \leq cg(n) \leq f(n)$$

成立。

这两种方法的区别？

大O表示法示例

考虑函数 $f(n) = 3n^3 + 2n^2 + 5n + 1$ 。

- 大O表示法关注函数在 n 趋于无穷大时的上界情况。当 n 足够大时，最高次项起主导作用。
- 对于 $f(n) = 3n^3 + 2n^2 + 5n + 1$ ，我们要找一个函数 $g(n)$ ，以及正常数 c 和 n_0 ，使得 $n \geq n_0$ 时， $f(n) \leq c \cdot g(n)$ 。
- 当 $n \geq 2$ 时：

$$\begin{aligned} 3n^3 + 2n^2 + 5n + 1 &\leq 3n^3 + n^3 + n^3 + n^3 \\ &= 4n^3 \end{aligned}$$

所以取 $n_0 = 2$ ， $c = 4$ 即可。

$f(n) = 3n^3 + 2n^2 + 5n + 1 = O(n^3)$ 。这表明 $f(n)$ 的增长速度不会超过 n^3 的某个常数倍。

大Ω表示法 (Big - Omega Notation) 示例

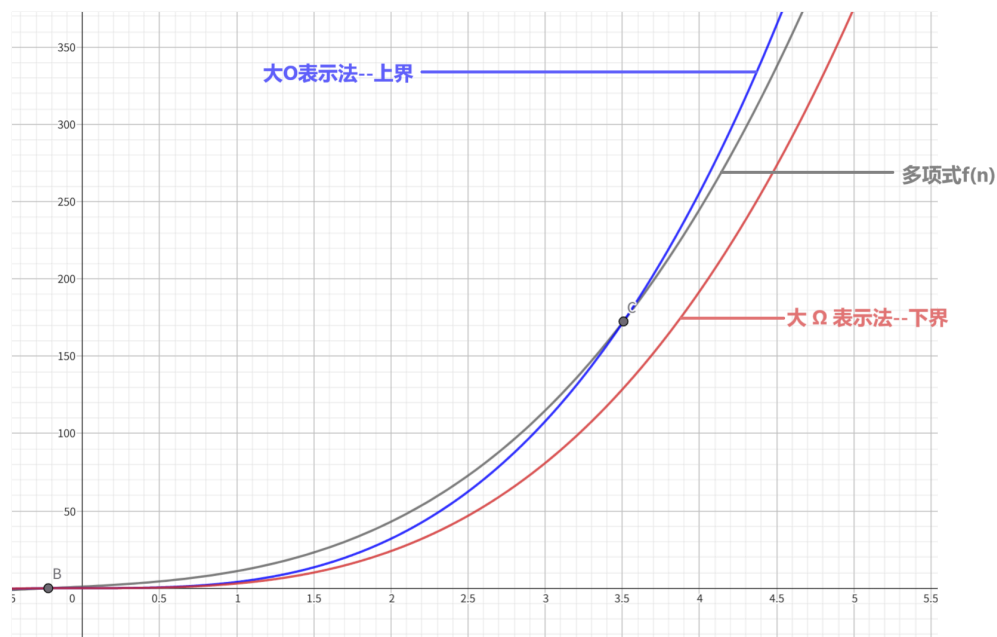
同样考虑函数 $f(n) = 3n^3 + 2n^2 + 5n + 1$ 。

- 大Ω表示法关注函数在 n 趋于无穷大时的下界情况。即要找一个函数 $h(n)$ ，以及正常数 c 和 n_0 ，使得 $n \geq n_0$ 时， $0 \leq c \cdot h(n) \leq f(n)$ 。
- 当 $n \geq 1$ 时：

$$f(n) = 3n^3 + 2n^2 + 5n + 1 \geq 3n^3$$

所以取 $c = 3$ ， $n_0 = 1$ 即可（之前说过大 O 表示法的 c 可以取常数+1，那么这里直接取常数就行了）

$f(n) = 3n^3 + 2n^2 + 5n + 1 = \Omega(n^3)$ 。这表明 $f(n)$ 的增长速度至少是 n^3 的某个常数倍。



大 Θ 表示法

大Theta表示法 (Big - Theta Notation) 综合了大O表示法和大Omega表示法的特性，给出函数的确切渐近界。

以函数 $f(n) = 3n^3 + 2n^2 + 5n + 1$ 为例：

- 定义及数学表示：** 设 $f(n)$ 和 $g(n)$ 是定义在自然数集上的函数，若存在正常数 c_1 、 c_2 和自然数 n_0 ，使得当 $n \geq n_0$ 时，满足 $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ ，则称 $f(n)$ 是 $g(n)$ 的 $\Theta(g(n))$ ，即 $f(n)$ 与 $g(n)$ 同阶。
- 结合例子分析：** 对于 $f(n) = 3n^3 + 2n^2 + 5n + 1$ ，当 n 足够大时，最高次项 $3n^3$ 起主导作用。我们取 $g(n) = n^3$ ，可以找到正常数 $c_1 = 3$ 、 $c_2 = 4$ ，以及 $n_0 = 2$ 。当 $n \geq 2$ 时， $3n^3 \leq 3n^3 + 2n^2 + 5n + 1 \leq 4n^3$ 。所以 $f(n) = 3n^3 + 2n^2 + 5n + 1 = \Theta(n^3)$ 。这表明 $f(n)$ 的增长速度既不会低于 n^3 的某个常数倍（满足大Omega表示法意义下的下界），也不会高于 n^3 的某个常数倍（满足大O表示法意义下的上界），确切地刻画了 $f(n)$ 的渐近增长特性。
- 很显然，这里取 c 的方法就是取常数、以及常数+1，然后推导出 n_0 就好了。

大 Θ 表示法提供了对算法复杂度更精确的描述，相比大O表示法只给出上界，它同时兼顾了上下界，能更准确反映算法在平均情况和最坏情况下的时间或空间复杂度。

[2] 应用这三个复杂性

比如冒泡排序。我们知道他最好的情况就是 $O(n)$ ，一般情况是 $O(n^2)$ ，最坏的情况是 $O(n^2)$ 。

对应的起始就是：

- 大 O 表示法：最坏情况（上界）—— $O(n^2)$

$$f(n) = \frac{n(n-1)}{2}$$

- 大 Θ 表示法：一般情况—— $O(n^2)$

$$f(n) = \frac{n(n-1)}{2}$$

- 大 Ω 表示法：最好情况—— $O(n)$

$$f(n) = n - 1$$

[3] 目前我们所学内容

算法	时间复杂度
线性搜索（多项式时间）	$O(n)$
二分搜索（多项式时间）	$O(\log n)$
归并排序（多项式时间）	$O(n \log n)$
矩阵乘法（多项式时间）	$O(n^3)$
0/1背包问题（指数时间）	$O(2^n)$
旅行商问题（指数时间）	$O(2^n)$
哈密顿回路问题（指数时间）	$O(2^n)$

[4] 难解的计算问题

P 与 NP 问题

- 被克莱数学研究所列为七大“千禧年问题”之一。
- 解决该问题可让你获得100万美元奖金（并在数学和计算机科学史上留名）。
- 欲了解更多信息，请访问www.claymath.org/millennium/（选择“P与NP问题”链接）。

如果一个算法的运行时间受其输入规模的多项式约束，那么该算法是高效的（efficient）。

有些计算问题似乎很难解决：尽管进行了大量尝试，我们仍未找到针对这些问题的高效算法。

例如国际象棋某一个状态下的最优路线，很难评估其多项式约束是怎么样的。

我们也远未能证明这些问题确实难以解决。

用更正式的语言来说，我们不知道NP是否等于P。这是理论计算机科学中一个重要的基本问题！

哈密顿回路问题(Hamiltonian Circuit)、背包问题(Knapsack Problem)、电路可满足性问题(Circuit-SAT) 都属于P与NP问题。因为例如哈密顿回路问题的步骤不是多项式级别的，而是指数级别的。

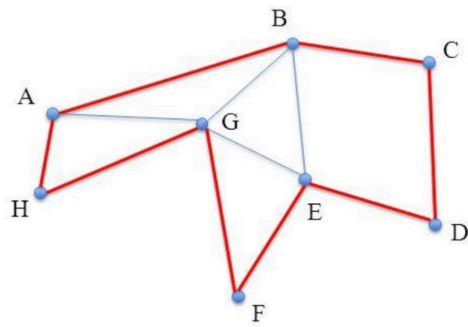
[5] 哈密顿回路

Hamiltonian Circuit

输入：一个连通图G

问题：图G是否存在哈密顿回路？

即，图G是否存在这样一条回路：除起始和终止顶点外，恰好经过每个顶点一次？



A - H - G - F - E - D - C - B - A

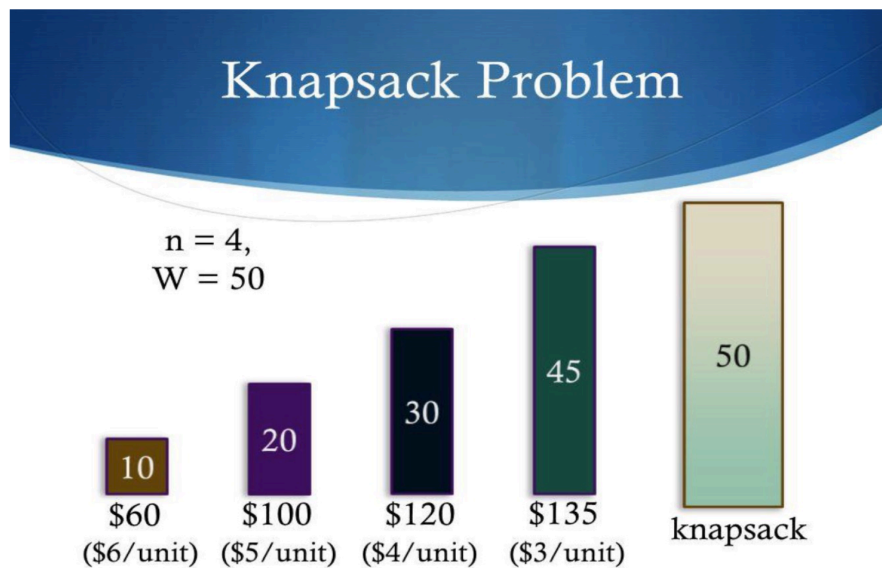
[6] 背包问题

Knapsack Problem

输入：给定 n 个物品，其整数重量分别为 $w_1, w_2, \dots, w_n$ ，整数价值分别为 $v_1, v_2, \dots, v_n$ ，以及一个容量为 W 的背包。

问题（优化版本）：找出物品的一个子集，使其总重量不超过 W ，且总价值最大化。（不允许取物品的部分）

也称为**0/1背包问题**（0/1 Knapsack Problem）

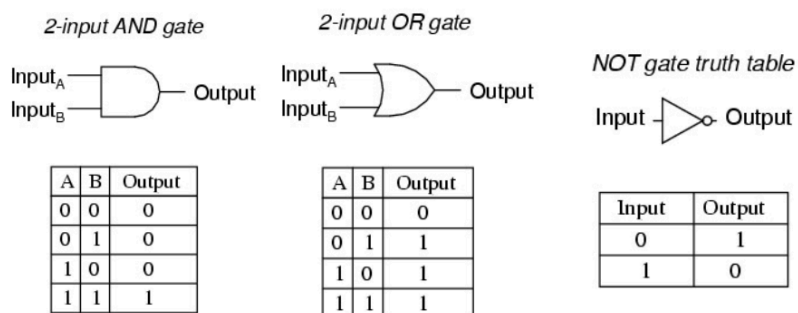


[7] 电路可满足性问题

Circuit-SAT

前置知识：布尔电路

Boolean Circuit



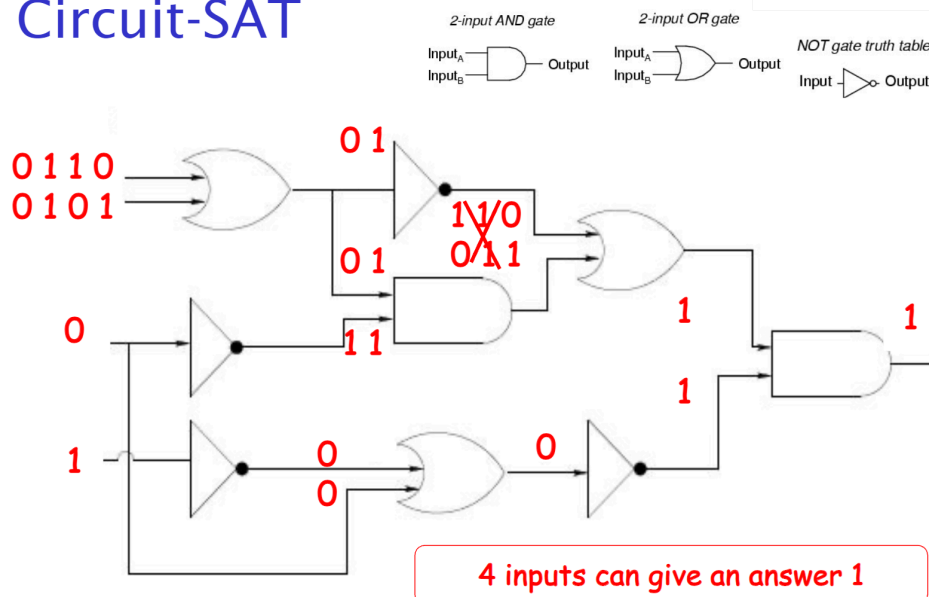
主问题

Circuit-SAT (SAT代表可满足性)

输入：一个具有单个输出顶点的布尔电路

问题：是否存在一种对输入赋值的方式，使得输出值为1？

Circuit-SAT



[8] 为什么研究 P 与 NP 问题

1. **理解计算限度：**知晓一个问题属于P类还是NP类，有助于理解解决某些问题的内在难度，以及随着输入规模增大，这些问题能否切实得到解决。
2. **算法开发：**如果一个问题属于P类，通常能高效解决，这会推动有效的算法开发。如果问题是NP完全或NP难的，尽管目前可能没有多项式时间解决方案，但它会促使人们去寻找能在实际中较好解决问题的近似算法或启发式算法。
3. **P 与 NP问题：**这是七大千禧年难题之一，克莱数学研究所为正确答案提供100万美元奖金。确定P是否等于NP对于理解计算的限度至关重要。

[9] 更多 P 与 NP 问题

判定/优化问题

Decision/Optimisation problems

判定问题是一种计算问题，其输出为“是”或“否”。

在优化问题中，我们试图最大化或最小化某个值。

如果给优化问题添加一个参数k，询问该优化问题的最优值是否至多（或至少）为k，那么优化问题就可以转化为判定问题。

注意，如果一个判定问题是**难解的**，那么与之相关的优化版本问题也必定是**难解的**。

简而言之，如果你给我一个数独的答案，让你确定这个是否为最优答案。你只需要遍历一遍，做一些判断，很快就能判断出来这个是否是最优答案。因此这个问题并不是难解的。

但是，如果我给你一个国际象棋的某个状态，告诉你某个路线是最优路线，那么你怎么确定呢？你只能再不断验证演算，才能判定这个路线是否是最优的。因此，既然这个判定是难解的，那么其优化版本问题也是难解的。

判定/优化问题例子1：最小生成树

MST - Minimum Spanning Tree

优化问题：给定一个边带有整数权重的图G，图G中最小生成树（MST）的权重是多少？

判定问题：给定一个边带有整数权重的图G以及一个整数 k ，图G是否存在一棵权重至多为 k 的最小生成树？

同样，你要判断是否存在权重为 k 的MST，你目前只能通过优化问题来得出结果

判定/优化问题例子2：背包问题

输入：给定 n 个物品，其整数重量分别为 $w_1, w_2, \dots, w_n$ ，整数价值分别为 $v_1, v_2, \dots, v_n$ ，一个容量为 W 的背包，以及一个数值 k 。

优化问题：找出物品的一个子集，使其总重量不超过 W ，且总价值最大化。

判定问题：对于任意整数 k ，是否存在一个物品子集，其总重量不超过 W ，且总价值至少为 k ？

练习：判定/优化问题

阐述下列问题的判定版本：

给定一个带权图G和源顶点a，找出从a到其他每个顶点的最短路径。

判定问题：给定一个带权图G、一个源顶点a和一个数值 k ，是否存在从a到其他每个顶点的最短路径，且每条路径的权重至多为 k ？

求解一个判定问题如何有助于找到优化问题的解决方案？

所有判定问题都能通过算法解决吗

- 答案是否定的。
- 无法通过算法解决的问题被称为**不可判定问题**。
- 停机问题就是这样一个例子（由艾伦·图灵在1936年提出）：“给定一个计算机程序及其输入，判断该程序在这个输入下是会停机，还是会无限运行下去。”

关于停机问题不可判定的证明虽然篇幅不长，但理解起来有难度。让我们看看证明是怎么样的：

停机问题证明概要

Halting Problem

假设存在一个**万能**的算法A，对于任意程序P和输入I：

$$A(P, I) = \begin{cases} 1, & \text{如果输入I在程序P上停机} \\ 0, & \text{如果输入I在程序P上不停机} \end{cases}$$

现在需要证明这个算法是否存在，不妨先假设它存在！

我们构建一个程序程序Q(Q被称为否定者)

$$Q(P) = \begin{cases} \text{停机}, & \text{如果 } A(P, P) = 0, \text{ 即 } P \text{ 在输入 } P \text{ 上不停机} \\ \text{不停机}, & \text{如果 } A(P, P) = 1, \text{ 即 } P \text{ 在输入 } P \text{ 上停机} \end{cases}$$

现在程序自己传入自己：

$$Q(Q) = \begin{cases} \text{停机}, & \text{如果 } A(Q, Q) = 0, \text{ 即 } Q \text{ 在输入 } Q \text{ 上不停机} \\ \text{不停机}, & \text{如果 } A(Q, Q) = 1, \text{ 即 } Q \text{ 在输入 } Q \text{ 上停机} \end{cases}$$

这里A算错了。如果 $A(Q, Q)$ 结果是0，那么应当不停机，但是Q停机了。

如果 $A(Q, Q)$ 结果是1，那么应当停机，但是Q没停机了。

查看[这个视频](#)了解这个过程。

解决问题与验证问题不同

- **解决问题**：给定一个输入，我们必须找到解决方案。
- **验证问题**：除了输入之外，还会给定一个“**凭证**”，我们要验证该凭证是否确实是一个解决方案。

我们可能不知道如何高效地解决一个问题，但我们可能知道如何验证一个候选对象是否真的是解决方案。

示例 - 哈密顿回路问题

假设通过某种（未明确说明的）方式（比如凭良好的猜测），我们找到了一个哈密顿回路的候选对象，也就是在输入图G中，一个可能构成哈密顿回路的顶点和边的列表。

要检查这是否真的是一个哈密顿回路很容易。需检查：

- 所有列出的边都在图G中；
- 它确实构成一个环；
- 它恰好访问了图G中的每个顶点一次。

如果这个候选解确实是一个哈密顿回路，那么它就是一个凭证，用以证明该判定问题的答案是“是”。

示例 - 0/1背包问题

考虑一个0/1背包问题（判定版本）的实例。

假设有人给出一个物品子集，很容易检查：

- 这些物品的总重量是否至多为W；
- 总价值是否至少为k。

· 如果这两个条件都满足，那么这个物品子集就是该判定问题的一个“凭证”。

- 也就是说，它证明了0/1背包判定问题的答案是“是”。

示例 - 电路可满足性问题（Circuit - SAT）

考虑一个布尔电路。

假设有人给出一组输入，很容易检查：

- 这些输入值是否能使输出最终值为1，
- 这通过检查每个逻辑门来实现。

如果输入真值能使最终输出值为1，那么这些值就构成了该判定问题的一个“凭证”。



[10] 复杂度类P 和 NP

复杂度类P是指所有在最坏情况下能够在多项式时间内解决的判定问题的集合。

复杂度类NP是指所有能够在多项式时间内验证的问题的集合。

P代表多项式（polynomial），NP代表非确定性多项式（non - deterministic polynomial）。

P类问题

最小生成树（MST）问题属于P类

- 给定一个带权图G和一个数值k，是否存在一棵权重至多为k的最小生成树？
- 运行 Kruskal 算法（该算法是多项式时间算法），如果找到的最小生成树权重至多为k，那么答案是“是”。
- Kruskal 算法的时间复杂度为 $O(E \log E)$ 。

单源最短路径问题属于P类

- 给定一个带权图G、一个源顶点s和一个数值k，是否存在从s到其他每个顶点的最短路径，且路径长度至多为k？
- 运行Dijkstra 算法（该算法是多项式时间算法），如果找到的路径长度至多为k，那么答案是“是”。

NP类问题

哈密顿回路问题属于NP类

- 我们能够在多项式时间内检查一个给定的回路是否为哈密顿回路。

0/1背包问题属于NP类

- 我们能够在多项式时间内检查一个给定的物品子集，其重量是否至多为W且价值是否至少为k。

电路可满足性问题（Circuit - SAT）属于NP类

- 我们能够在多项式时间内检查给定的一组值是否能使最终输出为1。

P = NP ?

注意，P是NP的子集。 $P \subseteq NP$

这个（价值百万美元的）问题在于，数学家和计算机科学家们尚不清楚P是否等于NP。

然而，人们普遍认为P和NP是不同的。

- 也就是说，存在某些属于NP但不属于P的问题。

数独

假设解决一个 $n \times n \times n \times n$ 数独问题需要 $S(n)$ 时间，验证其解需要 $V(n)$ 时间。

事实：验证所需时间 $V(n) = O(n^2 \times n^2)$

问题：是否存在某个常数，使得解决时间 $S(n) \leq n^{\text{常数}}$ ？

P与NP问题等同于：对于 $n \times n \times n \times n$ 的数独问题，是否存在一种算法，其运行时间为关于 n 的某个多项式 $p(n)$ ？

只要是 $O(n^{\text{常数}})$ 级别的都算作高效算法，都是多项式时间（复杂度）。

但是只要涉及到 n 在指数，就不是多项式时间，也不是高效算法了。

例如 $O(n^{\log n})$ 或者 $O(2^n)$ 或者 $O(n!)$

多项式时间归约（证明难度）

给定任意两个判定问题A和B，我们称：

(1) A可在多项式时间内归约（reduction）到B，或者

(2) 存在从A到B的多项式时间归约，

是指：对于A的任意输入 α ，我们能够在多项式时间内构造出B的一个输入 β ，使得 α 的答案为“是”当且仅当 β 的答案为“是”。

我们用符号 $A \leq_p B$ 表示。

直观地说，这意味着问题B至少和问题A一样难。

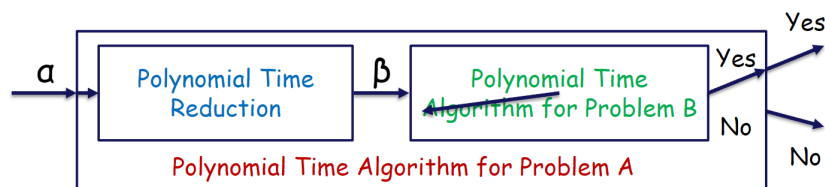
多项式时间规约问题例子

哈密顿回路问题（A）

问题描述：给定一个图 $G = (V, E)$ ，是否存在一条回路，能够恰好访问每个顶点一次，然后回到起始顶点？（判定问题）

旅行商问题（B）

问题描述：给定一组城市（顶点）、每对城市之间的距离（带权边），以及总距离上限 k ，是否存在一条路线，能够恰好访问每个城市一次，然后回到起始城市，且总行程距离小于或等于 k ？（判定问题）

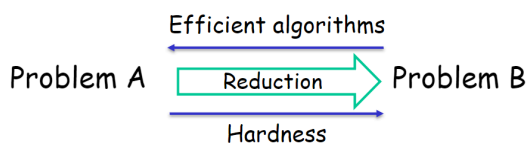


如果问题A能在多项式时间内归约到问题B，那么哪个问题更容易？是A还是B？

答案是A。因为问题B至少和问题A一样难！

使用归约的两种方式

归约：Reduction



假设问题A可归约到问题B：

- **解决问题**：如果存在解决问题B的**高效算法**，那么我们就能够高效地解决问题A。
- **证明难度 (Hardness)**：如果问题A很难，那么问题B也很难。

NP难问题(NP-hardness)

如果NP中的任意其他问题都能在多项式时间内**归约到问题M**，那么问题M被称为**NP难问题(NP-Hard)**。

- 直观来讲，这意味着**M至少和NP中的所有问题一样难**。

如果问题M满足以下条件，则进一步被称为NP完全问题：

1. M属于NP类；
2. M是NP难问题。

NP完全问题(NP-complete problem)是NP中最难的一些问题。

NP完全问题(NP-complete problem, NPC)

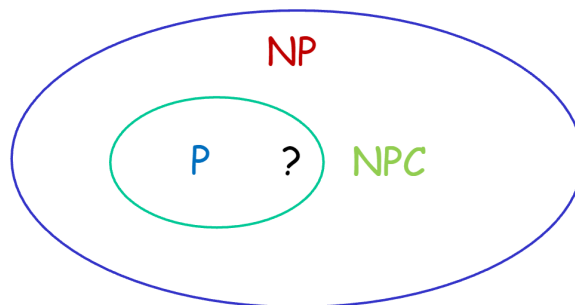
如果问题A满足以下条件，则它是**NP完全问题**：

1. 问题A属于NP类。
2. 对于NP中的任意问题A'，A' 能在多项式时间内**归约到A**。

NPC类：所有NP完全问题构成的类，它是NP类的一个子类。

- 是NP中最难的问题。
- 解决NPC类中的一个问题，就能解决NP中的所有问题。

如果NP完全问题（NPC）与P类的交集不为空，那么 $NP = P$ 。



第一个被证明的NP完全问题

库克 - 勒维定理（Cook-Levin Theorem）表明，电路可满足性问题（Circuit-SAT）是NP完全问题（NPC）（第一个被证明的NP完全问题）。

利用多项式时间归约性，我们能够证明其他NP完全问题的存在。

一个用于证明NP完全性的有用结论：

引理

如果 $L1 \leq_p L2$ 且 $L2 \leq_p L3$ ，那么 $L1 \leq_p L3$ 。

NP完全性的证明

给定问题A，证明A是NP完全问题：

证明方案1

- 证明问题A属于NP（相对简单的部分）。
- 对于NP中的所有问题，在多项式时间内将它们归约到问题A。
 - 对于3SAT问题（第一个被证明的NP完全问题），已经完成了这样的证明。

证明方案2

- 证明问题A属于NP（相对简单的部分）。
- 对于NP完全问题类（NPC）中的任意问题A'，在多项式时间内将A'归约到问题A。
 - 这种方法会简单得多。

其他NPC问题

我们已经了解过这些NP完全问题：

- 哈密顿回路问题
- 0/1背包问题
- 电路可满足性问题（Circuit - SAT）

其他NP完全问题：

- 合取范式可满足性问题（CNF - SAT）和3 - SAT问题（合取范式可满足性问题的一种）
- 顶点覆盖问题（Vertex-Cover）

合取范式（CNF）

- 一个布尔公式如果是由多个子句通过“与”(·) 运算符组合而成，且每个子句是由文字（变量或其否定形式）通过“或”(+) 运算符组合而成，那么该布尔公式就是合取范式。
- 示例： $(x_1 + x_2 + \neg x_4 + x_5) \cdot (x_2 + x_1 + x_4)$

合取范式可满足性问题（CNF - SAT）与3 - 可满足性问题（3 - SAT）

合取范式可满足性问题（CNF - SAT）

- **输入：**一个合取范式（CNF）的布尔公式。
- **问题：**是否存在对其变量的布尔值赋值，使得该公式的计算结果为真？（即该公式是可满足的）

3 - 可满足性问题（3 - SAT）

- **输入：**一个合取范式（CNF）的布尔公式，且每个子句恰好有3个文字。

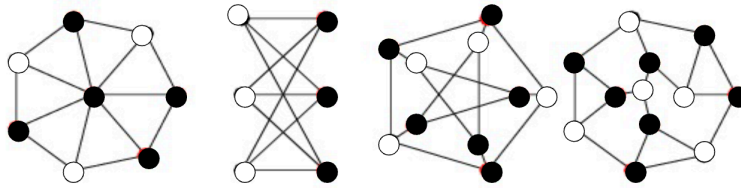
CNF - SAT和3 - SAT都是NP完全问题

顶点覆盖（Vertex Cover）

给定一个图 $G = (V, E)$ 。

一个顶点覆盖是顶点集 V 的一个子集 $C \subseteq V$ ，使得对于边集 E 中的任意一条边 (v, w) ，都有 $v \in C$ 或者 $w \in C$ 。

（下方展示了一些图及其顶点覆盖，黑色顶点为顶点覆盖中的顶点）



优化问题是：要找到尽可能小的顶点覆盖。

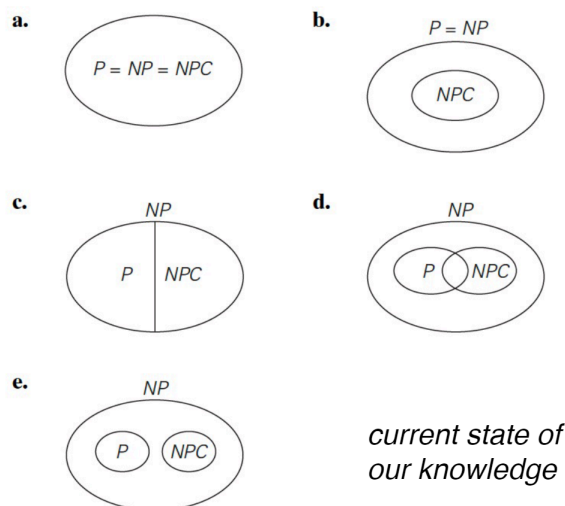
顶点覆盖问题是一个判定问题，它以图 G 和整数 k 作为输入，询问是否存在一个顶点覆盖，使得 G 中最多包含 k 个顶点。

顶点覆盖问题是NP完全问题。

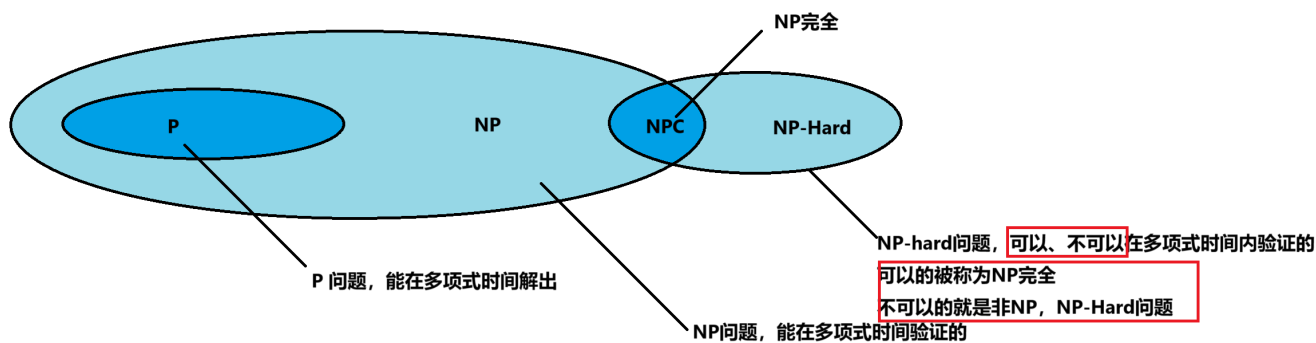
[11] 需要了解的内容.....

- P和NP的定义
- NP完全（NPC）和NP难（NP - hard）的定义
- NP完全问题的例子
- P、NP、NPC、NP - hard

如果能在多项式时间内解决其中任意一个问题，就能在多项式时间内解决所有这些问题



1. **图a**：此图表明P、NP和NPC都是等价的，这意味着每个P问题都是NP完全问题，且每个NP问题都属于P。这与当前的认知相悖，在没有确凿证据的情况下具有很强的推测性。该图与已知的复杂度理论相矛盾。
2. **图b**：在此图中，P等于NP，但NP完全（NPC）问题构成了NP（因而也是P）的一个真子集。这意味着所有NP问题，包括NP完全问题，都能在多项式时间内解决。虽然此图确实反映了 $P = NP$ 的一种情形，但它表明并非所有NP问题都是NP完全问题，若 $P = NP$ ，这是准确的。该图并不与已知理论相矛盾，尽管它描绘了复杂度理论中一个尚未解决的情形。
3. **图c**：此图将P和NP表示为不相交的集合，这是不正确的。我们知道所有P中的问题也都在NP中（P是NP的一个子集）。该图与已知的复杂度理论相矛盾。
4. **图d**：此图显示P是NP的一个子集，NPC是NP的一个子集，且P和NPC有交集。这意味着一些P问题是NP完全问题，这与NP完全性的定义相矛盾（NP完全问题是NP中最难的问题，若其中任何一个能在多项式时间内解决，那么 $P = NP$ ）。该图与已知的复杂度理论相矛盾。
5. **图e**：此图正确地显示P是NP的一个子集，NP完全问题也是NP的一个子集。P和NPC没有重叠，意味着除非 $P = NP$ ，否则P中没有问题是NP完全问题，这与目前的认知一致，因为我们还不知道P是否等于NP。该图不与已知的复杂度理论相矛盾。



1. 排序问题是P问题, 因为可以在多项式时间解决
2. 排序问题也是NP问题, 因为可以在多项式 (甚至常数) 时间内验证
3. 哈密顿回路是NP问题, 因为解决需要 $O(n!)$ 复杂度, 但是验证则可以在常数时间内解决
4. 旅行商问题是NP-Hard问题, 就算给出一条路径, 要判断是否是最优路径, 仍然要验证其他路径, 因此验证并非多项式时间
5. 旅行商问题加上条件限制 (例如要求路径长度 $<k$ 即可) 那么这个问题就是NP问题, 因为其验证是多项式时间, 同时也是NP-Hard问题, 因为找出解可能需要 $O(n!)$ 复杂度, 因此, 介于两者之间, 属于NPC问题
6. 哈密顿回路是NP问题, 它可以归约 (Reduction) 到旅行商加了限制后的问题 ($<k$), 也就是说这个NP可以被归约到NPC

P问题: 常见的问题例如排序和查找

NP但不是NPC问题: (没被证明是NPC的NP问题)

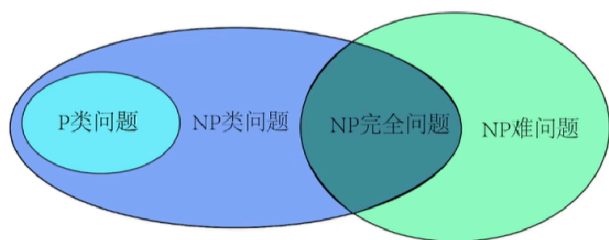
1. 因式分解问题 Factorization problem

NPC 问题:

1. 合取范式 (CNF)
2. 顶点覆盖 (Vertex Cover)
3. 0/1 背包问题
4. 子集和问题 (Subset sum problem)
5. 哈密顿回路
6. SAT问题 (Circuit-SAT)
7. 带限制的旅行商问题(权重 $<k$)

NP-Hard 且不是NPC问题:

1. 停机问题
2. n-皇后问题
3. 旅行商问题 (找出最短路)



■ [Week 11 Lecture + Tutorial]

[1] 应对困难的组合问题

应对困难的组合问题（NP难问题）主要有两种方法：

- 采用一种能保证精确求解问题，但不保证在多项式时间内找到解的策略。
- 采用近似算法，该算法能在多项式时间内找到一个近似（次优）解。

[2] 精确求解策略

穷举搜索（暴力法 Brute Force）

- 仅适用于小规模实例。

回溯法 Backtracking

- 排除一些不必要的情况。
- 对许多实例能在合理时间内得出解，但最坏情况时间复杂度仍为指数级。

分支限界法 Branch-and-bound

- 针对优化问题，进一步细化回溯法的思路。

动态规划 DP

- 适用于某些问题（如背包问题）。

[3] 算法设计技术

- **贪心算法 Greedy**
 - 用于最短路径、最小生成树等问题。
- **分治法 Divide and conquer**
 - 将问题分解为较小的子问题，求解子问题，再合并得到整体解。
 - 常采用递归方式。
 - 快速排序、归并排序是典型例子。
- **动态规划**
 - 穷举所有可能解，存储子问题的解以避免重复计算。
- **回溯法**
 - 一种巧妙的穷举搜索方式。

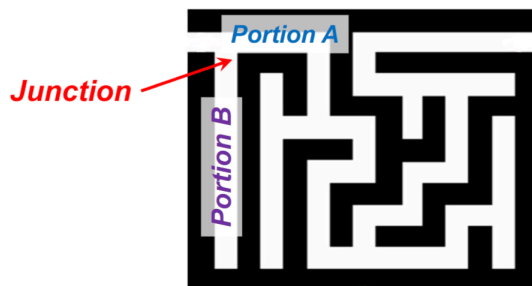
[4] 回溯法

Backtracking

深度优先搜索会停止探索以那些无法得出解的节点，然后回溯到该节点的父节点，继续进行搜索

回溯法是一种用于解决搜索空间较大问题的技术，通过系统地尝试并排除各种可能性来求解。

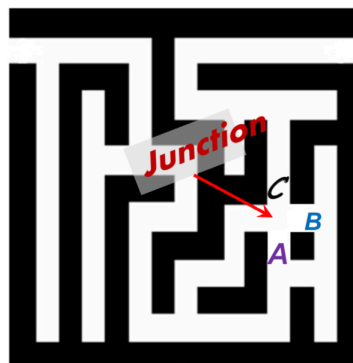
回溯法的一个经典例子是走迷宫：在某些位置，你可能会面临两个方向的选择：



一种策略是**尝试走过迷宫的A部分**。如果你在找到出口之前陷入死胡同，那么你就“回溯”到岔路口。

此时你知道A部分无法带你走出迷宫，所以你接着开始探索B部分。

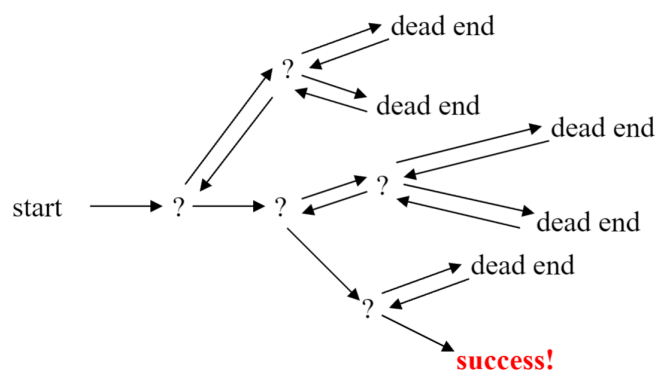
显然，在一个岔路口，你可能有不止两种选择。



回溯策略是依次尝试每一种选择：

- 如果你陷入困境，就“回溯”到岔路口，然后尝试下一个选择。
- 如果你尝试了所有选择，仍未找到出路，那么这个迷宫就没有解。

大概就像...



也就是**搜索与回溯**

n皇后问题

也就是八皇后问题，但是可以拓展为n皇后问题。

在一个 $n \times n$ 的棋盘上放置 n 个皇后，使得任意两个皇后都不在同一行、同一列或同一对角线上。

查看 [Leetcode题目](#)(PPT上给出的链接，可以自己试着解一下)：

51. N 皇后

困难

🔖 相关标签

🏢 相关企业

Ax

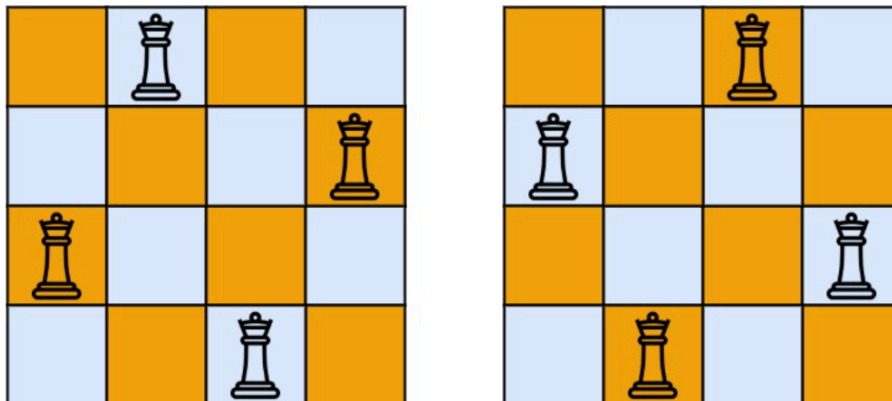
按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。

n 皇后问题 研究的是如何将 n 个皇后放置在 $n \times n$ 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 n ，返回所有不同的 **n 皇后问题** 的解决方案。

每一种解法包含一个不同的 **n 皇后问题** 的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例 1:



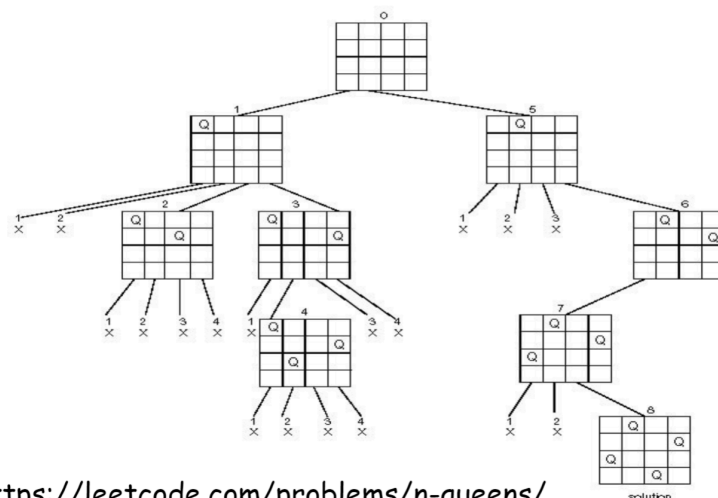
输入: $n = 4$

输出: `[[".Q..", "...Q", "Q...", "..Q."], ["....Q.", "Q...", "...Q.", ".Q.."]]`

解释: 如上图所示, 4 皇后问题存在两个不同的解法。

具体实现解法在 [Algorithm.pdf](#) 中

实际上, 就是一个搜索树, 当能搜索到树的末尾的时候, **就是一个解**, 否则, 这个节点下的所有子节点都不用再管了。



<https://leetcode.com/problems/n-queens/>

这张图就很好地展现了, 一个状态就是一个节点! 当某个状态不合法, 那么就不需要再管子节点了!

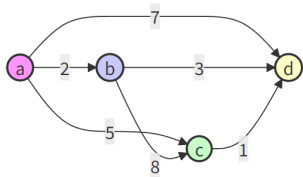
时间复杂度 $O(N!)$

哈密顿回路（旅行商问题）

还记得之前讲的哈密顿回路（旅行商问题）吗？

旅行商问题

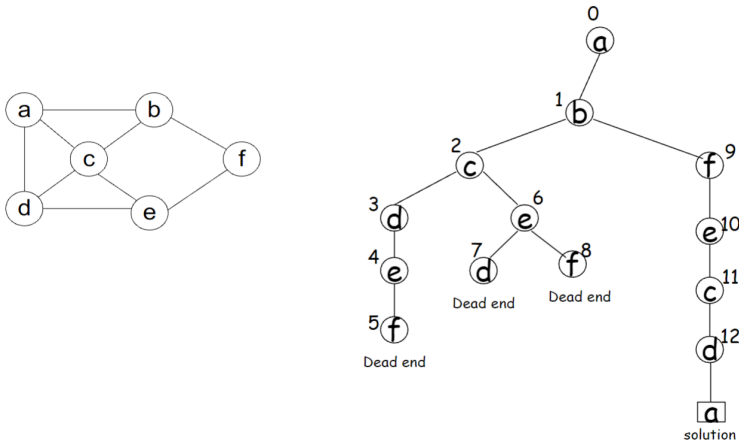
输入：共有 n 个城市。
输出：找到从某一特定城市出发的最短路径，要求恰好访问每个城市一次，并最终返回到起点城市。
这个问题被称为**哈密顿回路**（Hamiltonian Circuit）。



路径	计算方式	总距离
a -> b -> c -> d -> a	2 + 8 + 1 + 7	18
a -> b -> d -> c -> a	2 + 3 + 1 + 5	11
a -> c -> b -> d -> a	5 + 8 + 3 + 7	23
a -> c -> d -> b -> a	5 + 1 + 3 + 2	11
a -> d -> b -> c -> a	7 + 3 + 8 + 5	23
a -> d -> c -> b -> a	7 + 1 + 8 + 2	18

它的时间复杂度为 $O((n - 1)!)$ 也是和n皇后差不多的复杂度。

问题：找出一条经过图中每个顶点恰好一次，最后回到起始顶点的回路。



比如这个图，图中a只画出了一个分支（还有一个d分支没画出来）。

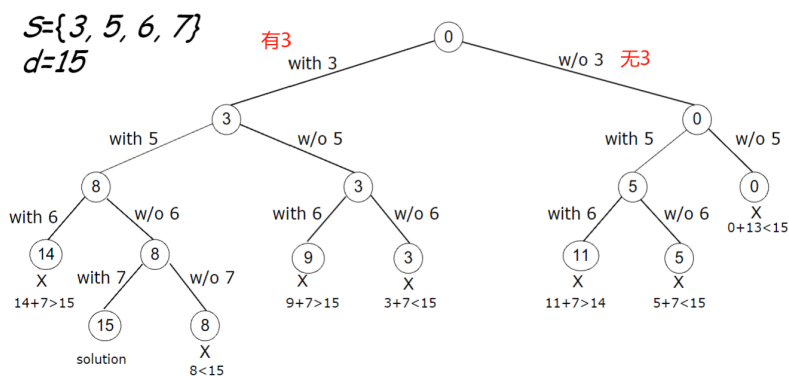
子集和问题

Subset-sum problem

给定一个由 n 个正整数组成的集合 $S = \{s_1, \dots, s_n\}$ ，找出该集合的一个子集，使得子集中元素的和等于给定的正整数 d 。

例如： $S = \{3, 5, 6, 7\}$ ， $d = 15$

- 什么是状态空间？
- 状态之间的关系是怎样的？



通常，回溯法适用于那些难以找到精确解的高效算法的困难组合问题。

与穷举搜索方法不同，穷举搜索对于问题的所有实例都注定极其缓慢，而回溯法至少有望在可接受的时间内解决一些规模不算小的实例。

简而言之就是，通过减去子树这种操作省时间！

[5] 分支限界法

是回溯法的一种增强，适用于优化问题。

对于状态空间树的每个节点（部分解），计算该节点所有子节点（部分解的扩展）目标函数值的边界。

边界的用途：

- 将某些节点判定为“无前景”节点从而对树进行剪枝 —— 如果一个节点的边界值不比目前找到的最佳解更优。
- 引导在状态空间中的搜索。

分配问题

Assignment Problem

在成本矩阵(Cost matrix) C 的每一行中选择一个元素，需满足：

- 任意两个所选元素不在同一列
- 所选元素的总和最小

其实就是雇佣人干事情，一个人只能干一件事，有4件事和4个人，怎么让成本最低。

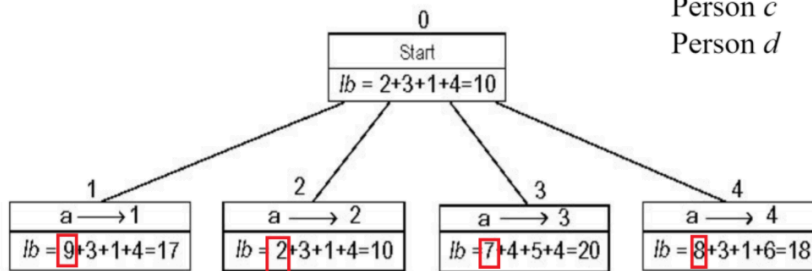
示例

	Job 1	Job 2	Job 3	Job 4
Person a	9	2	7	8
Person b	6	4	3	7
Person c	5	8	1	8
Person d	7	6	9	4

下界：此问题的任何解的总成本至少为：2 + 3 + 1 + 4（取每行的最小值）（或5 + 2 + 1 + 4，取每列的最小值）

虽然这个下界是不可能实现的因为Job3有两个人一起干了。但是我们仍然可以把这个下界作为一个指标。

	Job 1	Job 2	Job 3	Job 4
Person <i>a</i>	9	2	7	8
Person <i>b</i>	6	4	3	7
Person <i>c</i>	5	8	1	8
Person <i>d</i>	7	6	9	4



会排除的！当选择9的时候，下面的节点不再允许选择同样的column

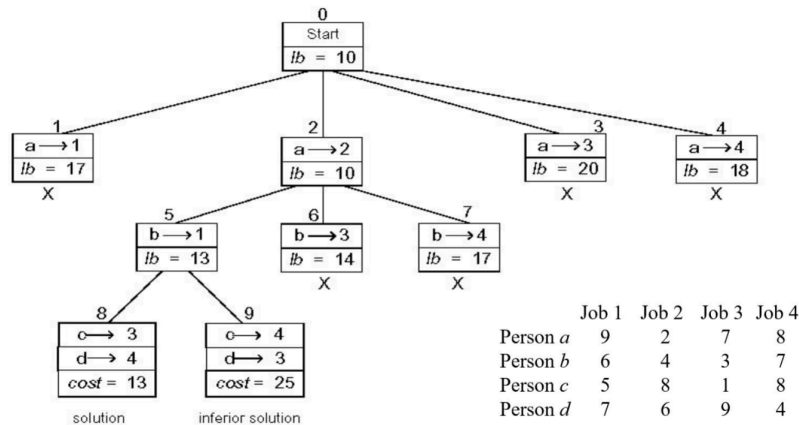
	Job 1	Job 2	Job 3	Job 4
Person <i>a</i>	9	2	7	8
Person <i>b</i>	6	4	3	7
Person <i>c</i>	5	8	1	8
Person <i>d</i>	7	6	9	4

	Job 1	Job 2	Job 3	Job 4
Person <i>a</i>	9	2	7	8
Person <i>b</i>	6	4	3	7
Person <i>c</i>	5	8	1	8
Person <i>d</i>	7	6	9	4

	Job 1	Job 2	Job 3	Job 4
Person <i>a</i>	9	2	7	8
Person <i>b</i>	6	4	3	7
Person <i>c</i>	5	8	1	8
Person <i>d</i>	7	6	9	4

	Job 1	Job 2	Job 3	Job 4
Person <i>a</i>	9	2	7	8
Person <i>b</i>	6	4	3	7
Person <i>c</i>	5	8	1	8
Person <i>d</i>	7	6	9	4

第一层，我们决定让Person a 参加哪个工作。我们选取lb（Lower Bound）最低的来进行讨论



然后一层层往下，淘汰那些不是最优的节点。

可能的疑惑：

如果最开始lb=10，让a选1、2、3、4这四个工作的lb分别是

17、10、11、18. 固然选10，但是会不会像贪心算法那样陷入局部最优解呢？

答案是不会的。因为最初lb就是全局最最低的lb了，因为最开始的lb=10是包含重复的，因此，这种贪心的算法是合理的。不会陷入局部最优解而错过全局最优解。

进一步说，就是就算我们选择了a->2 也就是lb = 10，那么接下来让b选择的时候只可能大于第一层的lb=11而不可能更小！

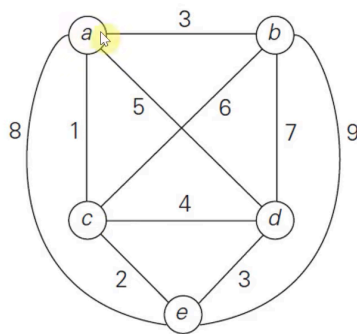
旅行商优化问题

这个视频和课件上的例子一样：

[Travelling salesman problem | Branch and bound | Scholarly things - YouTube](#)

课件讲的很烂，B站上的讲的也很烂。这个视频是**唯一**能够把旅行商问题（分支限界法）讲好的

旅行商问题和分配问题一样，最开始的lb是全局的最低最低的值。也就是每个点最低的相邻两条边之和除2，这个也很好理解。



比如在这个图中，lb初始为 $lb = \lceil [(1+3) + (3+6) + (1+2) + (3+4) + (2+3)] \div 2 \rceil = 14$ ，其中 $\lceil \rceil$ 是向上取整符号，表示对中括号内计算结果除以2后的数值向上取整，最终结果为14。当然，因为是全局最低，所以可以重复边，但也显然不是一个环。

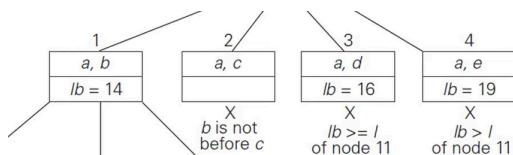
下面一层是选择a和哪个点相连（并且为了简化，添加一个约束，这个约束是：b一定要在c前面。添加这个约束不会使我们的最终答案改变，但是会简化很多步骤）

（这里就不写向上取整了，但是还是需要向上取整的）这一层，选择ab的lb为 $[(1+3)+(3+6)+(1+2)+(3+4)+(2+3)] \div 2 = 14$

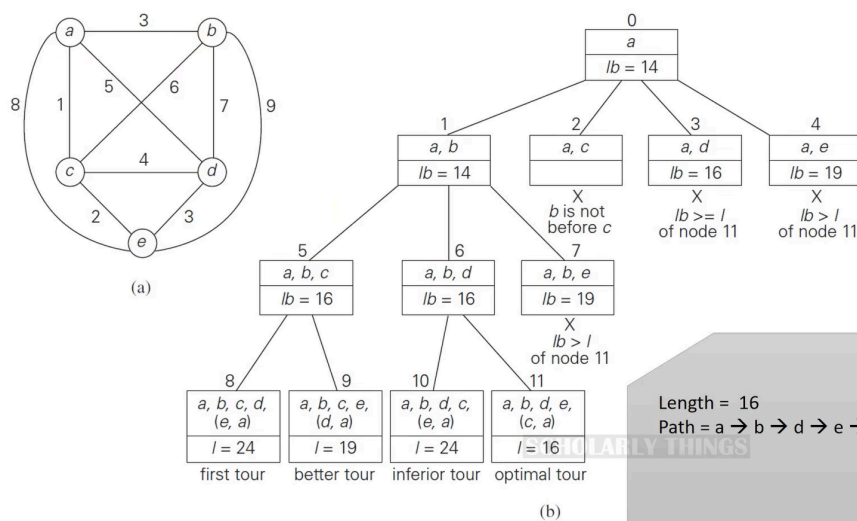
选择ac的lb不存在（因为b在c前面）

选择ad的lb为 $[(1+5)+(3+6)+(1+2)+(3+5)+(2+3)] \div 2 = 15.5$ 向上取整为16

然后ae的lb为 $[(1+8)+(3+6)+(1+2)+(3+4)+(2+8)] \div 2 = 19$



然后下面几层也是如此

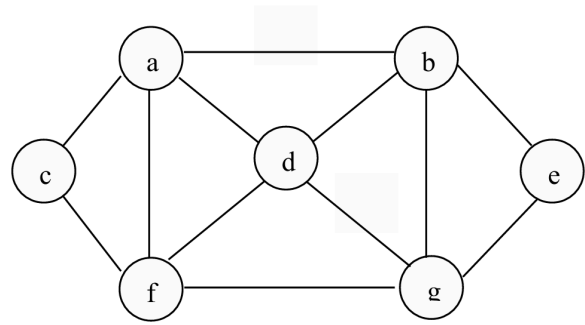


最后就得到了这个图了。

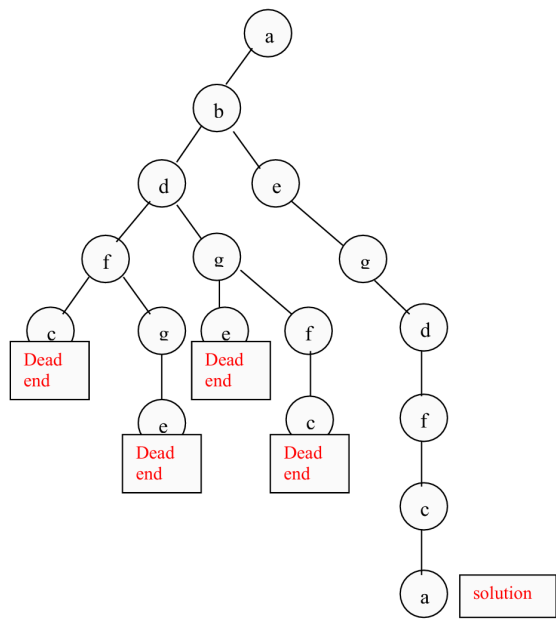
[6] Tutorial

寻找一个哈密顿环的解

Apply backtracking to the problem of finding a Hamiltonian circuit in the following graph (starting from vertex *a*)



Solution:

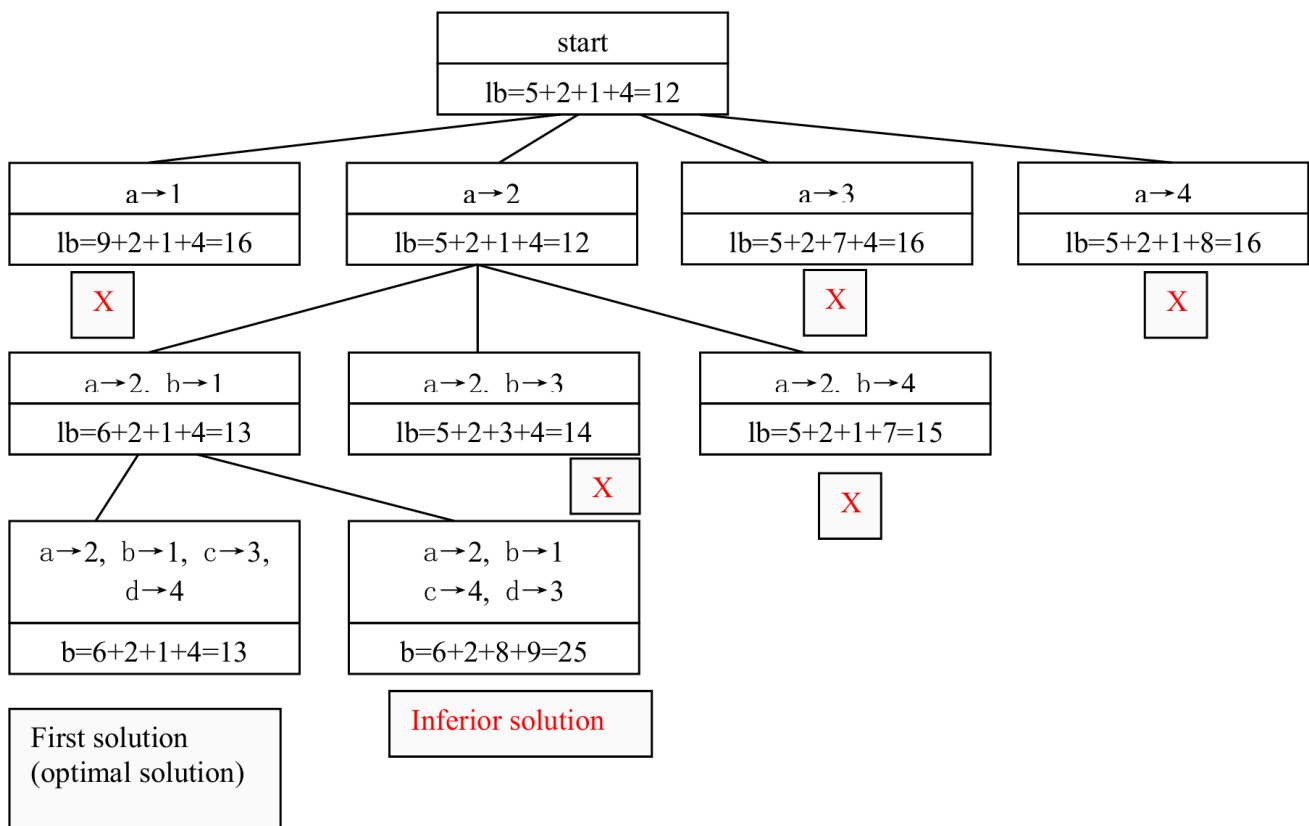


分配问题

Solve the same instance of the assignment problem as the one solved in the section by the best-first branch-and-bound algorithm with the bounding function based on matrix columns rather than rows.

	Job1	Job1	Job1	Job1
Person a	9	2	7	8
Person b	6	4	3	7
Person c	5	8	1	8
Person d	7	6	9	4

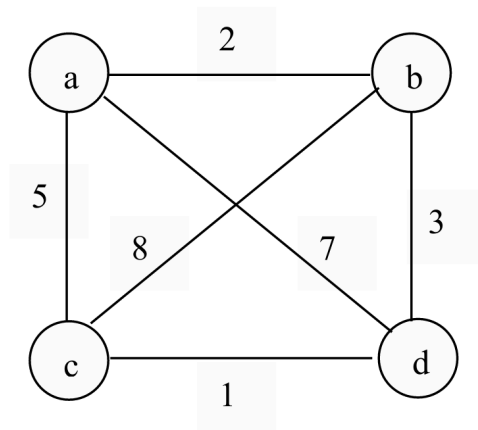
Low bound =



当然除了纵向看也能横向看。

分支限界法解决旅行商问题

Apply the branch-and-bound algorithm to solve the travelling salesman problem for the following graph.



Suggested Solution:

To reduce the complexity, we can assume that, b is before c in the tour. The lower bound for each node can be computed by

Node 0: $lb = \lceil [(2+5) + (2+3) + (1+5) + (1+3)] / 2 \rceil = 11$

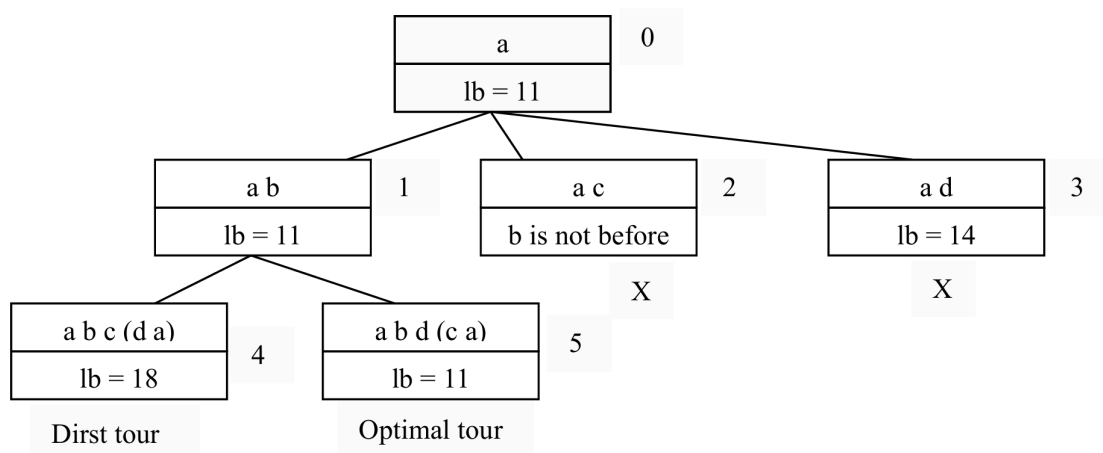
Node 1: $lb = \lceil [(2+5) + (2+3) + (1+5) + (1+3)] / 2 \rceil = 11$

Node 2: ignored as b is not before c.

Node 3: $lb = \lceil [(2+7) + (2+3) + (1+5) + (1+7)] / 2 \rceil = 14$

Node 4 leads to a tour with $lb = 18$

Node 5 leads to a optimal tour with $lb=11$.



Solutions: a b d c a

■ [Week 12 Lecture]

01 背包问题

暴力时间复杂度 $O(2^n)$ 属于NP/NPC问题

这个问题实际上就是，给定一个容量为 V 的背包，和若干重量 w 、价值 p 的物品，问，你这个背包要装什么物品最有价值？

也就是对于每一个物品而言，你要装还是不要装。

其实这个就是一个动态规划问题

$$F(x) = \max \begin{cases} F'(x) & \text{不纳入当前物品} \\ F'(x - w) + v & \text{纳入当前物品} \end{cases}$$

这里的 F' 是前一个状态。在图表中就是上一行。

	0	1	2	3	4	5	6
对于每一个单元格 i.weight > j?							
0	0	0	0	0	0	0	0
1  (2, 3)	0	0	3	3	3	3	3
2  (3, 5)	0	0	3	5	5	8	
3  (4, 6)	0						

例如，此时要填写红色框，因为01背包问题不允许矿泉水用两次（前面3->5用了一次），只能用一次，所以这里的 F' 上一个状态就指的是葡萄那一栏（矿泉水还没进入考虑）。意思是我们要比较：

1. 不装矿泉水 $F'(x)$
2. 装矿泉水（需要3个空间，再加入矿泉水价值5） $F'(x - 3) + 5$

但是如果是完全背包，那么状态转移方程就是

$$F(x) = \max \begin{cases} F(x) & \text{不纳入当前物品} \\ F(x - w) + v & \text{纳入当前物品} \end{cases}$$

也就是不需要再向上一个状态寻找了，因为矿泉水可以用多次。